EXPERT INSIGHT

Early Access

Modern Time Series Forecasting with Python

Industry-ready machine learning and deep learning time series analysis with PyTorch and pandas

Second Edition

Manu Joseph
Jeffrey Tackes

packt

Modern Time Series Forecasting with Python

Early Access Publication: Modern Time Series Forecasting with Python

Early Access Production Reference: B22389

Published by Packt Publishing Ltd.

Livery Place

35 Livery Street

Birmingham

B3 2PB, UK

ISBN: 978-1-83588-318-1

www.packt.com

Table of Contents

Modern Time Series Forecasting with Python, Second Edition: Industry-ready machine learning and deep learning time series analysis with PyTorch and pandas

Welcome to Packt Early Access. We're giving you an exclusive preview of this book before it goes on sale. It can take many months to write a book, but our authors have cutting-edge information to share with you today. Early Access gives you an insight into the latest developments by making chapter drafts available. The chapters may be a little rough around the edges right now, but our authors will update them over time.

You can dip in and out of this book or follow along from start to finish; Early Access is designed to be flexible. We hope you enjoy getting to know more about the process of writing a Packt book.

1 Introducing Time Series

Join our book community on Discord

https://packt.link/EarlyAccess/

Welcome to *Modern Time Series Forecasting with Python*! This book is intended for data scientists or **machine learning** (**ML**) engineers who want to level up their time series analysis skills by learning new and advanced techniques from the ML world. **Time series analysis** is something that is commonly overlooked in regular ML books, courses, and so on. They typically start with classification, touch upon regression, and then move on. But it is also something that is immensely valuable and ubiquitous in business. We look at the world in a three-dimensional perspective. But time is the hidden dimension which we rarely think about, but is all-pervasive. And as long as time is one of the four dimensions in the world we live in, time series data is all-pervasive.

Analyzing time series data unlocks a lot of value for a business. Time series analysis isn't new—it's been around since the 1920s. But in the current age

of data, the time series that are collected by businesses are growing larger and wider by the minute. Combined with an explosion in the quantum of data collected and the renewed interest in ML, the landscape of time series analysis also changed considerably. This book attempts to take you beyond classical statistical methods such as **AutoRegressive Integrated Moving Average** (**ARIMA**) and introduce to you the latest techniques from the ML world in time series analysis.

We are going to start with some fundamental concepts and quickly scale up to more complex topics. In this chapter, we're going to cover the following main topics:

- What is a time series?
- Data-generating process (DGP)
- What can we forecast?
- Forecasting terminology and notation

Technical requirements

You will need to set up the **Anaconda** environment following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks.

The associated code for the chapter can be found at https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-/tree/main/notebooks/Chapter01.

What is a time series?

To keep it simple, a **time series** is a set of observations taken sequentially in time. The focus is on the word *time*. If we keep taking the same observation at different points in time, we will get a time series. For example, if you keep recording the number of bars of chocolate you have in a month, you'll end up with a time series of your chocolate consumption. Suppose you are recording your weight at the beginning of every month. You get another time series of your weight. Is there any relation between the two time series? Most likely, yeah. But we can analyze that scientifically by the end of this book.

A few other examples of time series are the weekly closing price of a stock that you follow, daily rainfall or snow in your city, or hourly readings of your heartbeat from your smartwatch.

Types of time series

There are two types of time series data based on time-intervals, as outlined here:

Regular time series: This is the most common type of time series where we have observations coming in at regular intervals of time, such as every hour or every month. For example, if we take a time series of temperature in a cuty, we will get the time series in a regular interval (whichever frequency we chose for observation).

Irregular time series: There are a few time series where we do not have observations at a regular interval of time. For example, consider we have a sequence of readings from lab tests of a patient. We see an observation in the time series only when the patient heads to the clinic and carries out the lab test, and this may not happen in regular intervals of time.

NOTE

> *This book only focuses on regular time series, which are evenly spaced in time. Irregular time series are slightly more advanced and require specialized techniques to handle them. A couple of survey papers on the topic is a good way to get started on irregular time series and you can find them in the Further reading section of this chapter.*

Main areas of application for time series analysis

There are broadly three important areas of application for time series analysis, outlined as follows:

Time series forecasting: Predicting the future values of a time series, given the past values—for example, predict the next day's temperature using the last 5 years of temperature data. This use-case is one of the most popular and important because any kind of planning we need to do needs some visibility into the future. For instance, planning how many chocolates to produce next month needs a forecast of expected demand.

Time series classification: Sometimes, instead of predicting the future value of the time series, we may also want to predict an action based on past values. For example, given a history of an **electroencephalogram** (**EEG**; tracking electrical activity in the brain) or an **electrocardiogram** (**EKG**; tracking electrical activity in the heart), we need to predict whether the result of an EEG or an EKG is normal or abnormal.

- **Outlier Detection**: There are some situations where we only want to detect if something is going wrong or if something is out of the ordinary. In such cases, we need to use classification or forecasting, but instead we can do outlier detection. For instance, the wearable tech on your body records the accelerometer readings across time and can use outlier detection to identify falls or accidents.

Interpretation and causality: Understand the whats and whys of the time series based on the past values, understand the interrelationships among several related time series, or derive causal inference based on time series data. For example, we have a time series of market share for a brand and

also another time series of advertising spends. Using interpretation and causality techniques, we can start to understand how much advertising spends are affecting the market share and possibly take appropriate actions around it.

NOTE

> *The focus of this book is predominantly on time series forecasting, but the techniques that you learn will help you approach time series classification problems also, with minimal change in the approach. Interpretation is also addressed, although only briefly, but causality is an area that this book does not address because it warrants a whole different approach.*

Now that we have an overview of the time series landscape, let's build a mental model on how time series data is generated.

Data-generating process (DGP)

We saw that time series data is a collection of observations made sequentially along the time dimension. Any time series is, in turn, generated by some kind of *mechanism*. For example, time series data of daily shipments of your favorite chocolate from the manufacturing plant is affected by a lot of factors such as the time of the year, the holiday season,

the availability of cocoa, the uptime of the machines working on the plant, and so on. In statistics, this underlying process that generated the time series is referred to as the **DGP**. Time series data is produced by stochastic and deterministic process. The deterministic processes involve quantities that evolve in a predictable manner over time. An example of this is the radioactive decay of an element, where the remaining quantity diminishes according to a precise mathematical formula, leading to a consistent reduction over time. But most of the interesting time series (from a forecasting perspective) are generated by a stochastic process. A stochastic process is a way to describe how things change over time in a random but somewhat predictable manner, like how the weather changes daily with some patterns and probabilities involved. So, let's discuss more about time series generated from stochastic processes.

If we had complete and perfect knowledge of reality, all we must do is put this DGP together in a mathematical form and you will get the most accurate forecast possible. But sadly, nobody has complete and perfect knowledge of reality. So, what we try to do is approximate the DGP, mathematically, as much as possible so that our imitation of the DGP gives us the best possible forecast (or any other output we want from the analysis). This imitation is called a **model** that provides a useful approximation to the DGP.

But we must remember that the model is not the DGP, but a representation of some essential aspects of reality. For example, let's consider an aerial

view of Bengaluru and a map of Bengaluru, as represented here:

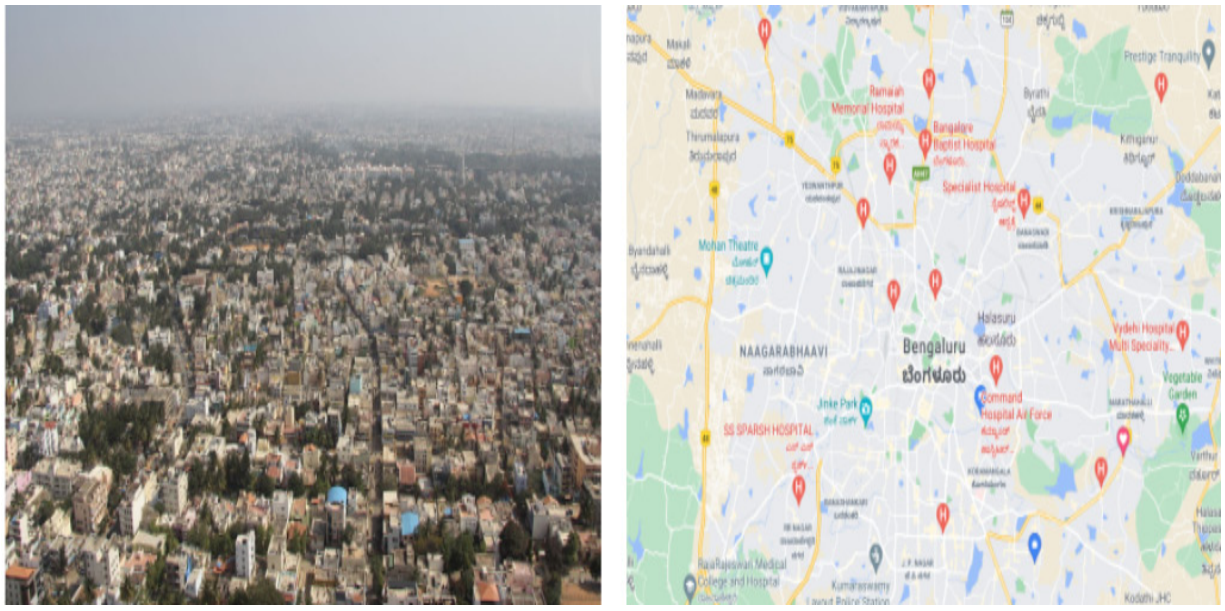


Figure 1.1 – An aerial view of Bengaluru (left) and a map of Bengaluru (right) (Color)

The map of Bengaluru is certainly useful—we can use it to go from point A to point B. But a map of Bengaluru is not the same as a photo of Bengaluru. It doesn't showcase the bustling nightlife or the insufferable traffic. A map is just a model that represents some useful features of a location, such as roads and places. The following diagram might help us internalize the concept and remember it:

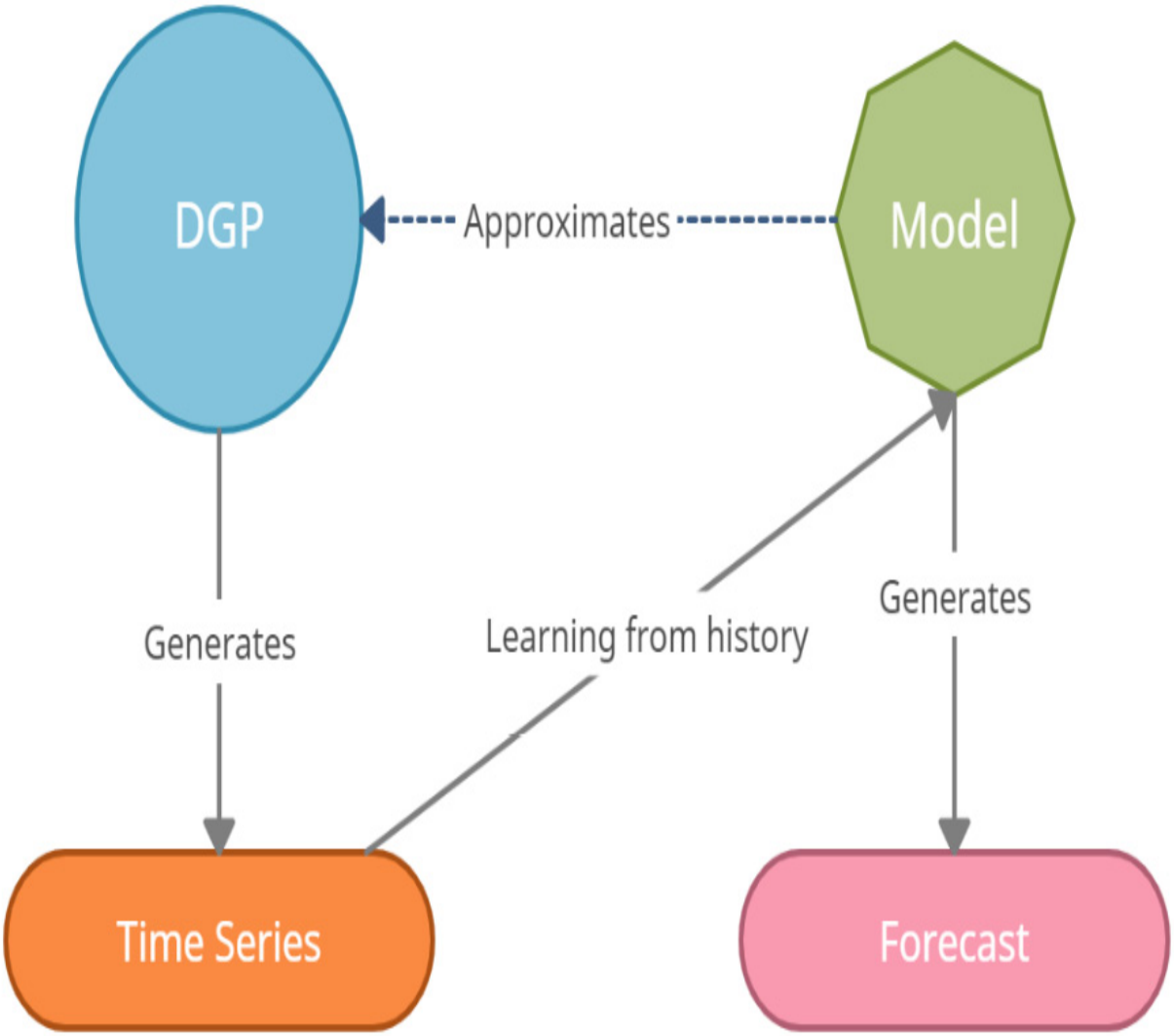


Figure 1.2 – DGP, model, and time series (Color)

Naturally, the next question would be this: *Do we have a useful model?* Every model has it's own limitations and challenges. As we saw already, a map of Bengaluru does not perfectly represent Bengaluru. But if our purpose is to navigate Bengaluru, then a map is a very useful model. What if we want to understand the culture? A map doesn't give you a flavor of

that. So, now, the same model that was useful is utterly useless in the new context.

Different kinds of models are required in different situations and for different objectives. For example, the best model for forecasting may not be the same as the best model to make a causal inference.

We can use the concept of DGPs to generate multiple synthetic time series, of varying degrees of complexity.

Generating synthetic time series

Synthetic time series, or artificial time series, are excellent tools which which you can understand the time series space, experiment with different ways, and even use as ways to test new models or modelling setup. These time series are designed to be predictable, even though a bit challenging. Let's take a look at a few practical examples where we can generate a few time series using a set of fundamental building blocks. You can get creative and mix and match any of these components, or even add them together to generate a time series of arbitrary complexity.

White and red noise

An extreme case of a stochastic process that generates a time series is a **white noise** process. It has a sequence of random numbers with zero mean

and constant variance. This is also one of the most popular assumptions of noise in a time series.

Let's see how we can generate such a time series and plot it, as follows:

```
# Generate the time axis with sequential numbers
time = np.arange(200)
# Sample 200 hundred random values
values = np.random.randn(200)*100
plot_time_series(time, values, "White Noise")
```

Here is the output:

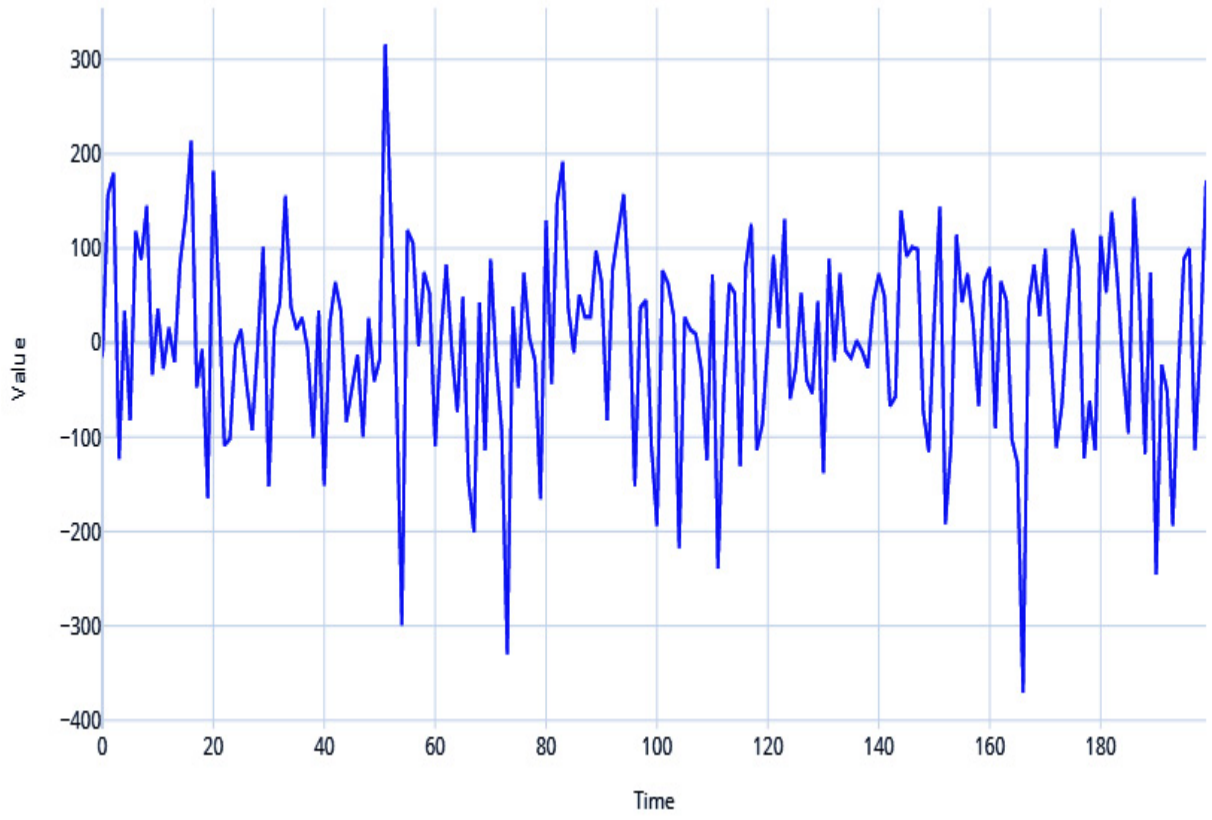


Figure 1.3 – White noise process

Red noise, on the other hand, has zero mean and constant variance but is serially correlated in time. This serial correlation or redness is parameterized by a correlation coefficient *r*, such that:

$$x_{j+1} = \mathrm{r} \cdot x_j + (1 - r^2)^{\frac{1}{2}} \cdot w$$

where *w* is a random sample from a white noise distribution.

Let's see how we can generate that, as follows:

```
# Setting the correlation coefficient
r = 0.4
# Generate the time axis
time = np.arange(200)
# Generate white noise
white_noise = np.random.randn(200)*100
# Create Red Noise by introducing correlation bet
values = np.zeros(200)
for i, v in enumerate(white_noise):
    if i==0:
        values[i] = v
    else:
        values[i] = r*values[i-1]+ np.sqrt((1-np
plot_time_series(time, values, "Red Noise Process
```

Here is the output:

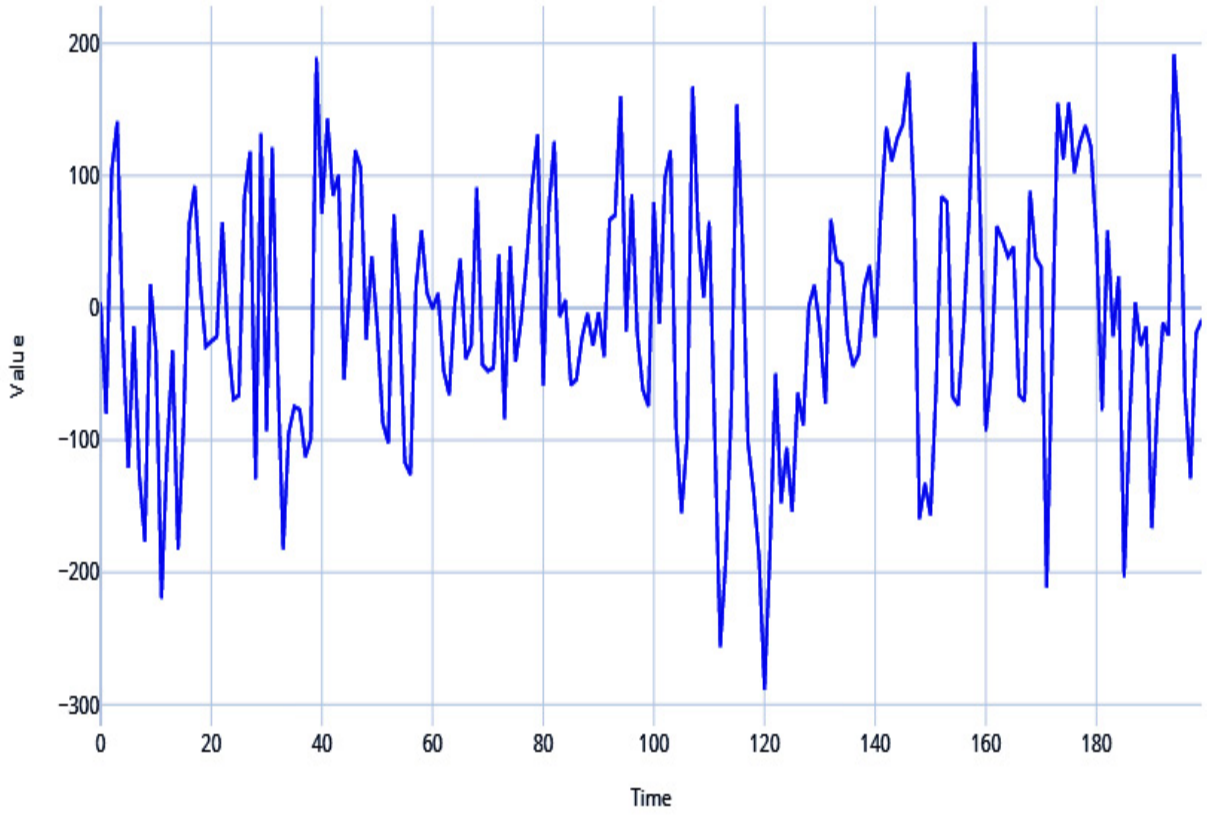


Figure 1.4 – Red noise process

Cyclical or seasonal signals

Among the most common signals you see in time series are seasonal or cyclical signals. Therefore, you can introduce seasonality into your generated series in a few ways.

Let's take the help of a very useful library to generate the rest of the time series—**TimeSynth**. For more information, refer to https://github.com/TimeSynth/TimeSynth.

This is a useful library to generate time series. It has all kinds of DGPs that you can mix and match and create authentic synthetic time series.

NOTE

For the exact code and usage, please refer to the associated Jupyter notebooks.

Let's see how we can use a sinusoidal function to create cyclicity. There is a helpful function in **TimeSynth** called **generate_timeseries** that helps us combine signals and generate time series. Have a look at the following code snippet:

```
#Sinusoidal Signal with Amplitude=1.5 & Frequency
signal_1 =ts.signals.Sinusoidal(amplitude=1.5, fr
#Sinusoidal Signal with Amplitude=1 & Frequency=0
signal_2 = ts.signals.Sinusoidal(amplitude=1, fre
#Generating the time series
samples_1, regular_time_samples, signals_1, error
samples_2, regular_time_samples, signals_2, error
plot_time_series(regular_time_samples,
                 [samples_1, samples_2],
                 "Sinusoidal Waves",
                 legends=["Amplitude = 1.5 | Freq
```

Here is the output:

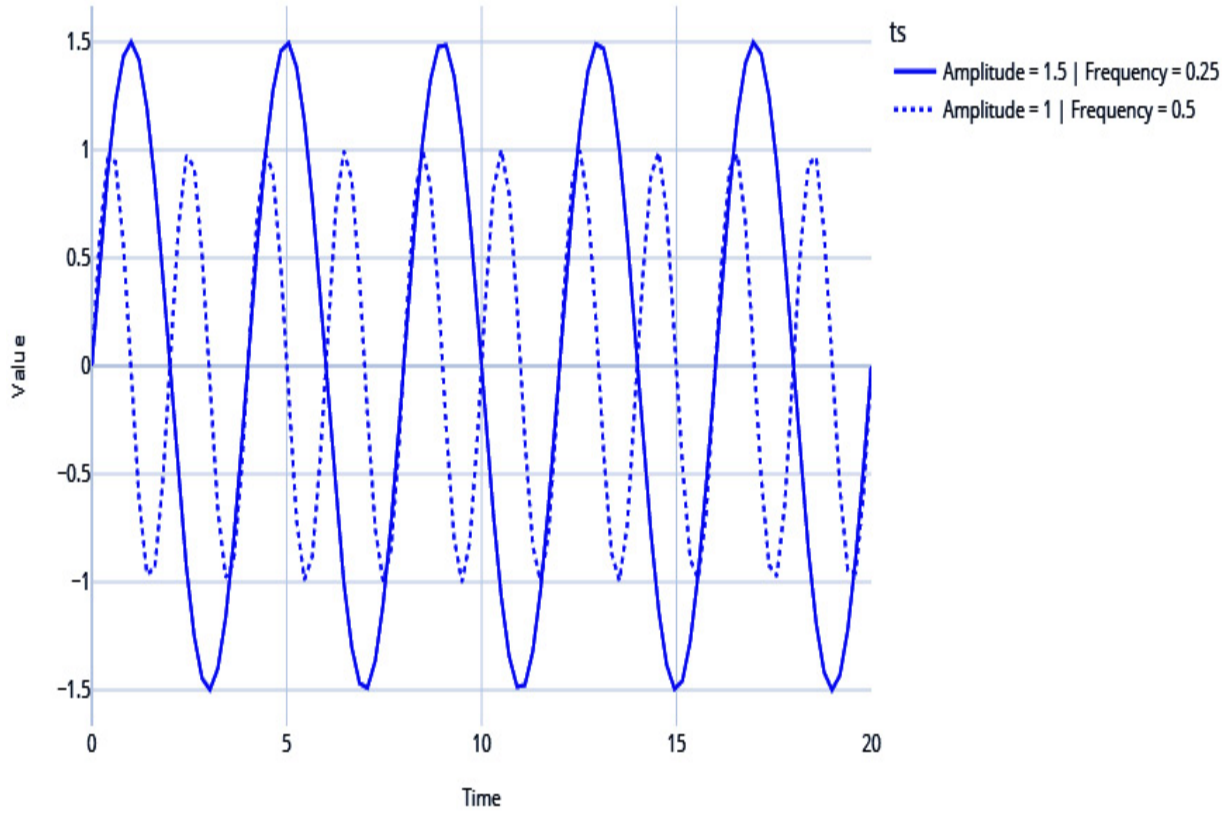


Figure 1.5 – Sinusoidal waves

Note the two sinusoidal waves are different with respect to the frequency (how fast the time series crosses zero) and amplitude (how far away from zero the time series travels).

TimeSynth also has another signal called **PseudoPeriodic**. This is like the **Sinusoidal** class, but the frequency and amplitude itself has some stochasticity. We can see in the following code snippet that this is more realistic than the vanilla sine and cosine waves from the **Sinusoidal** class:

```
# PseudoPeriodic signal with Amplitude=1 & Freque
signal = ts.signals.PseudoPeriodic(amplitude=1, f
#Generating Timeseries
samples, regular_time_samples, signals, errors =
plot_time_series(regular_time_samples,
                 samples,
                 "Pseudo Periodic")
```

Here is the output:

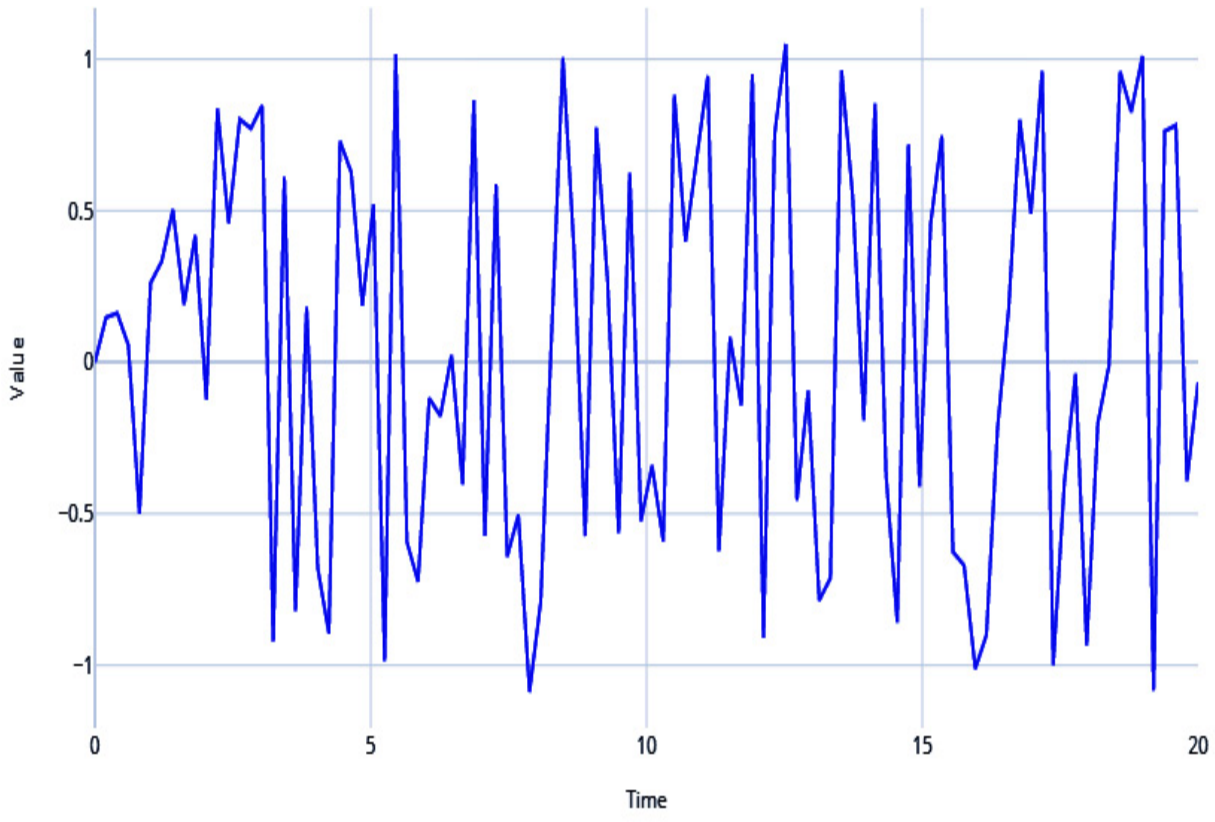


Figure 1.6 – Pseudo-periodic signal

Autoregressive signals

Another very popular signal in the real world is an **autoregressive (AR) signal**. We will go into this in more detail in Chapter 4, *Setting a Strong Baseline Forecast*, but for now, an AR signal refers to when the value of a time series for the current timestep is dependent on the values of the time series in the previous timesteps. This serial correlation is a key property of the AR signal, and it is parametrized by a few parameters, outlined as follows:

- Order of serial correlation—or, in other words, the number of previous timesteps the signal is dependent on

Coefficients to combine the previous timesteps

Let's see how we can generate an AR signal and see what it looks like, as follows:

```
# We have re-implemented the class in src because
from src.synthetic_ts.autoregressive import AutoR
# Autoregressive signal with parameters 1.5 and -
# y(t) = 1.5*y(t-1) - 0.75*y(t-2)
signal= AutoRegressive(ar_param=[1.5, -0.75])
#Generate Timeseries
samples, regular_time_samples, signals, errors =
plot_time_series(regular_time_samples,
```

```
                    samples,
                    "Auto Regressive")
```

Here is the output:

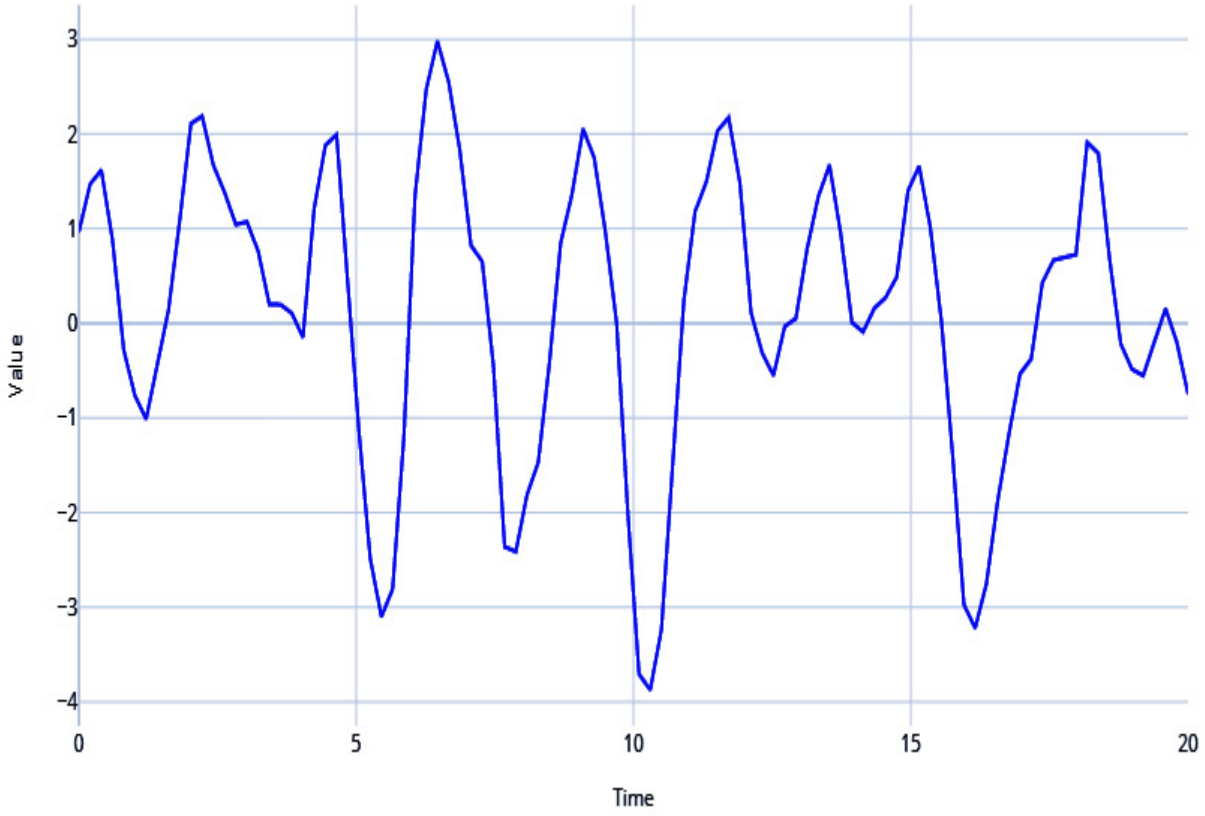


Figure 1.7 – AR signal

Mix and match

There are many more components you can use to create your DGP and thereby generate a time series, but let's quickly look at how we can combine the components we have already seen to generate a realistic time series.

Let's use a pseudo-periodic signal with white noise and combine it with an AR signal, as follows:

```
#Generating Pseudo Periodic Signal
pseudo_samples, regular_time_samples, _, _ = gene
# Generating an Autoregressive Signal
ar_samples, regular_time_samples, _, _ = generate
# Combining the two signals using a mathematical
ts = pseudo_samples*2+ar_samples
plot_time_series(regular_time_samples,
                 ts,
                 "Pseudo Periodic with AutoRegres
```

Here is the output:

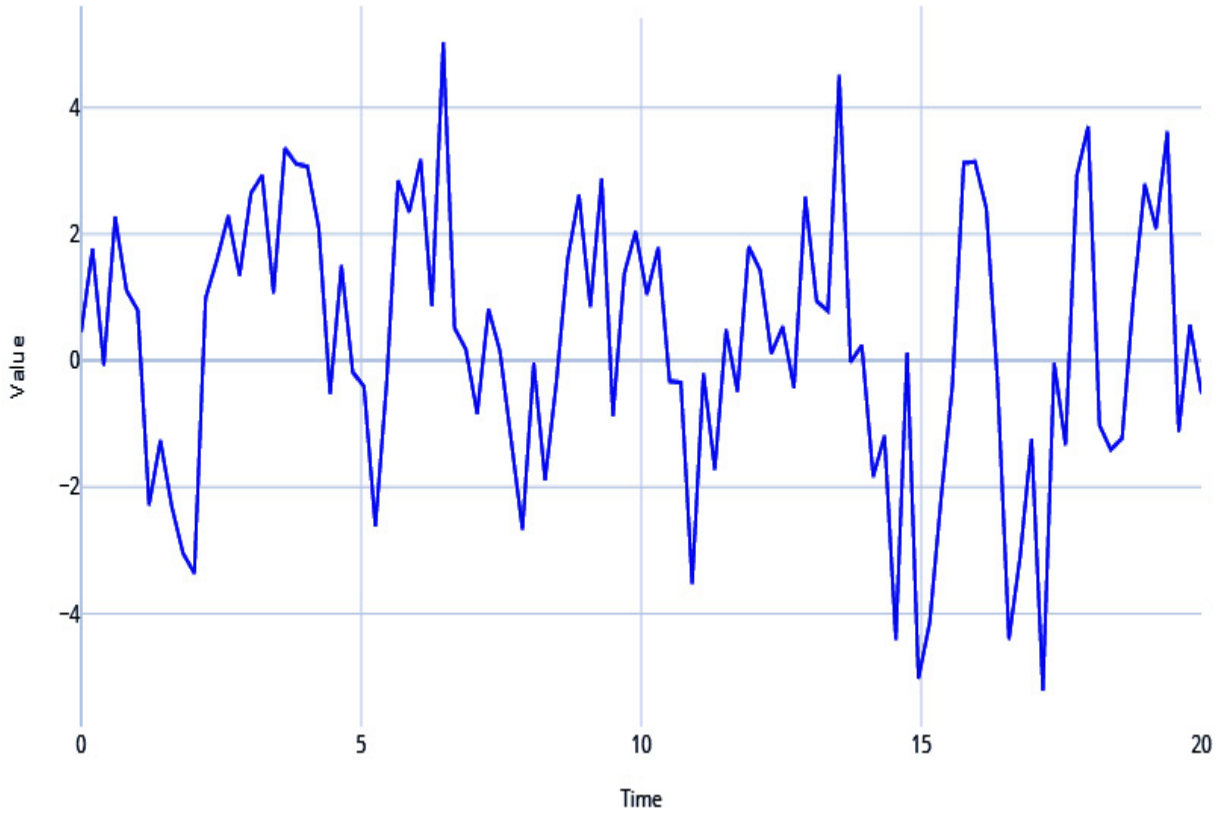


Figure 1.8 – Pseudo-periodic signal with AR and white noise

Stationary and non-stationary time series

In time series, **stationarity** is of great significance and is a key assumption in many modeling approaches. Ironically, many (if not most) real-world time series are non-stationary. So, let's understand what a stationary time series is from a layman's point of view.

There are multiple ways to look at stationarity, but one of the clearest and most intuitive ways is to think of the probability distribution or the data distribution of a time series. We call a time series stationary when the

probability distribution remains the same at every point in time. In other words, if you pick different windows in time, the data distribution across all those windows should be the same.

A standard Gaussian distribution is defined by two parameters—the mean and the variance. So, there are two ways the stationarity assumption can be broken, as outlined here:

- Change in mean over time

Change in variance over time

Let's look at these assumptions in detail and understand them better.

Change in mean over time

This is the most popular way a non-stationary time series presents itself. If there is an upward/downward trend in the time series, the mean across two windows of time would not be the same.

Another way non-stationarity manifests itself is in the form of seasonality. Suppose we are looking at the time series of average temperature measurements in a month for the last 5 years. From our experience, we know that temperature peaks during summer and falls in winter. So, when we take the mean temperature of winter and mean temperature of summer, they will be different.

Let's generate a time series with trend and seasonality and see how it manifests, as follows:

```
# Sinusoidal Signal with Amplitude=1 & Frequency=
signal=ts.signals.Sinusoidal(amplitude=1, freque
# White Noise with standard deviation = 0.3
noise=ts.noise.GaussianNoise(std=0.3)
# Generate the time series
sinusoidal_samples, regular_time_samples, _, _ =
# Regular_time_samples is a linear increasing tim
trend = regular_time_samples*0.4
# Combining the signal and trend
ts = sinusoidal_samples+trend
plot_time_series(regular_time_samples,
                 ts,
                 "Sinusoidal with Trend and White
```

Here is the output:

Figure 1.9 – Sinusoidal signal with trend and white noise

If you examine the time series in *Figure 1.9*, you will be able to see a definite trend and the seasonality, which together make the mean of the data distribution change wildly across different windows of time.

Change in variance over time

Non-stationarity can also present itself in the fluctuating variance of a time series. If the time series starts off with low variance and as time progresses, the variance keeps getting bigger and bigger, we have a non-stationary time series. In statistics, there is a scary name for this phenomenon—

heteroscedasticity. The Air Passengers dataset, which is the 'iris dataset' of time series (most popular, over-used, and use-less) is a classic example of heteroscedastic time series. Let's look at the plot.

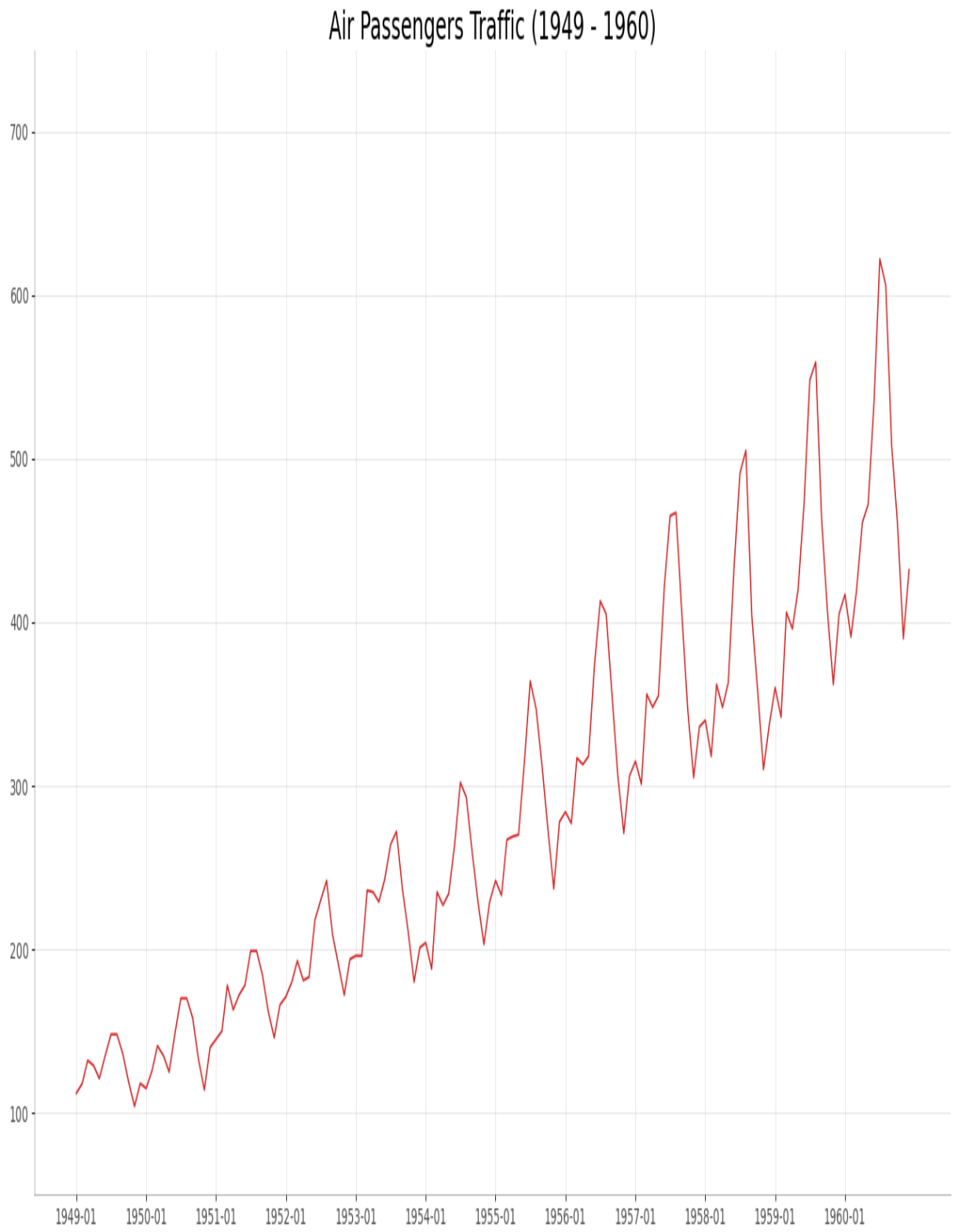

Air Passengers Traffic (1949 - 1960)
700
600
500
400
300
200
100
1949-01
1950-01
1951-01
1952-01
1953-01
1954-01
1955-01
1956-01
1957-01
1958-01
1959-01
1960-01

Figure 1.10 – Air Passengers Dataset – Example of Heteroscedastic Time Series

In the figure, you can see that the seasonal peaks keeps getting wider and wider as we move along the time and this is a classic sign that the time series is heteroscedastic. But all heteroscedastic time series aren't easy to spot. We have statistical tests to check for each of the stationarity cases which we will cover in Chapter 7 – Target Transformations for Time Series Forecasting.

This book just tries to give you intuition about stationary versus non-stationary time series. There is a lot of statistical theory and depth in this discussion that we are skipping over to keep our focus on the practical aspects of time series.

Armed with the mental model of the DGP, we are at the right place to think about another important question: *What can we forecast?*

What can we forecast?

Before we move ahead, there is another aspect of time series forecasting that we have to understand—*the predictability of a time series*. The most basic assumption when we forecast a time series is that the future depends on the past. But not all time series are equally predictable.

Let's take a look at a few examples and try to rank these in order of predictability (from easiest to hardest), as follows:

- High tide next Monday
- Lottery numbers next Sunday
- The stock price of Tesla next Friday

Intuitively, it is very easy for us to rank them. High tide next Monday is going to be the easiest to predict because it is so predictable, the lottery numbers are going to be very hard to predict because these are pretty much random, and the stock price of Tesla next Friday is going to be difficult to predict, but not impossible.

NOTE

> *However, for people thinking that they can forecast stock prices with the advanced techniques covered in the book and get rich, that won't happen(most likely). Although it is worthy of a lengthy discussion, we can summarize the key points in a short paragraph.*

Share prices are not a function of their past values but an anticipation of their future values, and this thereby violates our first assumption while forecasting. And if that is not bad enough, financial stock prices typically have a very low signal-to-noise ratio. The final wrench in the process is the **efficient-market hypothesis** (**EMH**). This seemingly innocent hypothesis

proclaims that all known information about a stock price is already factored into the price of the stock. The implication of the hypothesis is that if you can forecast accurately, many others will also be able to do that, and thereby the market price of the stock already reflects the change in price that this forecast brought about.

The M6 competition chose to tackle this problem head-on to evaluate if the EMH holds true by conducting a year long forecasting and investment strategy competition. Although not conclusive, the results show that the EMH holds true for the vast majority of the participants, barring a few top teams. And even in that, they found out that in the top teams, there was no significant correlation between forecasting accuracies and the selection of stocks into the portfolio, i.e. the teams weren't choosing stocks which they were able to forecast better (Full report is linked in Further Reading).

Coming back to the topic at hand—predictability—three main factors form a mental model for this, as follows:

- **Understanding the DGP**: The better you understand the DGP, the higher the predictability of a time series.
- **Amount of data**: The more data you have, the better your predictability is.
- **Adequately repeating pattern**: For any mathematical model to work well, there should be an adequately repeating pattern in your time series. The more repeatable the pattern is, the better your predictability is.

Even though you have a mental model of how to think about predictability, we will look at more concrete ways of assessing the predictability of time series in *Chapter 3*, *Analyzing and Visualizing Time Series Data*, but the key takeaway is that not all time series are equally predictable.

In order to fully follow the discussion in the coming chapters, we need to establish a standard notation and get updated on terminology that is specific to time series analysis.

Forecasting terminology

There are a few terminologies that will help you follow the book as well as other literature on time series. These are described in more detail here:

- **Forecasting**

Forecasting is the prediction of future values of a time series using the known past values of the time series and/or some other related variables. This is very similar to prediction in ML where we use a model to predict unseen data.

- **Multivariate forecasting**

Multivariate time series consist of more than one time series variable that is not only dependent on its past values but also has some dependency on the other variables. For example, a set of macroeconomic indicators such as

gross domestic product (**GDP**), inflation, and so on of a particular country can be considered as a multivariate time series. The aim of multivariate forecasting is to come up with a model that captures the interrelationship between the different variables along with its relationship with its past and forecast all the time series together in the future.

- **Explanatory forecasting**

In addition to the past values of a time series, we might use some other information to predict the future values of a time series. For example, for predicting retail store sales, information regarding promotional offers (both historical and future ones) is usually helpful. This type of forecasting, which uses information other than its own history, is called explanatory forecasting.

- **Backtesting**

Setting aside a validation set from your training data to evaluate your models is a practice that is common in the ML world. Backtesting is the time series equivalent of validation, whereby you use the history to evaluate a trained model. We will cover the different ways of doing validation and cross-validation for time series data later.

- **In-sample and out-sample**

Again, drawing parallels with ML, in-sample refers to training data and out-sample refers to unseen or testing data. When you hear in-sample metrics, this is referring to metrics calculated on training data, and out-sample metrics is referring to metrics calculated on testing data.

- **Exogenous and endogenous variables**

Exogenous variables are parallel time series variables that are not modeled directly for output but used to help us model the time series that we are interested in. Typically, exogenous variables are not affected by other variables in the system. Endogenous variables are variables that are affected by other variables in the system. A purely endogenous variable is a variable that is entirely dependent on the other variables in the system. Relaxing the strict assumptions a bit, we can consider the target variable as the endogenous variable and the explanatory regressors we include in the model as exogenous variables.

- **Forecast combination**

Forecast combinations in the time series world are similar to ensembles from the ML world. It is a process by which we combine multiple forecasts by using some function, either learned or heuristic-based, such as a simple average of three forecast models.

There are a lot more terms specific to time series, some of which we will be covering throughout the book. But to start with a basic familiarity in the

field, these terms should be a good starting point.

Summary

We had our first dip into time series as we understood the different types of time series, looked at how a DGP generates a time series, and saw how we can think about the important question: *How well can we forecast a time series?* We also had a quick review of the terminology and notation required to understand the rest of the book. In the next chapter, we will be getting our hands dirty and will learn how to work with time series data, how to preprocess a time series, how to handle missing data and outliers, and so on. If you have not set up the environment yet, take a break and put some time into doing that.

Further reading

- *A Survey on Principles, Models and Methods for Learning from Irregularly Sampled Time Series: From Discretization to Attention and Invariance* by S.N. Shukla and B.M. Marlin (2020): https://arxiv.org/abs/2012.00168
- *Learning from Irregularly-Sampled Time Series: A Missing Data Perspective* by S.C. Li and B.M. Marlin (2020), ICML: https://arxiv.org/abs/2008.07599
- *The M6 forecasting competition: Bridging the gap between forecasting and investment decisions by Spyros Makridakis et al. (2023)*

https://arxiv.org/abs/2310.13357

2 Acquiring and Processing Time Series Data

Join our book community on Discord

https://packt.link/EarlyAccess/

In the previous chapter, we learned what a time series is and established a few standard notations and terminologies. Now, let's switch tracks from theory to practice. In this chapter, we are going to get our hands dirty and start working with data. Although we said time series data is everywhere, we are still yet to get our hands dirty with a few time series datasets. We are going to start working on the dataset we have chosen to work on throughout this book, process it in the right way, and learn about a few techniques for dealing with missing values.

In this chapter, we will cover the following topics:

- Understanding the time series dataset
- pandas datetime operations, indexing, and slicing – a refresher
- Handling missing data
- Mapping additional information
- Saving and loading files to disk
- Handling longer periods of missing data

Technical requirements

You will need to set up the `Anaconda` environment following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks.

The code for this chapter can be found at https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-/tree/main/notebooks/Chapter02.

Handling time series data is like handling other tabular datasets, but with a focus on the temporal dimension. As with any tabular dataset, **pandas** is perfectly equipped to handle time series data as well.

Let's start getting our hands dirty and work through a dataset from the beginning. We are going to use the *London Smart Meters* dataset throughout this book. If you have not downloaded the data already as part of the environment setup, go to the *Preface* and do that now.

Understanding the time series dataset

This is the key first step in any new dataset you come across, even before `Exploratory Data Analysis` (`EDA`), which we will be covering in Chapter 3, *Analyzing and Visualizing Time Series Data*. Understanding where the data is coming from, the data generating process behind it, and the source domain is essential to having a good understanding of the dataset.

London Data Store, a free and open data-sharing portal, provided this dataset, which was collected and enriched by Jean-Michel D and uploaded on Kaggle.

The dataset contains energy consumption readings for a sample of 5,567 London households that took part in the UK Power Networks-led Low Carbon London project between November 2011 and February 2014. Readings were taken at half-hourly intervals. Some metadata about the households is also available as part of the dataset. Let's look at what metadata is available as part of the dataset:

- CACI UK segmented the UK's population into demographic types, called Acorn. For each household in the data, we have the corresponding Acorn classification. The Acorn classes (Lavish Lifestyles, City Sophisticates, Student Life, and so on) are grouped into parent classes (Affluent Achievers, Rising Prosperity, Financially Stretched, and so on). A full list of Acorn classes can be found in *Table 2.1*. The complete documentation detailing each class is available at https://acorn.caci.co.uk/downloads/Acorn-User-guide.pdf.
- The dataset contains two groups of customers – one group who was subjected to `dynamic time-of-use` (`dToU`) energy prices throughout 2013, and another group who were on flat-rate tariffs. The tariff prices for the dToU were given a day ahead via the Smart Meter IHD or via text message.
- Jean-Michel D also enriched the dataset with weather and UK bank holidays data.

The following table shows the Acorn classes:

Table 2.1 – ACORN classification

NOTE

> *The Kaggle dataset also preprocesses the time series data daily and combines all the separate files. Here, we will ignore those files and start with the raw files, which can be found in the* `hhblock_dataset` *folder. Learning to work with the raw files is an integral part of working with real-world datasets in the industry.*

Preparing a data model

Once we understand where the data is coming from, we can look at the data, understand the information present in the different files, and figure out a mental model of how to relate the different files. You may call it old school, but Microsoft Excel is an excellent tool for gaining this first-level understanding. If the file is too big to open in Excel, we can also read it in Python and save a sample of the data to an Excel file and open it. However, keep in mind that Excel sometimes messes with the format of the data, especially dates, so we need to take care to not save the file and write back the formatting changes Excel made. If you are allergic to Excel, you can do it in Python as well, albeit with a lot more keystrokes. The purpose of this exercise is to *see* what the different data files contain, explore the relationship between the different files, and so on. We can make this more formal and explicit by drawing a data model, similar to the one shown in the following diagram:

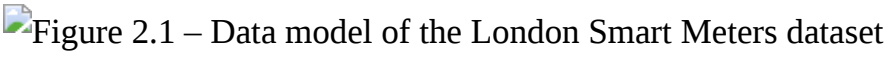


Figure 2.1 – Data model of the London Smart Meters dataset

The data model is more for us to understand the data rather than any data engineering purpose. Therefore, it only contains bare-minimum information, such as the key columns on the left and the sample data on the right. We also have arrows connecting different files, with keys used to link the files.

Let's look at a few key column names and their meanings:

- `LCLid` : The unique consumer ID for a household
- `stdorTou` : Whether the household has dToU or standard
- `Acorn` : The ACORN class
- `Acorn_grouped` : The ACORN group
- `file` : The block number

Each `LCLid` has a unique time series attached to it. The time series file is formatted in a slightly tricky format – each day, there will be 48 observations at a half-hourly frequency in the columns of the file.

NOTEBOOK ALERT

To follow along with the complete code, use the `01 - Pandas Refresher & Missing Values Treatment.ipynb` *notebook in the* `chapter01` *folder.*

Before we start working with our dataset, there are a few concepts we need to establish. One of them is a concept in **pandas DataFrames**, which is of utmost importance – the pandas datetime properties and index. Let's quickly look at a few **pandas** concepts that will be useful.

NOTE

If you are familiar with the datetime manipulations in pandas, feel free to skip ahead to the next section.

pandas datetime operations, indexing, and slicing – a refresher

Instead of using our dataset, which is slightly complex, let's pick an easy, well-formatted stock exchange price dataset from the UCI Machine Learning Repository and look at the functionality of pandas:

```
df = pd.read_excel("https://archive.ics.uci.edu/ml/machi
```

The DataFrame that we read looks as follows:

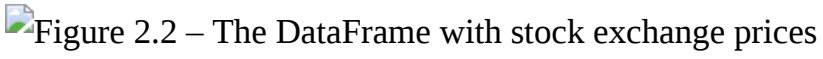


Figure 2.2 – The DataFrame with stock exchange prices

Now that we have read the `DataFrame`, let's start manipulating it.

Converting the date columns into pd.Timestamp/DatetimeIndex

First, we must convert the date column (which may not always be parsed as dates automatically by pandas) into **pandas** datetime. For that, **pandas** has a handy function called `pd.to_datetime`. It infers the datetime format automatically and converts the input into a `pd.Timestamp`, if the input is a `string`, or into a `DatetimeIndex` if the input is a `list` of strings. So, if we pass a single date as a string, `pd.to_datetime` converts it into `pd.Timestamp`, while if we pass a list of dates, it converts it into `DatetimeIndex`:

```
>>> pd.to_datetime("13-4-1987").strftime("%d, %B %Y")
'13, April 1987'
```

Now, let's look at a case where the automatic parsing fails. The date is January 4, 1987. Let's see what happens when we pass the string to the function:

```
>>> pd.to_datetime("4-1-1987").strftime("%d, %B %Y")
'01, April 1987'
```

Well, that wasn't expected, right? But if you think about it, anyone can make that mistake because we are not telling the computer whether the month or the day comes

first, and pandas assumes the month comes first. Let's rectify that:

```
>>> pd.to_datetime("4-1-1987", dayfirst=True).strftime("
'04, January 1987'
```

Another case where automatic date parsing fails is when the date string is in a non-standard form. In that case, we can provide a `strftime` formatted string to help pandas parse the dates correctly:

```
>>> pd.to_datetime("4|1|1987", format="%d|%m|%Y").strfti
'04, January 1987'
```

A full list of `strftime` conventions can be found at https://strftime.org/.

PRACTITIONER'S TIP

Because of the wide variety of data formats, pandas may infer the time incorrectly. While reading a file, pandas will try to parse the dates automatically and create an error. There are many ways we can control this behavior: we can use the `parse_dates` *flag to turn off date parsing, the* `date_parser` *argument to pass in a custom date parser, and* `year_first` *and* `day_first` *to easily denote two popular formats of dates. From version 2.0, pandas supports* `date_format` *which can be used to pass in the exact format of the date as a python dictionary with column name as the key.*

Out of all these options, I prefer to use `date_format`*, if using pandas >=2.0. We can keep* `parse_dates=True` *and then pass in the exact*

date format using `strftime` *conventions. This ensures that the date is parsed in the way we want it to be.*

If working with pandas <2.0, then I prefer to keep `parse_dates=False` *in both* `pd.read_csv` *and* `pd.read_excel` *to make sure pandas is not parsing the data automatically. After that, you can convert the date using the* `format` *parameter, which lets you explicitly set the date format of the column using* `strftime` *conventions. There are two other parameters in* `pd.to_datetime` *that will also make inferring dates less error-prone – `yearfirst` and `dayfirst`. If you don't provide an explicit date format, at least provide one of these.*

Now, let's convert the date column in our stock prices dataset into datetime:

```
df['date'] = pd.to_datetime(df['date'], yearfirst=True)
```

Now, the `'date'` column, `dtype`, should be either `datetime64[ns]` or `<M8[ns]`, which are both pandas/NumPy native datetime formats. But why do we need to do this?

It's because of the wide range of additional functionalities this unlocks. The traditional `min()` and `max()` functions will start working because pandas knows it is a datetime column:

```
>>> df.date.min(),df.date.max()
(Timestamp('2009-01-05 00:00:00'), Timestamp('2011-02-22
```

Let's look at a few cool features the datetime format gives us.

Using the .dt accessor and datetime properties

Since the column is now in date format, all the semantic information that is encoded in the date can be used through pandas datetime properties. We can access many datetime properties, such as `month`, `day_of_week`, `day_of_year`, and so on, using the `.dt` accessor:

```
>>> print(f"""
     Date: {df.date.iloc[0]}
     Day of year: {df.date.dt.day_of_year.iloc[0]}
     Day of week: {df.date.dt.dayofweek.iloc[0]}
     Month: {df.date.dt.month.iloc[0]}
     Month Name: {df.date.dt.month_name().iloc[0]}
     Quarter: {df.date.dt.quarter.iloc[0]}
     Year: {df.date.dt.year.iloc[0]}
     ISO Week: {df.date.dt.isocalendar().week.iloc[0]}
     """)
Date: 2009-01-05 00:00:00
Day of year: 5
Day of week: 0
Month: 1
Month Name: January
Quarter: 1
Year: 2009
ISO Week: 2
```

As of pandas 1.1.0, `week_of_year` has been deprecated because of the inconsistencies it produces at the end/start of the year. Instead, the ISO Calendar

standards (which are commonly used in government and business) have been adopted and we can access the ISO calendar to get the ISO weeks.

Slicing and indexing

The real fun starts when we make the date column the index of the DataFrame. By doing this, you can use all the fancy slicing operations that pandas supports but on the datetime axis. Let's take a look at few of them:

```
# Setting the index as the datetime column
df.set_index("date", inplace=True)
# Select all data after 2010-01-04(including)
df["2010-01-04":]
# Select all data between 2010-01-04 and 2010-02-06(not
df["2010-01-04": "2010-02-06"]
# Select data 2010 and before
df[: "2010"]
# Select data between 2010-01 and 2010-06(both including
df["2010-01": "2010-06"]
```

In addition to the semantic information and intelligent indexing and slicing, **pandas** also provide tools for creating and manipulating date sequences.

Creating date sequences and managing date offsets

If you are familiar with `range` in Python and `np.arange` in NumPy, then you will know they help us create `integer` / `float` sequences by providing a start point and an end point. pandas has something similar for datetime – `pd.date_range`. The function accepts start and end dates, along with a frequency

(daily or monthly, and so on) and creates the sequence of dates in between. Let's look at a couple of ways of creating a sequence of dates:

```
# Specifying start and end dates with frequency
pd.date_range(start="2018-01-20", end="2018-01-23", freq
# Output: ['2018-01-20', '2018-01-21', '2018-01-22', '20
# Specifying start and number of periods to generate in
pd.date_range(start="2018-01-20", periods=4, freq="D").a
# Output: ['2018-01-20', '2018-01-21', '2018-01-22', '20
# Generating a date sequence with every 2 days
pd.date_range(start="2018-01-20", periods=4, freq="2D").
# Output: ['2018-01-20', '2018-01-22', '2018-01-24', '20
# Generating a date sequence every month. By default it
pd.date_range(start="2018-01-20", periods=4, freq="M").a
# Output: ['2018-01-31', '2018-02-28', '2018-03-31', '20
# Generating a date sequence every month, but month star
pd.date_range(start="2018-01-20", periods=4, freq="MS").
# Output: ['2018-02-01', '2018-03-01', '2018-04-01', '20
```

We can also add or subtract days, months, and other values to/from dates using `pd.TimeDelta`:

```
# Add four days to the date range
(pd.date_range(start="2018-01-20", end="2018-01-23", fre
# Output: ['2018-01-24', '2018-01-25', '2018-01-26', '20
# Add four weeks to the date range
(pd.date_range(start="2018-01-20", end="2018-01-23", fre
# Output: ['2018-02-17', '2018-02-18', '2018-02-19', '20
```

There are a lot of these aliases in **pandas**, including `W`, `W-MON`, `MS`, and others. The full list can be found at https://pandas.pydata.org/docs/user_guide/timeseries.html#timeseries-offset-aliases.

In this section, we looked at a few useful features and operations we can perform on datetime indices and know how to manipulate DataFrames with datetime columns. Now, let's review a few techniques we can use to deal with missing data.

Handling missing data

While dealing with large datasets in the wild, you are bound to encounter missing data. If it is not part of the time series, it may be part of the additional information you collect and map. Before we jump the gun and fill it with a mean value or drop those rows, let's think about a few aspects:

- The first consideration should be whether the missing data we are worried about is missing or not. For that, we need to think about the `Data Generating Process` (`DGP`) (the process that is generating the time series). As an example, let's look at sales at a local supermarket. You have been given the `point-of-sale` (`POS`) transactions for the last 2 years and you are processing the data into a time series. While analyzing the data, you found that there are a few products where there aren't any transactions for a few days. Now, what you need to think about is whether the missing data is missing or whether there is some information that this missingness is giving you. If you don't have any transactions for a particular product for a day, it will appear as missing data while you are processing it, even though it is not missing. What that tells us is that there were no sales for that item, and that you should fill such missing data with zeros.
- Now, what if you see that, every Sunday, the data is missing – that is, there is a pattern to the missingness. This becomes tricky because how you fill in such gaps

depends on the model that you intend to use. If you fill in such gaps with zeros, a model that looks at the immediate past to predict the future might be thrown off, especially for Monday. However, if you tell the model that the previous day was Sunday, then the model still can learn to tell the difference.

- Lastly, what if you see zero sales on one of the best-selling products that always gets sold? This can happen because of something such as a POS machine malfunction, a data entry mistake, or an out-of-stock situation. These types of missing values can be imputed with a few techniques.

Let's look at an Air Quality dataset published by the ACT Government, Canberra, Australia under the CC by Attribution 4.0 International License (https://www.data.act.gov.au/Environment/Air-Quality-Monitoring-Data/94a5-zqnn) and see how we can impute such values using pandas (there are more sophisticated techniques available, all of which will be covered later in this chapter).

PRACTITIONER'S TIP

When reading data using a method such as `read_csv`*, pandas provides a few handy ways to handle missing values. pandas treats many values such as* `#N/A`*,* `null`*, and so on as* `NaN` *by default. We can control this list of allowable* `NaN` *values using the* `na_values` *and* `keep_default_na` *parameters.*

We have chosen region `Monash` and `PM2.5` readings and artificially introduced some missing values, as shown in the following diagram:

Figure 2.3 – Missing values in the Air Quality dataset

Now, let's look at a few simple techniques we can use to fill the missing values:

- `Last Observation Carried Forward or Forward Fill`: This imputation technique takes the last observed value and uses that to fill all the missing values until it finds the next observation. It is also called forward fill. We can do this like so:

```
df['pm2_5_1_hr'].ffill()
```

- `Next Observation Carried Backward of Backward Fill`: This imputation technique takes the next observation and backtracks to fill in all the missing values with this value. This is also called backward fill. Let's see how we can do this in pandas:

```
df['pm2_5_1_hr'].bfill()
```

- `Mean Value Fill`: This imputation technique is also pretty simple. We calculate the mean of the entire series and wherever we find missing values, we fill it with the mean value:

```
df['pm2_5_1_hr'].fillna(df['pm2_5_1_hr'].mean())
```

Let's plot the imputed lines we get from using these three techniques:

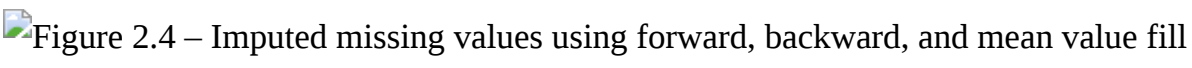


Figure 2.4 – Imputed missing values using forward, backward, and mean value fill

Another family of imputation techniques covers interpolation:

- `Linear Interpolation`: Linear interpolation is just like drawing a line between the two observed points and filling the missing values so that they lie on this line. This is how we do it:

```
df['pm2_5_1_hr'].interpolate(method="linear")
```

- `Nearest Interpolation`: This is intuitively like a combination of the forward and backward fill. For each missing value, the closest observed value is found and is used to fill in the missing value:

```
df['pm2_5_1_hr'].interpolate(method="nearest")
```

Let's plot the two interpolated lines:

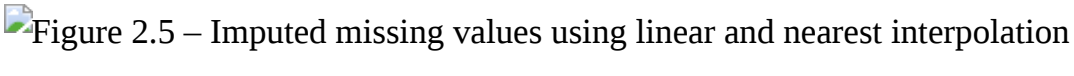


Figure 2.5 – Imputed missing values using linear and nearest interpolation

Figure 2.5 – Imputed missing values using linear and nearest interpolation

There are a few non-linear interpolation techniques as well:

- `Spline, Polynomial, and Other Interpolations`: In addition to linear interpolation, pandas also supports non-linear interpolation techniques that call a SciPy routine at the backend. Spline and polynomial interpolations are similar. They fit a spline/polynomial of a given order to the data and use that to fill in missing values. While using `spline` or `polynomial` as the method in `interpolate`, we should always provide `order` as well. The higher the order,

the more flexible the function that is used will be to fit the observed points. Let's see how we can use spline and polynomial interpolation:

```
df['pm2_5_1_hr'].interpolate(method="spline", order=2)
df['pm2_5_1_hr'].interpolate(method="polynomial", order=
```

Let's plot these two non-linear interpolation techniques:

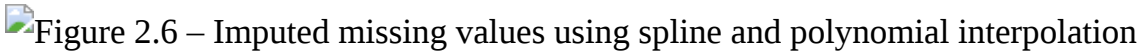


Figure 2.6 – Imputed missing values using spline and polynomial interpolation

For a complete list of interpolation techniques supported by `interpolate`, go to https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.interpolate.html and https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.interp1d.html#scipy.interpolate.interp1d.

Now that we are more comfortable with the way pandas manages datetime, let's go back to our dataset and convert the data into a more manageable form.

NOTEBOOK ALERT

> *To follow along with the complete code for pre-processing, use the* `02 - Preprocessing London Smart Meter Dataset.ipynb` *notebook in the* `chapter01` *folder.*

Converting the half-hourly block-level data (hhblock) into time series data

Before we start processing, let's understand a few general categories of information we will find in a time series dataset:

- **Time Series Identifiers**: These are identifiers for a particular time series. It can be a name, an ID, or any other unique feature – for example, the SKU name or the ID of a retail sales dataset or the consumer ID in the energy dataset that we are working with are all time series identifiers.
- **Metadata or Static Features**: This information does not vary with time. An example of this is the ACORN classification of the household in our dataset.
- **Time-Varying Features**: This information varies with time – for example, the weather information. For each point in time, we have a different value for weather, unlike the Acorn classification.

Next. Let's discuss formatting of a dataset.

Compact, expanded, and wide forms of data

There are many ways to format a time series dataset, especially a dataset with many related time series, like the one we have now. Apart from the standard terminology of `wide` data, we can also look at two non-standard ways of formatting time series data. Although there is no standard nomenclature for those, we will refer to them as `compact` and `expanded` in this book.

Compact form data is when any particular time series occupies only a single row in the pandas DataFrame – that is, the time dimension is managed as an array within a DataFrame row. The time series identifiers and the metadata occupy the columns with scalar values and then the time series values; other time-varying features occupy the columns with an array. Two additional columns are included to extrapolate time – `start_datetime` and frequency. If we know the start datetime and the frequency

of the time series, we can easily construct the time and recover the time series from the DataFrame. This only works for regularly sampled time series. The advantage is that the DataFrames take up much less memory and are easy and faster to work with:

Figure 2.7 – Compact form data

The expanded form is when the time series is expanded along the rows of a DataFrame. If there are *n* steps in the time series, it occupies *n* rows in the DataFrame. The time series identifiers and the metadata get repeated along all the rows. The time-varying features also get expanded along the rows. And instead of the start date and frequency, we have the datetime as a column:

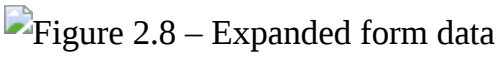


Figure 2.8 – Expanded form data

If the compact form had a time series identifier as the key, the time series identifier and the datetime column would be combined and become the key.

Wide-format data is more common in traditional time series literature. It can be considered a legacy format, which is limiting in many ways. Do you remember the stock data we saw earlier (*Figure 2.2*)? We have the date as an index or as one of the columns and the different time series as different columns of the DataFrame. As the number of time series increases, they become wider and wider, hence the name. This data format does not allow us to include any metadata about the time series. For instance, in our data, we have information about whether a particular household is under standard or dynamic pricing. There is no way for us to include such metadata in

the wide format. From an operational perspective, the wide format also does not play well with relational databases because we have to keep adding columns to the table when we get new time series. We won't be using this format in this book.

Enforcing regular intervals in time series

One of the first things you should check and correct is whether the regularly sampled time series data that you have has equal intervals of time. In practice, even regularly sampled time series have some samples missing in between because of some data collection error or some other peculiar way data is collected. So, while working with the data, we will make sure we enforce regular intervals in the time series.

BEST PRACTICE

> *While working with datasets with multiple time series, it is best practice to check the end dates of all the time series. If they are not uniform, we can align them with the latest date across all the time series in the dataset.*

In our smart meters dataset, some `LCLid` columns end much earlier than the rest. Maybe the household opted out of the program, or they moved out and left the house empty; the reason could be anything. However, we need to handle that while we enforce regular intervals.

We will learn how to convert the dataset into a time series format in the next section. The code for this process can be found in the `02 - Preprocessing London Smart Meter Dataset.ipynb` notebook.

Converting the London Smart Meters dataset into a time series format

For each dataset that you come across, the steps you would have to take to convert it into either a compact or expanded form would be different. It depends on how the original data is structured. Here, we will look at how the London Smart Meters dataset can be transformed so that we can transfer those learnings to other datasets.

There are two steps we need to do before we can start processing the data into either compact or expanded form:

1. `Find the Global End Date`: We must find the maximum date across all the block files so that we know the global end date of the time series.
2. `Basic Preprocessing`: If you remember how `hhblock_dataset` is structured, you will remember that each row had a date and that along the columns, we have half-hourly blocks. We need to reshape that into a long form, where each row has a date and a single half-hourly block. It's easier to handle that way.

Now, let's define separate functions for converting the data into compact and expanded forms and `apply` that function to each of the `LCLid` columns. We will do this for each `LCLid` separately since the start date for each `LCLid` is different.

Expanded form

The function for converting into expanded form does the following:

1. Finds the start date.
2. Create a standard DataFrame using the start date and the global end date.
3. Left merges the DataFrame for `LCLid` to the standard DataFrame, leaving the missing data as `np.nan`.
4. Returns the merged DataFrame.

Once we have all the `LCLid` DataFrames, we must perform a couple of additional steps to complete the expanded form processing:

1. Concatenate all the DataFrames into a single DataFrame.
2. Create a column called offset, which is the numerical representation of the half-hour blocks; for example, `hh_3` → `3`.
3. Create a timestamp by adding a 30-minute offset to the day and dropping the unnecessary columns.

For one block, this representation takes up ~47 MB of memory.

Compact form

The function for converting into compact form does the following:

1. Finds the start date and time series identifiers.
2. Creates a standard DataFrame using the start date and the global end date.
3. Left merges the DataFrame for `LCLid` to the standard DataFrame, leaving the missing data as `np.nan`.
4. Sorts the values on the date.
5. Returns the time series array, along with the time series identifier, start date, and the length of the time series.

Once we have this information for each `LCLid`, we can compile it into a DataFrame and add `30min` as the frequency.

For one block, this representation takes up only ~0.002 MB of memory.

We are going to use the compact form because it is easy to work with and much less resource hungry.

Mapping additional information

From the data model that we prepared earlier, we know that there are three key files that we have to map: *Household Information*, *Weather*, and *Bank Holidays*.

The `informations_households.csv` file contains metadata about the household. There are static features that are not dependent on time. For this, we just need to left merge `informations_households.csv` to the compact form based on `LCLid`, which is the time series identifier.

BEST PRACTICE

> *While doing a pandas merge, one of the most common and unexpected outcomes is that the number of rows before and after the operation is not the same (even if you are doing a left merge). This typically happens because there are duplicates in the keys on which you are merging. As a best practice, you can use the* `validate` *parameter in the pandas merge, which takes in inputs such as* `one_to_one` *and* `many_to_one` *so that this check is done while merging and will throw an error if the assumption is not met. For more information, go to https://pandas.pydata.org/docs/reference/api/pandas.merge.html*

Bank Holidays and Weather, on the other hand, are time-varying features and should be dealt with accordingly. The most important aspect to keep in mind is that while we map this information, it should perfectly align with the time series that we have already stored as an array.

`uk_bank_holidays.csv` is a file that contains the dates of the holidays and the kind of holiday. The holiday information is quite important here because the energy

consumption patterns would be different on a holiday when the family members are at home spending time with each other or watching television, and so on. Follow these steps to process this file:

1. Convert the date column into the datetime format and set it as the index of the DataFrame.
2. Using the `resample` function we saw earlier, we must ensure that the index is resampled every 30 minutes, which is the frequency of the times series.
3. Forward fill the holidays within a day and fill in the rest of the `NaN` values with `NO_HOLIDAY`.

Now, we have converted the holiday file into a DataFrame that has a row for each 30-minute interval. On each row, we have a column that specifies whether that day was a holiday or not.

`weather_hourly_darksky.csv` is a file that is, once again, at the daily frequency. We need to downsample it to a 30-minute frequency because the data that we need to map to this is at a half-hourly frequency. If we don't do this, the weather will only be mapped to the hourly timestamps, leaving the half-hourly timestamps empty.

The steps we must follow to process this file are also similar to the way we processed holidays:

1. Convert the date column into the datetime format and set it as the index of the DataFrame.
2. Using the `resample function`, we must ensure that the index is resampled every 30 minutes, which is the frequency of the times series.

3. Forward fill the weather features to fill the missing values that were created while resampling.

Now that you have made sure the alignment between the time series and the time-varying features is ensured, you can loop over each of the time series and extract the weather and bank holiday array before storing it in the corresponding row of the DataFrame.

Saving and loading files to disk

The fully merged DataFrame in its compact form takes up only ~10 MB. But saving this file requires a little bit of engineering. If we try to save the file in CSV format, it will not work because of the way we have stored arrays in pandas columns (since the data is in its compact form). We can save it in `pickle` or `parquet` format, or any of the binary forms of file storage. This can work, depending on the size of the RAM available in our machines. Although the fully merged DataFrame is just ~10 MB, saving it in `pickle` format will make the size explode to ~15 GB.

What we can do is save this as a text file while making a few tweaks to accommodate the column names, column types, and other metadata that is required to read the file back into memory. The resulting file size on disk still comes out to ~15 GB but since we are doing it as an I/O operation, we are not keeping all that data in our memory. We call this the time series (`.ts`) format. The functions for saving a compact form in `.ts` format, reading the `.ts` format, and converting the compact form into expanded form are available in this book's GitHub repository under `src/data_utils.py`.

If you don't need to store all of the DataFrame in a single file, you can split it into multiple chunks and save them individually in a binary format, such as `parquet`.

For our datasets, let's follow this route and split the whole DataFrame into chunks of blocks and save them as `parquet` files. This is the best route for us because of a few reasons:

- It leverages the compression that comes with the format
- It reads in parts of the whole data for quick iteration and experimentation
- The data types are retained between the read and write operations, leading to less ambiguity

NOTE

For very large datasets, we can use some pandas alternatives which makes it easier to process datasets which are out of memory. Polars is a great library which has lazy loading and is very fast. And for truly huge datasets, pyspark with a distributed cluster might be the right choice.

Now that we have processed the dataset and stored it on disk, let's read it back into memory and look at a few more techniques to handle missing data.

Handling longer periods of missing data

We saw some techniques for handling missing data earlier – forward and backward filling, interpolation, and so on. Those techniques usually work if there are one or two missing data points. But if a large section of data is missing, then these simple techniques fall short.

NOTEBOOK ALERT

To follow along with the complete code for missing data imputation, use the `03 - Handling Missing Data (Long Gaps).ipynb`

notebook in the `chapter02` *folder.*

Let's read blocks 0-7 `parquet` from memory:

```
block_df = pd.read_parquet("data/london_smart_meters/pre
```

The data that we have saved is in compact form. We need to convert it into expanded form because it is easier to work with time series data in that form. Since we only need a subset of the time series (for faster demonstration purposes), we are just extracting one block from these seven blocks. To convert compact form into expanded form, we can use a helpful function in `src/utils/data_utils.py` called `compact_to_expanded`:

```
#Converting to expanded form
exp_block_df = compact_to_expanded(block_df[block_df.fil
static_cols = ["frequency", "series_length", "stdorToU",
time_varying_cols = ['holidays', 'visibility', 'windBear
       'pressure', 'apparentTemperature', 'windSpeed', '
       'humidity', 'summary'],
ts_identifier = "LCLid")
```

One of the best ways to visualize the missing data in a group of related time series is by using a very helpful package called `missingno`:

```
# Pivot the data to set the index as the datetime and th
plot_df = pd.pivot_table(exp_block_df, index="timestamp"
# Generate Plot. Since we have a datetime index, we can
msno.matrix(plot_df, freq="M")
```

The preceding code produces the following output:

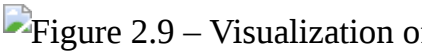


Figure 2.9 – Visualization of the missing data in block 7

IMPORTANT NOTE

> *Only attempt the* `missingno` *visualization on related time series where there are less than 25 time series. If you have a dataset that contains thousands of time series (such as in our full dataset), applying this visualization will give us an illegible plot and a frozen computer.*

This visualization tells us a lot of things at a single glance. The *Y*-axis contains the dates that we are plotting the visualization for, while the *X*-axis contains the columns, which in this case are the different households. We know that all the time series are not perfectly aligned – that is, not all of them start at the same time and end at the same time. The big white gaps we can see at the beginning of many of the time series show that data collection for those consumers started later than the others. We can also see that a few time series finish earlier than the rest, which means either they stopped being consumers or the measurement phase stopped. There are also a few smaller white lines in many time series, which are real missing values. We can also notice a sparkline to the right, which is a compact representation of the number of missing columns for each row. If there are no missing values (all time series have some value), then the sparkline would be at the far right. Finally, if there are a lot of missing values, the line will be to the left.

Just because there are missing values, we are not going to fill/impute them because the decision of whether to impute missing data or not comes later in the workflow. For

some models, we do not need to do the imputation, while for others, we do. There are multiple ways of imputing missing data and which one to choose is another decision we cannot make beforehand.

So, for now, let's pick one `LCLid` and dig deeper. We already know that there are some missing values between 2012-09-30 and 2012-10-31. Let's visualize that period:

```
# Taking a single time series from the block
ts_df = exp_block_df[exp_block_df.LCLid=="MAC000193"].se
msno.matrix(ts_df["2012-09-30": "2012-10-31"], freq="D")
```

The preceding code produces the following output:

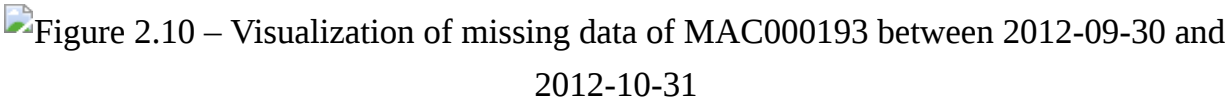


Figure 2.10 – Visualization of missing data of MAC000193 between 2012-09-30 and 2012-10-31

Here, we can see that the missing data is really between 2012-10-18 and 2012-10-19. Normally, we would go ahead and impute the missing data in this period, but since we are looking at this with an academic lens, we will take a slightly different route. Let's introduce an artificial missing data section and see how the different techniques we are going to look at impute the missing data:

```
# The dates between which we are nulling out the time se
window = slice("2012-10-07", "2012-10-08")
# Creating a new column and artificially creating missin
```

```
ts_df['energy_consumption_missing'] = ts_df.energy_consu
ts_df.loc[window, "energy_consumption_missing"] = np.nan
```

Now, let's plot the missing area in the time series:

Figure 2.11 – The energy consumption of MAC000193 between 2012-10-05 and 2012-10-10

Figure 2.11 – The energy consumption of MAC000193 between 2012-10-05 and 2012-10-10

We are missing 2 whole days of energy consumption readings, which means there are 96 missing data points (half-hourly). If we use one of the techniques we saw earlier, such as interpolation, we will see that it will mostly be a straight line because none of the methods are complex enough to capture the pattern over a long time.

There are a few techniques that we can use to fill in such large missing gaps in data. We will cover these now.

Imputing with the previous day

Since this is a half-hourly time series of energy consumption, it stands to reason that there might be a pattern that is repeating day after day. The energy consumption between 9:00 A.M. and 10:00 A.M. might be higher as everybody gets ready to go to the office and a slump during the day when most houses may be empty. So, the simplest way to fill in the missing data would be to use the last day energy readings so that the energy reading at 10:00 A.M. 2012-10-18 can be filled with the energy reading at 10:00 A.M. 2012-10-17:

```
#Shifting 48 steps to get previous day
ts_df["prev_day"] = ts_df['energy_consumption'].shift(48
#Using the shifted column to fill missing
ts_df['prev_day_imputed'] =  ts_df['energy_consumption_m
ts_df.loc[null_mask,"prev_day_imputed"] = ts_df.loc[null
mae = mean_absolute_error(ts_df.loc[window, "prev_day_im
```

Let's see what the imputation looks like:

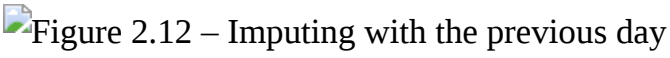


Figure 2.12 – Imputing with the previous day

While this looks better, this is also very brittle. When we are copying the previous day, we are also assuming that any kind of variation or anomalous behavior is also repeated. We can already see that the patterns for the day before and the day after are not the same.

Hourly average profile

A better approach would be to calculate an hourly profile from the data – the mean consumption for every hour – and use the average to fill the missing data:

```
#Create a column with the Hour from timestamp
ts_df["hour"] = ts_df.index.hour
#Calculate hourly average consumption
hourly_profile = ts_df.groupby(['hour'])['energy_consump
hourly_profile.rename(columns={"energy_consumption": "ho
#Saving the index because it gets lost in merge
idx = ts_df.index
```

```
#Merge the hourly profile dataframe to ts dataframe
ts_df = ts_df.merge(hourly_profile, on=['hour'], how='le
ts_df.index = idx
#Using the hourly profile to fill missing
ts_df['hourly_profile_imputed'] = ts_df['energy_consumpt
ts_df.loc[null_mask,"hourly_profile_imputed"] = ts_df.lo
mae = mean_absolute_error(ts_df.loc[window, "hourly_prof
```

Let's see if this is better:

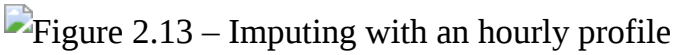


Figure 2.13 – Imputing with an hourly profile

This is giving us a much more generalized curve that does not have the spikes that we saw for the individual days. The hourly ups and downs have also been captured as per our expectations. The `mean absolute error` (`MAE`) is also lower than before.

The hourly average for each weekday

We can further refine this rule by introducing a specific profile for each weekday. It stands to reason that the usage pattern on a weekday is not going to be the same on a weekend. Hence, we can calculate the average hourly consumption for each weekday separately so that we have one profile for Monday, another for Tuesday, and so on:

```
#Create a column with the weekday from timestamp
ts_df["weekday"] = ts_df.index.weekday
#Calculate weekday-hourly average consumption
day_hourly_profile = ts_df.groupby(['weekday','hour'])['
day_hourly_profile.rename(columns={"energy_consumption":
```

```
#Saving the index because it gets lost in merge
idx = ts_df.index
#Merge the day-hourly profile dataframe to ts dataframe
ts_df = ts_df.merge(day_hourly_profile, on=['weekday', '
ts_df.index = idx
#Using the day-hourly profile to fill missing
ts_df['day_hourly_profile_imputed'] = ts_df['energy_cons
ts_df.loc[null_mask,"day_hourly_profile_imputed"] = ts_d
mae = mean_absolute_error(ts_df.loc[window, "day_hourly_
```

Let's see what this looks like:

Figure 2.14 – Imputing the hourly average for each weekday

This looks very similar to the other one, but this is because the day we are imputing is a weekday and the weekday profiles are similar. The MAE is also lower than the day profile. The weekend profile is slightly different, which you can see in the associated Jupyter Notebook.

Seasonal interpolation

Although calculating seasonal profiles and using them to impute works well, there are instances, especially when there is a trend in the time series, where such a simple technique falls short. The simple seasonal profile doesn't capture the trend at all and ignores it completely. For such cases, we can do the following:

1. Calculate the seasonal profile, similar to how we calculated the averages earlier.

2. Subtract the seasonal profile and apply any of the interpolation techniques we saw earlier.
3. Add the seasonal profile back to the interpolated series.

This process has been implemented in this book's GitHub repository in the `src/imputation/interpolation.py` file. We can use it as follows:

```
from src.imputation.interpolation import SeasonalInterpo
# Seasonal interpolation using 48*7 as the seasonal peri
recovered_matrix_seas_interp_weekday_half_hour = Seasona
ts_df['seas_interp_weekday_half_hour_imputed'] = recover
```

The key parameter here is `seasonal_period`, which tells the algorithm to look for patterns that repeat every `seasonal_period`. If we mention `seasonal_period=48`, it will look for patterns that repeat every 48 data points. In our case, they are after each day (because we have 48 half-hour timesteps in a day). In addition to this, we need to specify what kind of interpolation we need to perform.

ADDITIONAL INFORMATION

Internally, we use something called seasonal decomposition (`statsmodels.tsa.seasonal.seasonal_decompose`), which will be covered in Chapter 3, Analyzing and Visualizing Time Series Data, to isolate the seasonality component.

Here, we have done seasonal interpolation using 48 (half-hourly) and 48*7 (weekday to half-hourly) and plotted the resulting imputation:

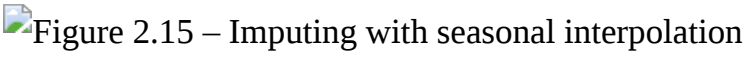


Figure 2.15 – Imputing with seasonal interpolation

Here, we can see that both have captured the seasonality patterns, but the half-hourly profile every weekday has captured the peaks in the first day better, so they have a lower MAE. There is no improvement in terms of hourly averages, mostly because there is no strong increasing or decreasing patterns in the time series.

With this, we have come to the end of this chapter. We are now officially into the nitty-gritty of juggling time series data, cleaning it, and processing it. Congratulations on finishing this chapter!

Summary

After a short refresher on pandas DataFrames, especially on the datetime manipulations and simple techniques for handling missing data, we learned about the two forms of storing and working with time series data – compact and expanded. With all this knowledge, we took our raw dataset and built a pipeline to convert it into compact form. If you have run the accompanying notebook, you should have the preprocessed dataset saved on disk. We also had an in-depth look at some techniques for handling long gaps of missing data.

Now that we have the processed datasets, in the next chapter, we will learn how to visualize and analyze a time series dataset.

3 Analyzing and Visualizing Time Series Data

Join our book community on Discord

https://packt.link/EarlyAccess/

In the previous chapter, we learned where to obtain time series datasets, as well as how to manipulate time series data using **pandas**, handle missing values, and so on. Now that we have the processed time series data, it's time to understand the dataset, which data scientists call **Exploratory Data Analysis** (**EDA**). It is a process by which the data scientist analyzes the data by looking at aggregate statistics, feature distributions, visualizations, and so on to try and uncover patterns in the data that they can leverage in modeling. In this chapter, we will look at a couple of ways to analyze a time series dataset, a few specific techniques that are tailor-made for time series, and review some of the visualization techniques for time series data.

In this chapter, we will cover the following topics:

- Components of a time series

- Visualizing time series data
- Decomposing a time series
- Detecting and treating outliers

Technical requirements

You will need to set up the **Anaconda** environment following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks.

You will need to run **02 - Preprocessing London Smart Meter Dataset.ipynb** notebook from **Chapter02** folder.

The code for this chapter can be found at https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-/tree/main/notebooks/Chapter03.

Components of a time series

Before we start analyzing and visualizing time series, we need to understand the structure of a time series. Any time series can contain some or all of the following components:

1. **Trend**
2. **Seasonal**

3. **Cyclical**
4. **Irregular**

These components can be mixed in different ways, but two very commonly assumed ways are *additive* (*Y = Trend + Seasonal + Cyclical + Irregular*) and *multiplicative* (*Y = Trend * Seasonal * Cyclical * Irregular*).

The trend component

The **trend** is a long-term change in the mean of a time series. It is the smooth and steady movement of a time series in a particular direction. When the time series moves upward, we say there is an *upward or increasing trend*, while when it moves downward, we say there is a *downward or decreasing trend*. At the time of writing, if we think about the revenue of Tesla over the years, as shown in the following figure, we can see that it has been increasing consistently for the last few years:

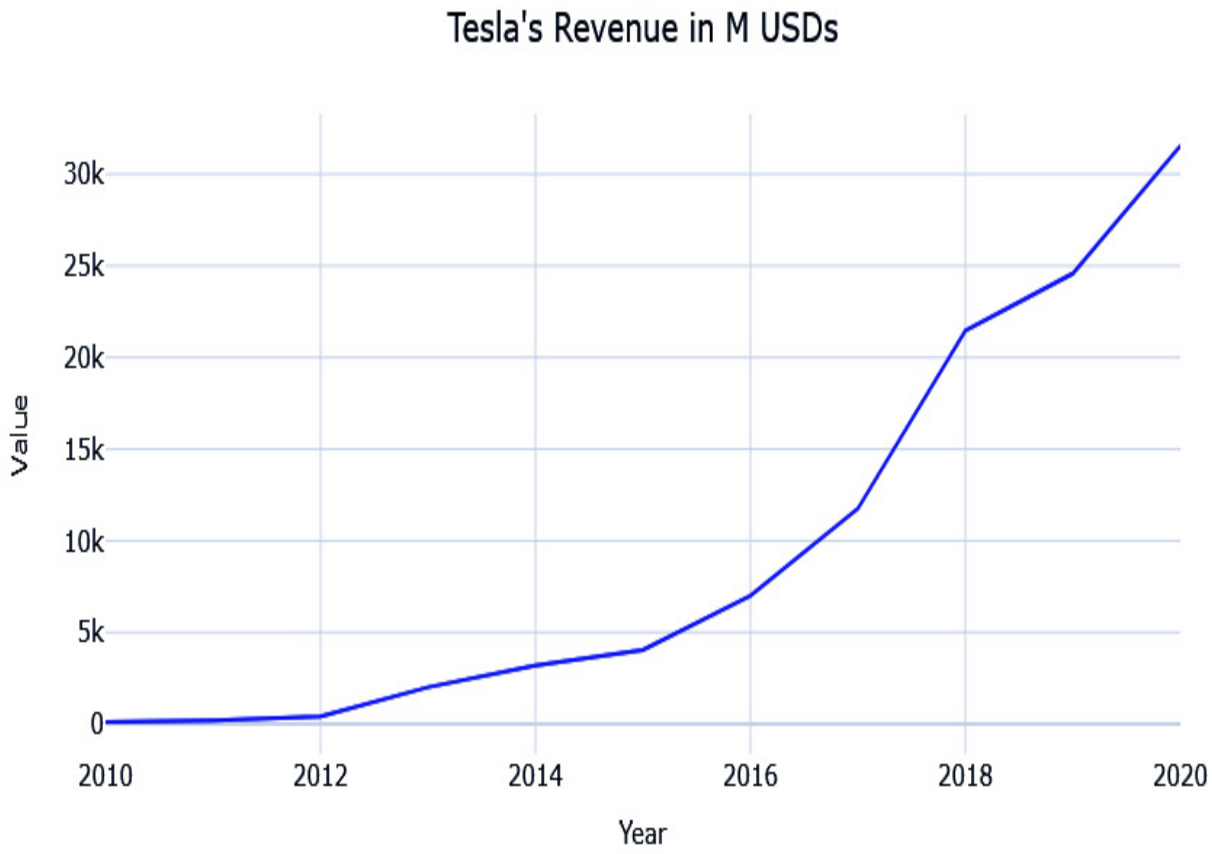


Figure 3.1 – Tesla's revenue in millions of USD

Looking at the preceding figure, we can say that Tesla's revenue is having an increasing trend. The trend doesn't need to be linear; it can also be non-linear.

The seasonal component

When a time series exhibits regular, repetitive, up-and-down fluctuations, we call that **seasonality**. For instance, retail sales typically shoot up during the holidays, specifically Christmas in western countries. Similarly,

electricity consumption peaks during the summer months in the tropics and the winter months in colder countries. In all these examples, you can see a specific up-and-down pattern repeating every year. Another example is sunspots, as shown in the following figure:

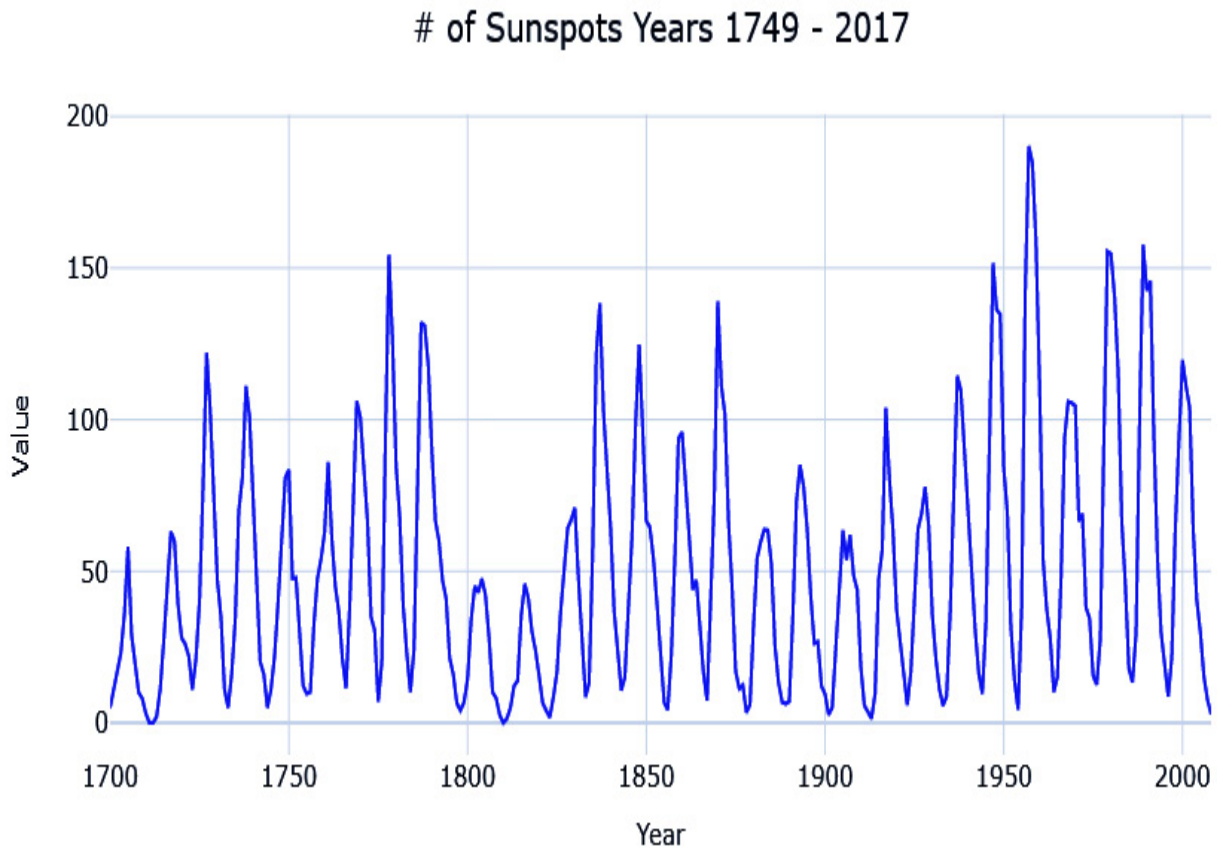


Figure 3.2 – Number of sunspots from 1749 to 2017

As you can see, sunspots peak every 11 years.

The cyclical component

The **cyclical** component is often confused with seasonality, but it stands apart due to a very subtle difference. Like seasonality, the cyclical component also exhibits a similar up-and-down pattern around the trend line, but instead of repeating the pattern every period, the cyclical component is irregular. A good example of this is economic recession, which happens over a 10-year cycle. However, this doesn't happen like clockwork; sometimes, it can be fewer or more than every 10 years.

The irregular component

This component is left after removing the trends, seasonality, and cyclicity from a time series. Traditionally, this component is considered *unpredictable* and is also called the *residual* or *error term*. In common classical statistics-based models, the point of any "model" is to capture all the other components to the point that the only part that is not captured is the irregular component. In modern machine learning, we do not consider this component entirely unpredictable. We try to capture this component, or parts of it, by using exogenous variables. For instance, the irregular component of retail sales may be explained as the different promotional activities they run. When we have this additional information, the "unpredictable" component starts to become predictable again. But no matter how many additional variables you add to the model, there will always be some component, which is the true irregular component (or true error), that is left behind.

Now that we know what the different components of a time series are, let's see how we can visualize them.

Visualizing time series data

In Chapter 2, *Acquiring and Processing Time Series Data*, we learned how to prepare a data model as a first step toward analyzing a new dataset. If preparing a data model is like approaching someone you like and making that first contact, then EDA is like dating that person. At this point, you have the dataset, and you are trying to get to know them, trying to figure out what makes them tick, what the person likes and dislikes, and so on.

EDA often employs visualization techniques to uncover patterns, spot anomalies, form and test hypotheses, and so on. Spending some time understanding your dataset will help you a lot when you are trying to squeeze out every last bit of performance from the models. You may understand what sort of features you must create, or what kind of modeling techniques should be applied, and so on.

In this chapter, we will cover a few visualization techniques that are well suited for time series datasets.

> *NOTEBOOK ALERT*
>
> *To follow along with the complete code for visualizing time series, use the* **01-Visualizing Time Series.ipynb** *notebook in*

the **chapter03** *folder.*

Line charts

This is the most basic and common visualization that is used for understanding a time series. We just plot the time on the *X*-axis and the time series value on the *Y*-axis. Let's see what it looks like if we plot one of the households from our dataset:

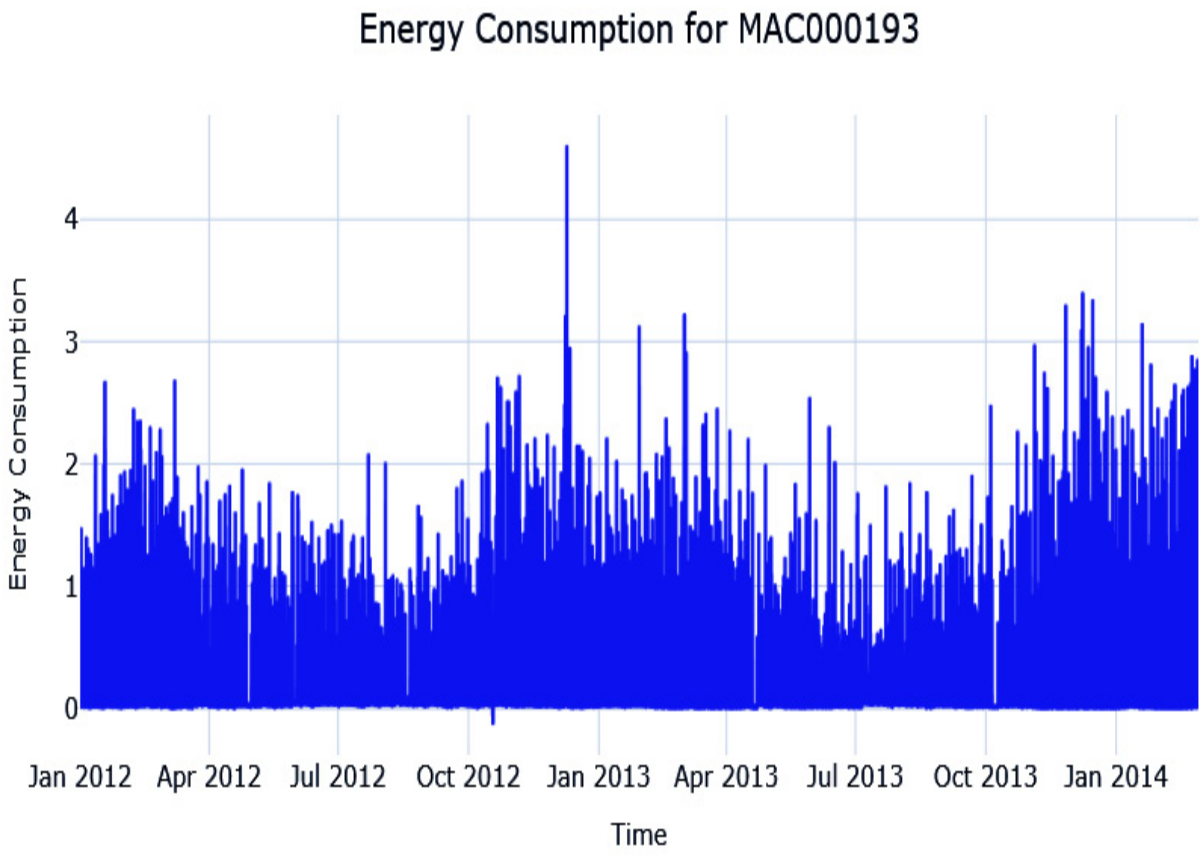


Figure 3.3 – Line plot of household MAC000193

When you have a long time series with high variation, as we have, the line plot can get a bit chaotic. One of the options to get a macroview of the time series in terms of trends and movement is to plot a smoothed version of the time series. Let's see what a rolling monthly average of the time series look like:

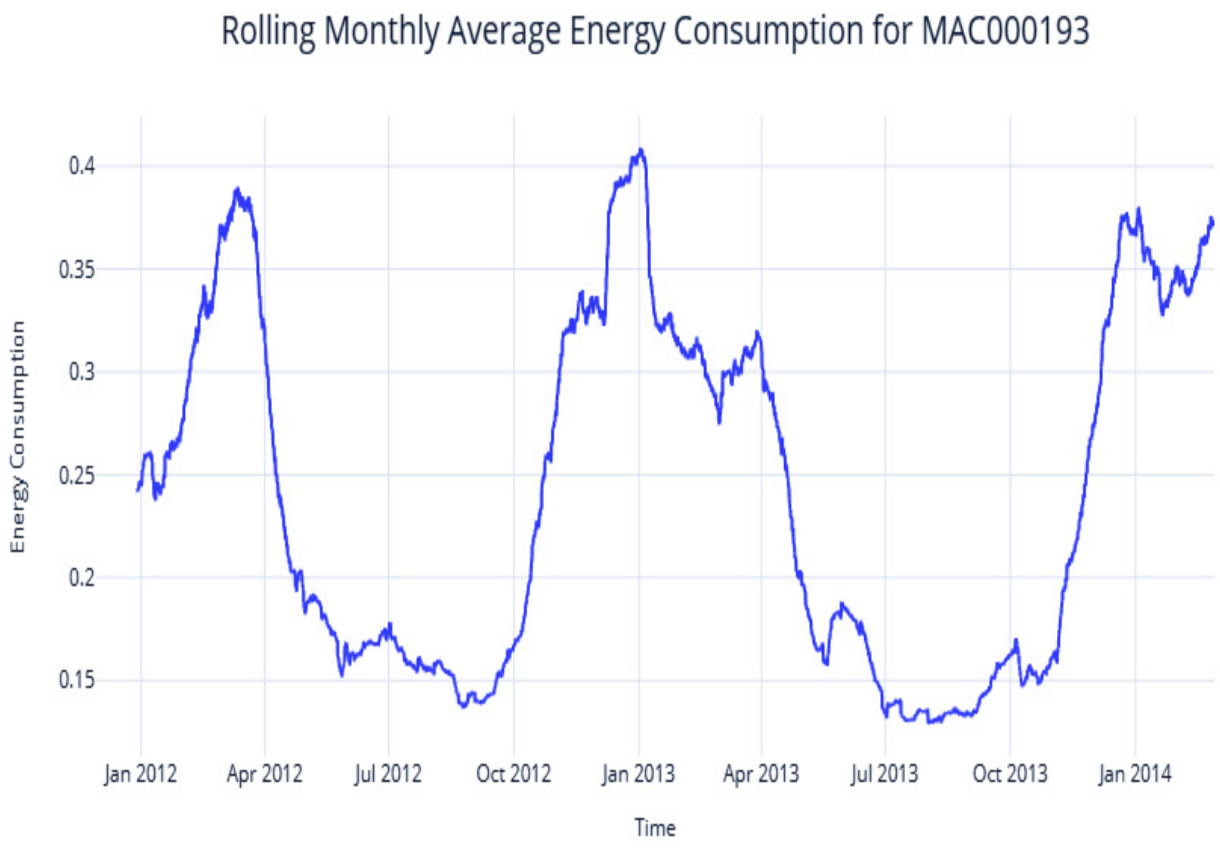
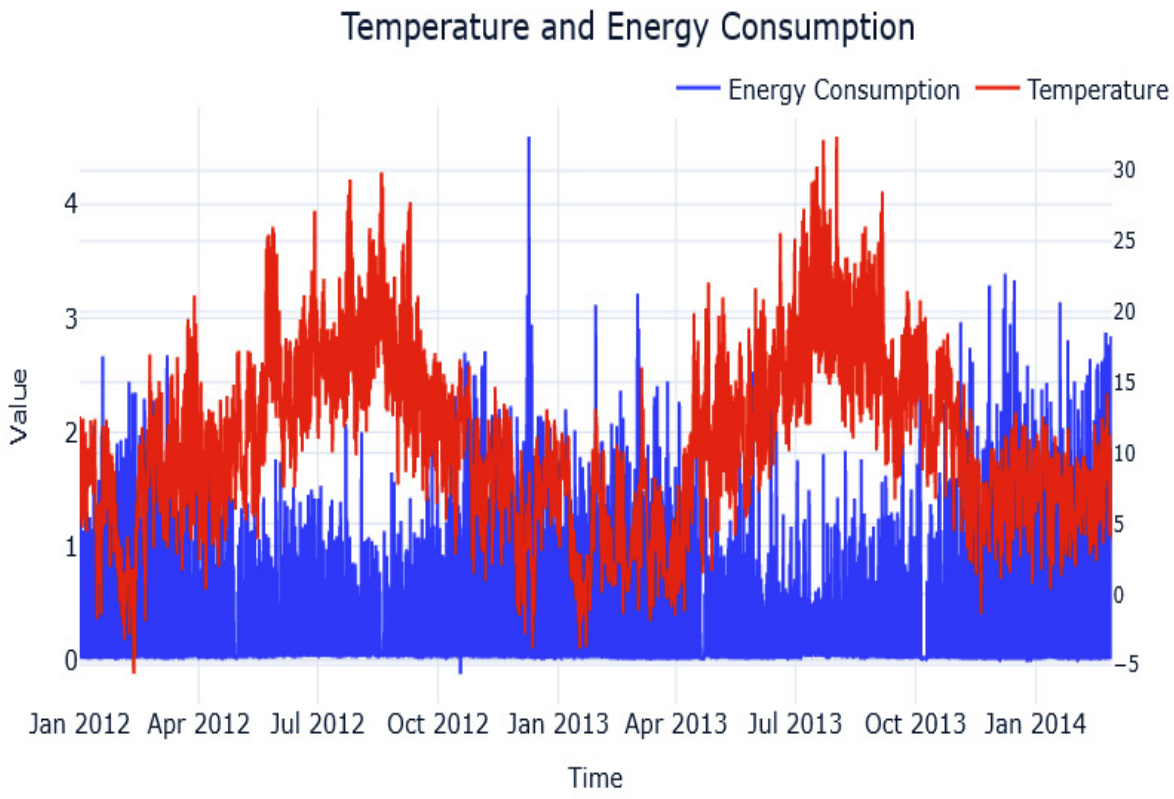


Figure 3.4 – Rolling monthly average energy consumption of household MAC000193

We can see the macro patterns much more clearly now. The seasonality is clear – the series peaks in winter and troughs during summer. And if you

think about it critically, it makes sense. This is London we are talking about, and the energy consumption would be higher during the winter because of lower temperatures and subsequent heating system usage. For a household in the tropics, for example, the pattern may be reversed, with the peaks coming in summer when air conditioners come into play.

Another use for the line chart is to visualize two or more time series together and investigate any correlations between them. In our case, let's try plotting the temperature along with the energy consumption and see if the hypothesis we have about temperature influencing energy consumption holds good:



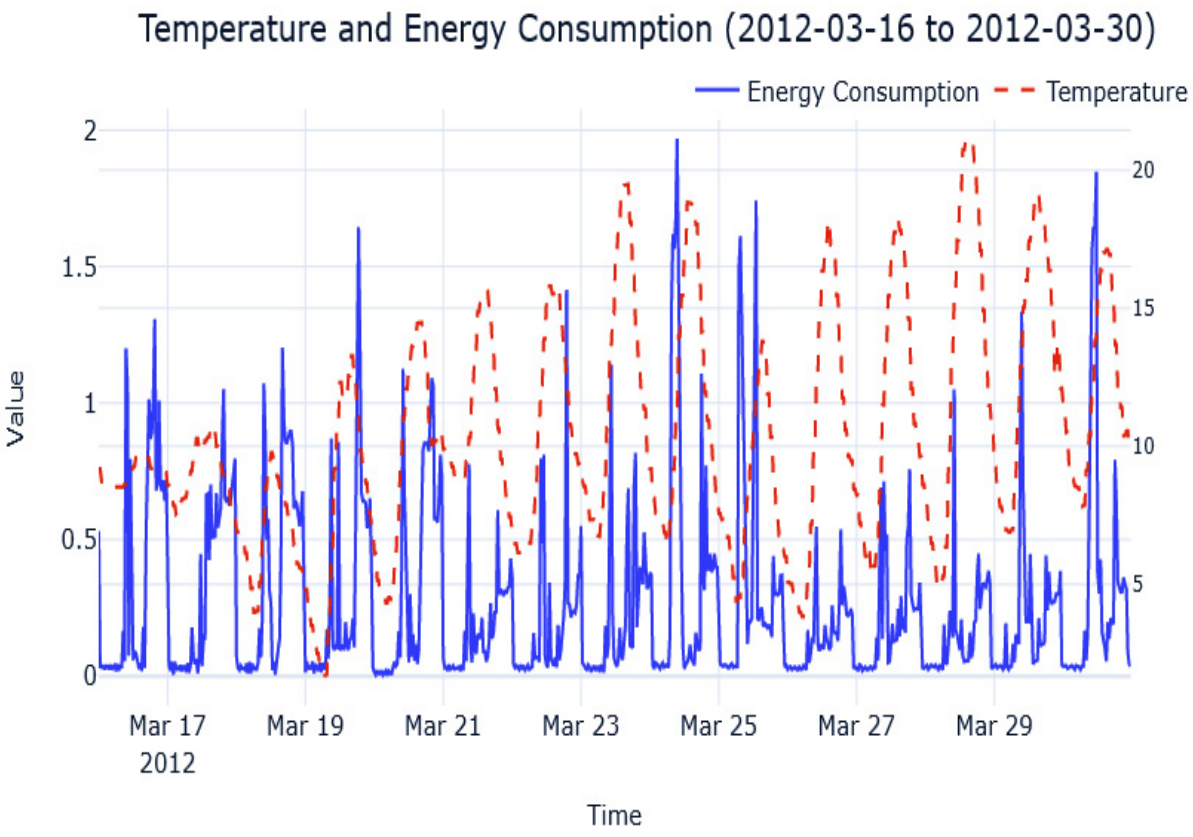


Figure 3.5 – Temperature and energy consumption (zoomed-in plot at the bottom) (Color Figure)

Here, we can see a clear negative correlation in yearly resolution between energy consumption and temperature. Winters show higher energy consumption on a macro scale. We can also see the daily patterns that are loosely correlated with temperature, but maybe because of other factors such as people coming back home after work and so on.

There are a few other visualizations that are more suited to bringing out seasonality in a time series. Let's take a look.

Seasonal plots

A **seasonal plot** is very similar to a line plot, but the key difference here is that the *X*-axis denotes the "seasons", the *Y*-axis denotes the time series value, and the different seasonal cycles are represented in different colors or line types. For instance, the yearly seasonality at a monthly resolution can be depicted with months on the *X*-axis and different years in different colors.

Let's see what this looks like for our household in question. Here, we have plotted the average monthly energy consumption across multiple years:

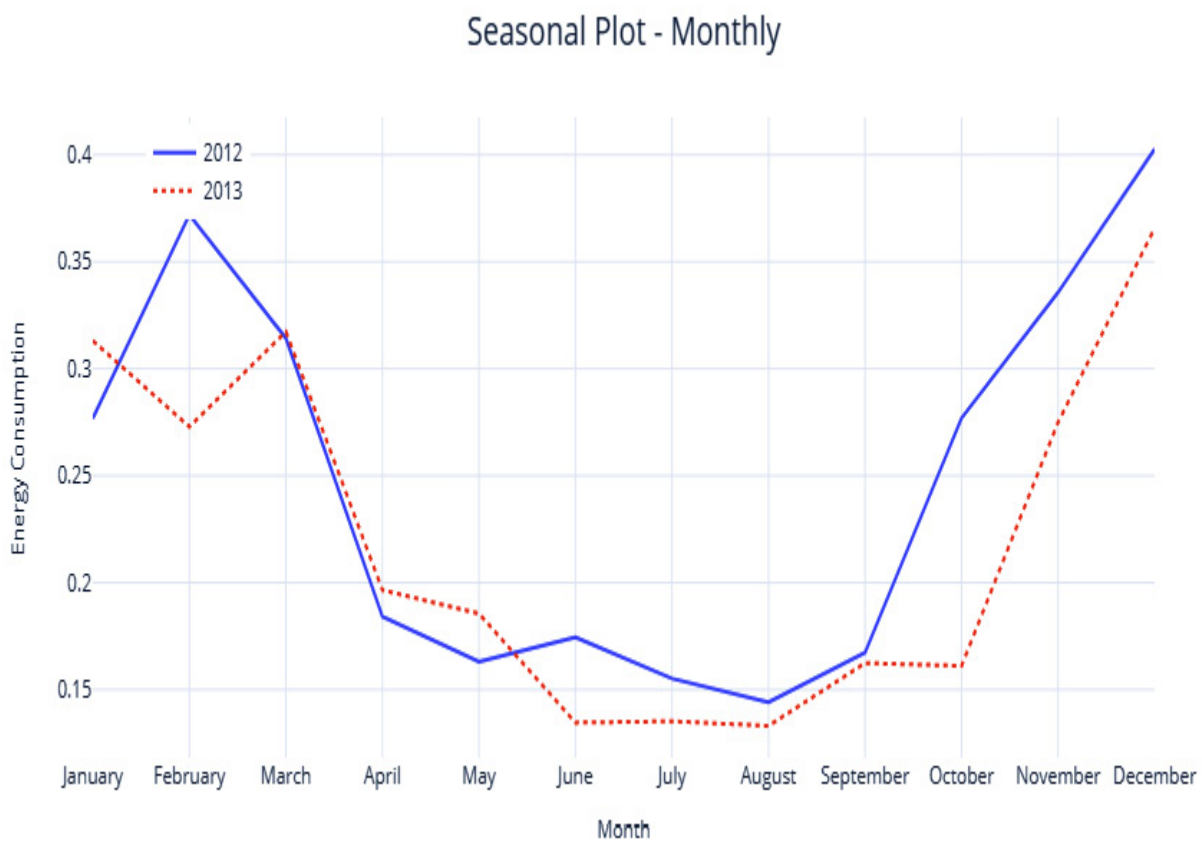


Figure 3.6 – Seasonal plot at a monthly resolution(Color Figure)

We can instantly see the appeal in this visualization because it lets us visualize the seasonality pattern easily. We can see that the consumption goes down in the summer months and we can also see that it happens consistently across multiple years. In the 2 years that we have data for, we can see that in October, the behavior in 2013 slightly deviated from 2012. Maybe there is something else that can help us explain this difference – what about temperature? We can also plot the seasonal plots with another variable of interest, such as the temperature:

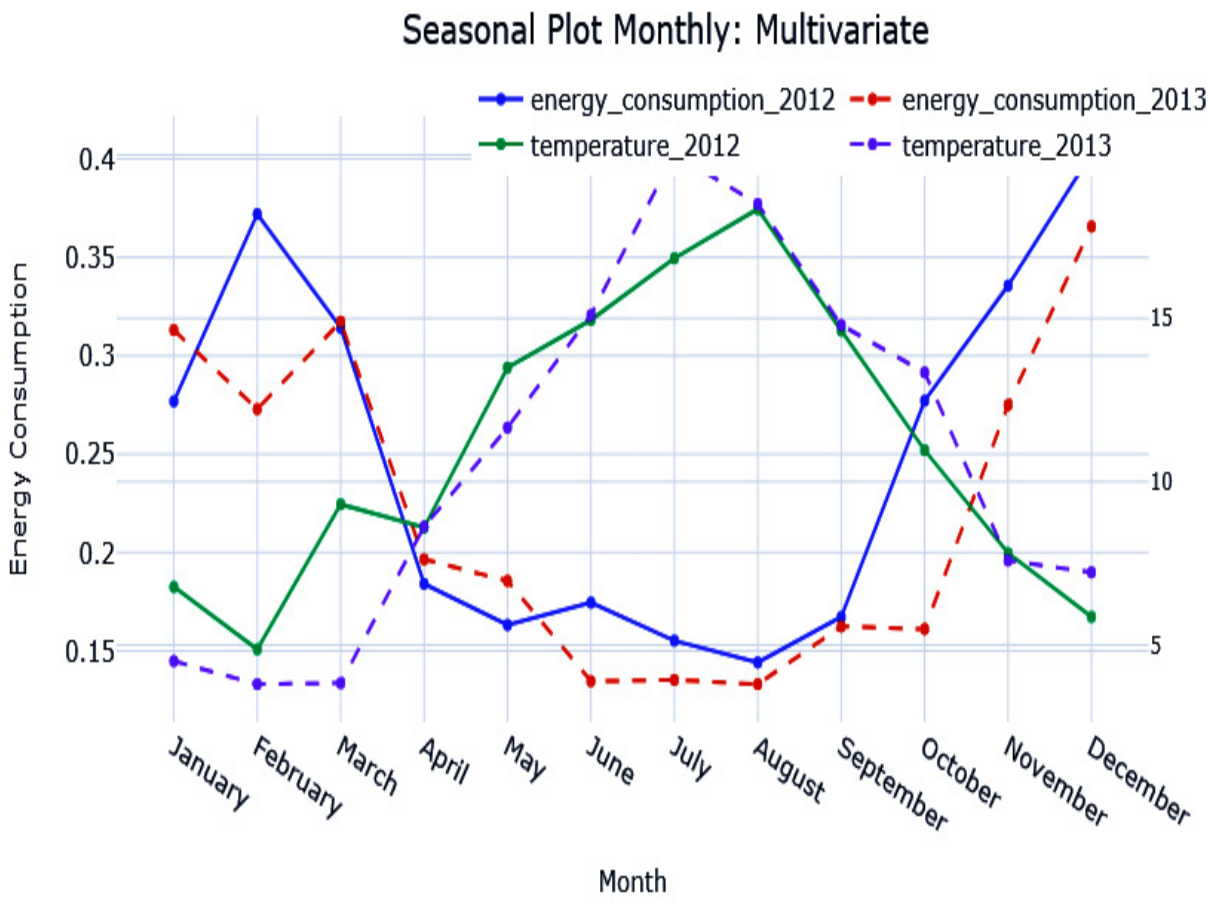


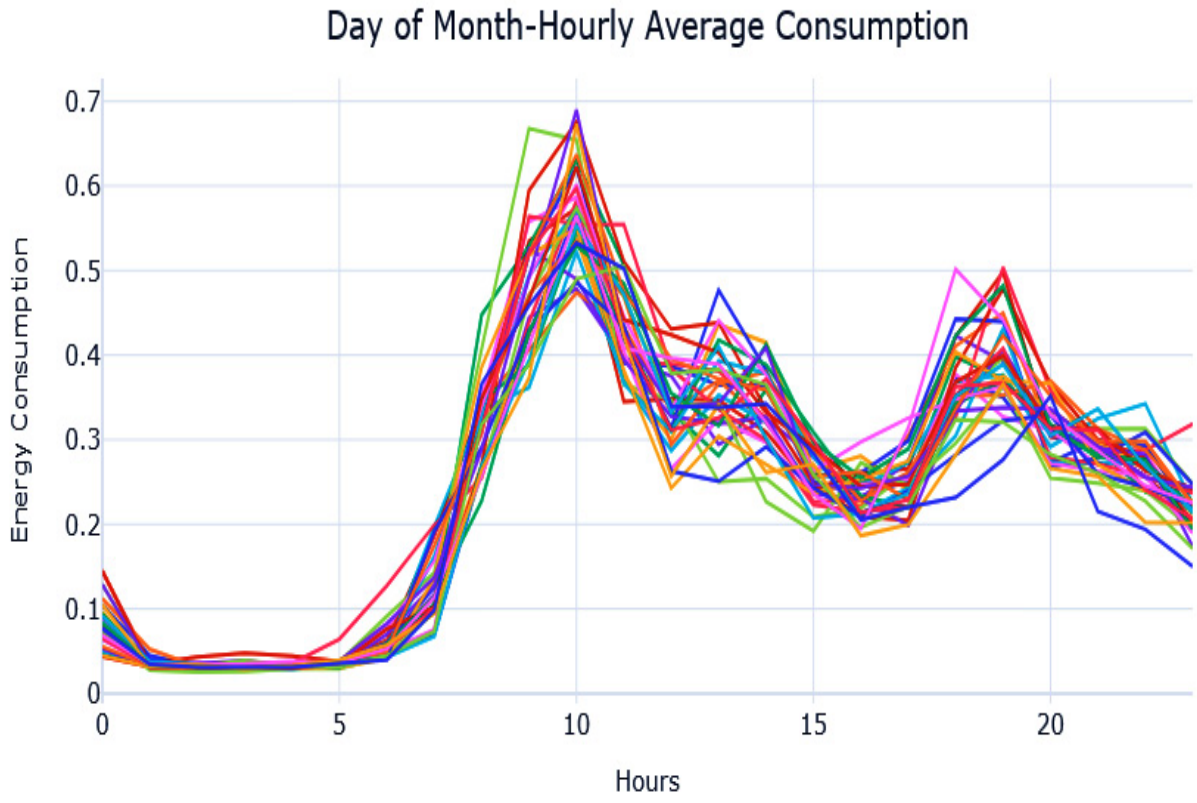
Figure 3.7 – Seasonal plot at a monthly resolution (energy consumption versus temperature) (Color Figure)

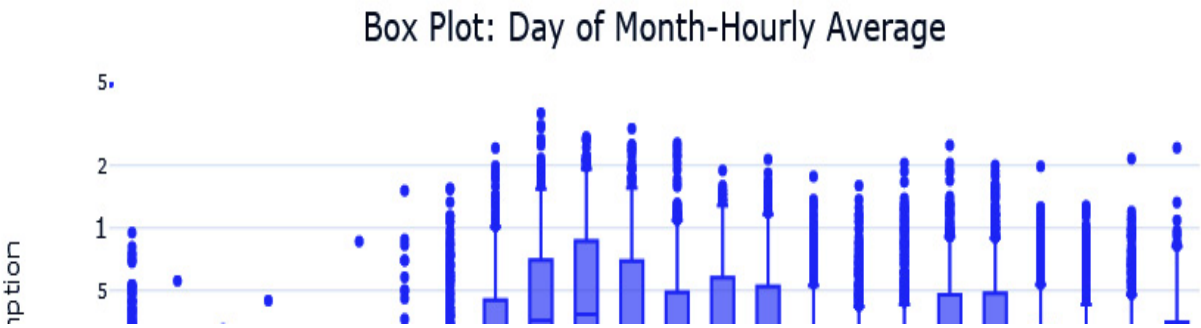
Notice October? In October 2013, the temperature stayed warmer for 1 month more, hence why the energy consumption pattern was slightly different from last year.

We can plot these kinds of plots at other resolutions as well, such as hourly seasonality. But when there are too many seasonal cycles to be plotted, it increases visual clutter. An alternative to a seasonal plot is a seasonal box plot.

Seasonal box plots

Instead of plotting the different seasonal cycles in different colors or line types, we can represent them as a box plot. This instantly clears up the clutter in the plot. The additional benefit you get from this representation is that it lets us understand the variability across seasonal cycles:





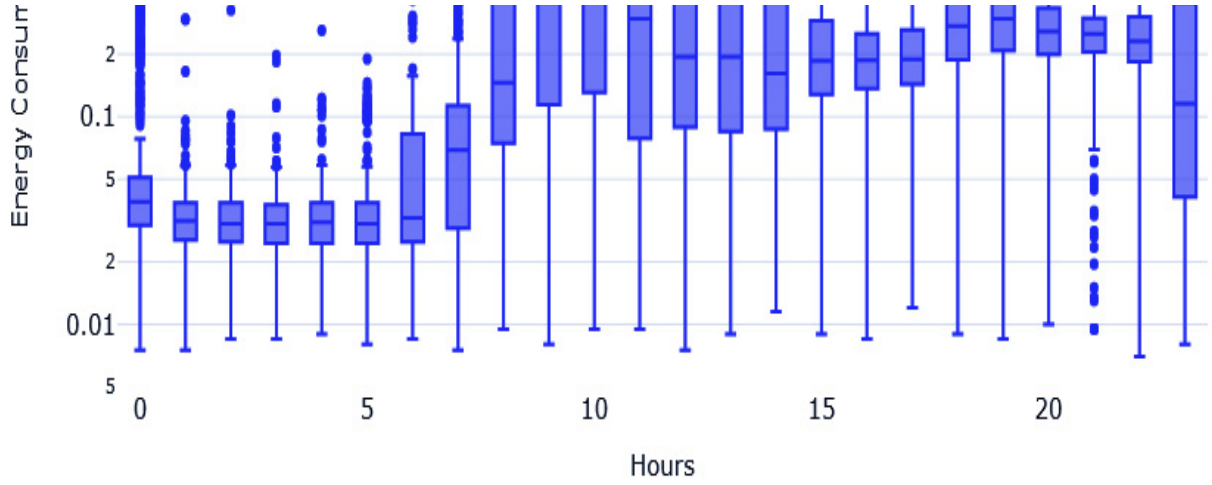


Figure 3.8 – Seasonal plot (top) and seasonal box plot (bottom) at an hourly resolution(Color Figure)

Here, we can see that the seasonal plot at this resolution is too cluttered to make out the pattern and the variation across seasonal cycles. However, the seasonal box plot is much more informative. The horizontal line in the box tells us about the median, the box is the **interquartile range** (**IQR**), and the points that are marked are the outliers. By looking at the medians, we can see that the peak consumption occurs from 9 A.M. onward. But the variability is also higher from 9 A.M. If you plot separate box plots for each week, for example, you will see that the patterns are slightly different on Sundays (additional visualizations are in the associated notebook).

However, there is another visualization that lets you inspect these patterns along two dimensions.

Calendar heatmaps

Instead of having separate box plots or separate line charts for each week of the day, it would be useful if we could condense that information into a single plot. This is where **calendar heatmaps** come in. A calendar heatmap uses colored cells in a rectangular block to represent the information. Along the two sides of the rectangle, we can find two separate granularities of time, such as month and year. In each intersection, the cell is colored relative to the value of the time series at that intersection.

Let's look at the hourly average energy consumption across the different weekdays in a calendar heatmap:

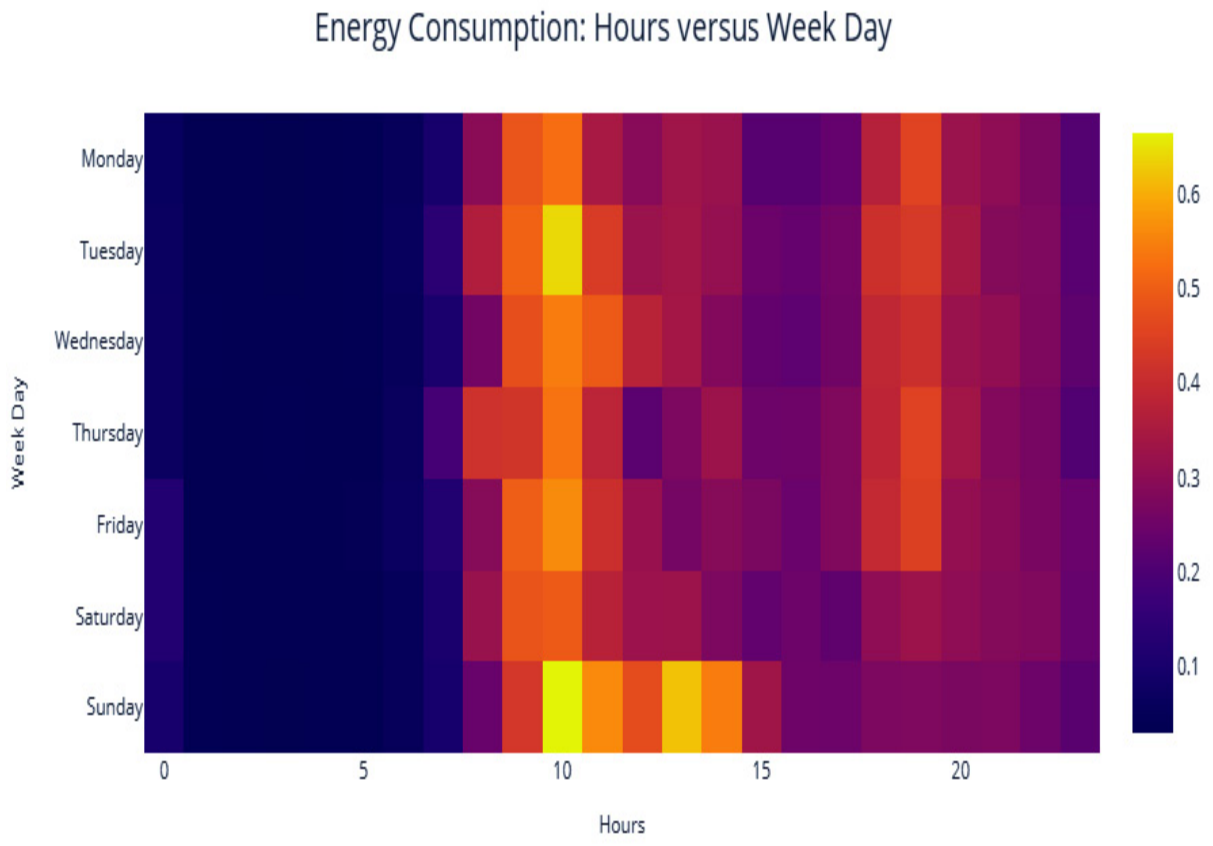


Figure 3.9 – A calendar heatmap for energy consumption(Color Figure)

From the color scale on the right, we know that lighter colors mean higher values. We can see how Monday to Saturday have similar peaks – that is, once in the morning and once in the evening. However, Sunday has a slightly different pattern, with higher consumption throughout the day.

So far, we've reviewed a lot of visualizations that can bring out seasonality. Now, let's look at a visualization for inspecting autocorrelation.

Autocorrelation plot

If correlation indicates the strength and direction of the linear relationship between two variables, autocorrelation is the correlation between the values of a time series in successive periods. Most time series have a heavy dependence on the value in the previous period, and this is a critical component in a lot of the forecasting models we will be seeing as well. Something such as ARIMA (which we will briefly look at in Chapter 4, *Setting a Strong Baseline Forecast*) is built on autocorrelation. So, it's always helpful to just visualize and understand how strong the dependence on previous time steps is.

This is where **autocorrelation plots** come in handy. In such plots, we have the different lags (*t-1*, *t-2*, *t-3*, and so on) on the *X*-axis and the correlations between *t* and the different lags on the *Y*-axis. In addition to autocorrelation, we can also look at **partial autocorrelation**, which is very similar to

autocorrelation but with one key difference: partial autocorrelation removes any indirect correlation that may be present before presenting the correlations. Let's look at an example to understand this. If *t* is the current time step, let's assume *t-1* is highly correlated to *t*. So, by extending this logic, *t-2* will be highly correlated with *t-1* and because of this correlation, the autocorrelation between *t* and *t-2* would be high. However, partial autocorrelation corrects this and extracts the correlation, which can be purely attributed to *t-2* and *t*.

One thing we need to keep in mind is that the autocorrelation and partial autocorrelation analysis works best if the time series is stationary (we will talk about stationarity in detail in Chapter 6, *Feature Engineering for Time Series Forecasting*).

BEST PRACTICE

> *There are many ways of making a series stationary, but a quick and dirty way is to use seasonal decomposition and just pick the residuals. It should be devoid of trends and seasonality, which are the major drivers of non-stationarity in a time series. But as we will see later in this book, this is not a foolproof method of making a series stationary in the truest sense.*

Now, let's see what these plots look like for our household from the dataset (after making it stationary):

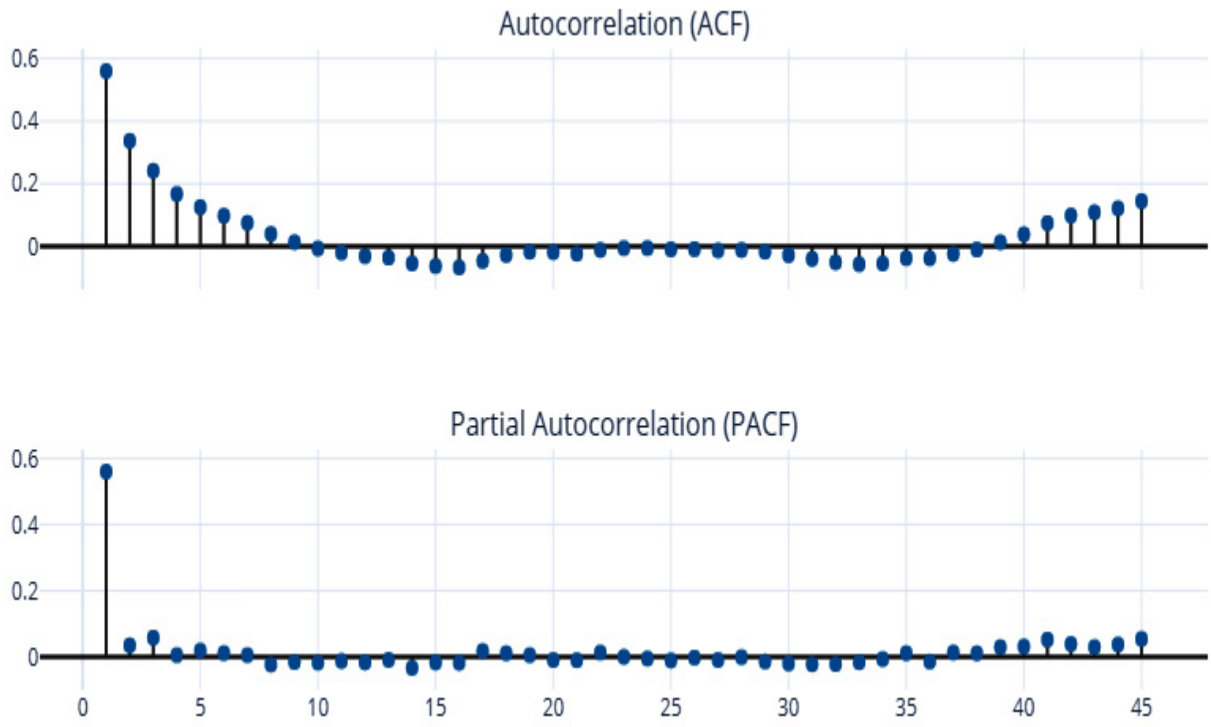


Figure 3.10 – Autocorrelation and partial autocorrelation plots

Here, we can see that the first lag (*t-1*) has the most influence and that its influence quickly drops down to close to zero in the partial autocorrelation plot. This means that the energy consumption of a day is highly correlated with the energy consumption the day before.

If you've seen such charts before, you would have seen an envelope over this showing the confidence intervals as a guide to selecting significant auto-correlations. While that is a good thumb of rule, it's not included here because I don't want you to use it as a rule. The relevance of the confidence intervals depends on some assumptions (normality and so on) which may not be satisfied all the time, especially in real world use cases.

With that, we've looked at the different components of a time series and learned how to visualize a few of them. Now, let's see how we can decompose a time series into its components.

Decomposing a time series

Seasonal decomposition is the process by which we deconstruct a time series into its components – typically, trend, seasonality, and residuals. The general approach for decomposing a time series is as follows:

1. **Detrending**: Here, we estimate the **trend component** (which is the smooth change in the time series) and remove it from the time series, giving us a **detrended time series**.
2. **Deseasonalizing**: Here, we estimate the seasonality component from the detrended time series. After removing the seasonal component, what is left is the residual.

Let's discuss them in detail.

Detrending

Detrending can be done in a few different ways. Two popular ways of doing it are by using **moving averages** and **locally estimated scatterplot smoothing** (**LOESS**) **regression**.

Moving averages

One of the easiest ways of estimating trends is by using a moving average along the time series. It can be seen as a window that is moved along the time series in steps, and at each step, the average of all the values in the window is recorded. This moving average is a smoothed-out time series and helps us estimate the slow change in time series, which is the trend. The downside is that the technique is quite noisy. Even after smoothing out a time series using this technique, the extracted trend will not be smooth; it will be noisy. The noise should ideally reside with the residuals and not the trend (see the trend line shown in *Figure 3.11*).

LOESS

The **LOESS** algorithm, which is also called *locally weighted polynomial regression*, was developed by Bill Cleveland through the 70s to the 90s. It is a non-parametric method that is used to fit a smooth curve onto a noisy signal. We use an ordinal variable that moves between the time series as the independent variable and the time series signal as the dependent variable. For each value in the ordinal variable, the algorithm uses a fraction of the closest points and estimates a smoothed trend using only those points in a weighted regression. The weights in the weighted regression are the closest points to the point in question. This is given the highest weight and it decays as we move farther away from it. This gives us a very effective tool for modeling the smooth changes in the time series (trend).

Deseasonalizing

The seasonality component can also be estimated in a few different ways. The two most popular ways of doing this are by using period-adjusted averages or a Fourier series.

Period adjusted averages

This is a pretty simple technique wherein we calculate a seasonality index for each period in the expected cycle by taking the average values of all such periods over all the cycles. To make that clear, let's look at a monthly time series where we expect an annual seasonality in this time series. So, the up-and-down pattern would complete a full cycle in 12 months, or the seasonality period is 12. In other words, every 12 points in the time series have similar seasonal components. So, we take the average of all January values as the period-adjusted average for January. In the same way, we calculate the period average for all 12 months. At the end of the exercise, we have 12 period averages, and we can also calculate an *average* period average. Now, we can make these period averages into an index by either subtracting the average of all period averages from each of the period averages (for additive) or dividing the average of all period averages from each of the period averages (multiplicative).

Fourier series

In the late 1700s, Joseph Fourier, a mathematician and physicist, while studying heat flow, realized something profound – *any* periodic function can be broken down into a simple series of sine and cosine waves. Let's

dwell on that for a minute. Any periodic function, no matter the shape, curve, or absence of it, or how wildly it oscillates around the axis, can be broken down into a series of sine and cosine waves.

ADDITIONAL INFORMATION

> *For the more mathematically inclined, the original theory proposes to decompose any periodic function into an integral of exponentials. Using Euler's identity,*
>
> $$e^{iy} = cos(y) + i \cdot sin(y)$$
>
> *, we can consider them as a summation of sine and cosine waves. The Further reading section contains a few resources if you want to delve deeper and explore related concepts, such as the Fourier transform.*

It is this property that we use to extract seasonality from a time series because seasonality is a periodic function, and any periodic function can be approximated by a combination of sine and cosine waves. The sine-cosine form of a Fourier series is as follows:

$$s_{N(x)} = \frac{a_0}{2} + \sum_{n=1}^{N}\left(a_n \cdot cos\left(\frac{2\pi}{P} \cdot n \cdot x\right) + b_n \cdot sin\left(\frac{2\pi}{P} \cdot n \cdot x\right)\right)$$

Here,

$$s_N$$

is the *N*-term approximation of the signal, *S*. Theoretically, when *N* is infinite, the resulting approximation is equal to the original signal. *P* is the maximum length of the cycle. We can use this Fourier series, or a few terms from the Fourier series, to model our seasonality. In our application, *P* is the maximum length of the cycle we are trying to model. For instance, for a yearly seasonality for monthly data, the maximum length of the cycle (*P*) is 12. *x* would be an ordinal variable that increases from *1* to *P*. In this example, *x* would be *1*, *2*, *3*, … *12*. Now, with these terms, all that is left to do is find

$$a_n$$

and

$$b_n$$

, which we can do by regressing on the signal.
We've seen that with the right combination of Fourier terms, we can replicate any signal. But the question is, should we? What we want to learn from data is a generalized seasonality profile that does well with unseen data as well. So, we use *N* as a hyperparameter to extract as complex a signal as we want from the data.

This is a good time to brush up on your trigonometry and remember what sine and cosine waves look like. The first Fourier term (*n=1*) is your age-old sine and cosine waves, which complete one full cycle in the maximum cycle length (*P*). As we increase **n**, we get sine and cosine waves that have multiple cycles in the maximum cycle length (*P*). This can be seen in the following figure:

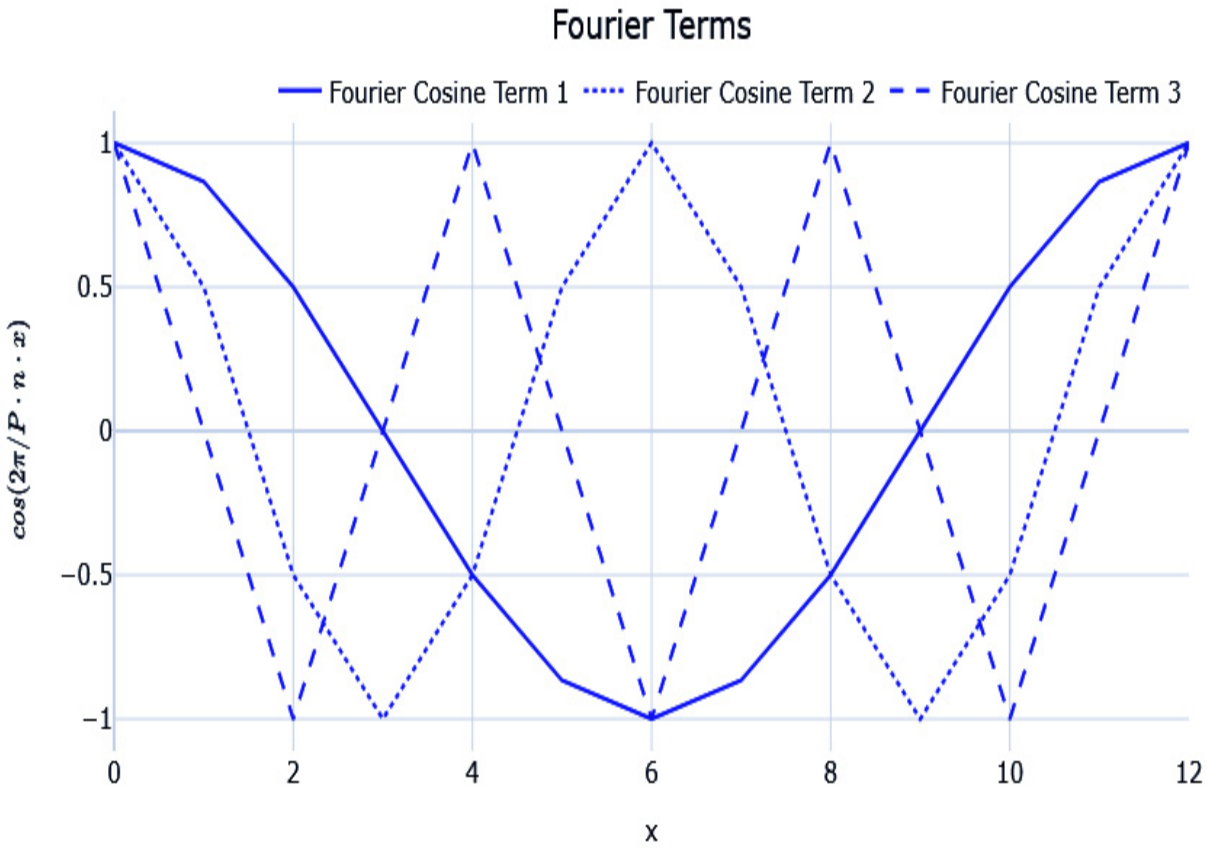


Figure 3.11 – Cosine Fourier terms (n=1, 2, 3)

The sine and cosine waves are complementary to each other, as shown in the following figure:

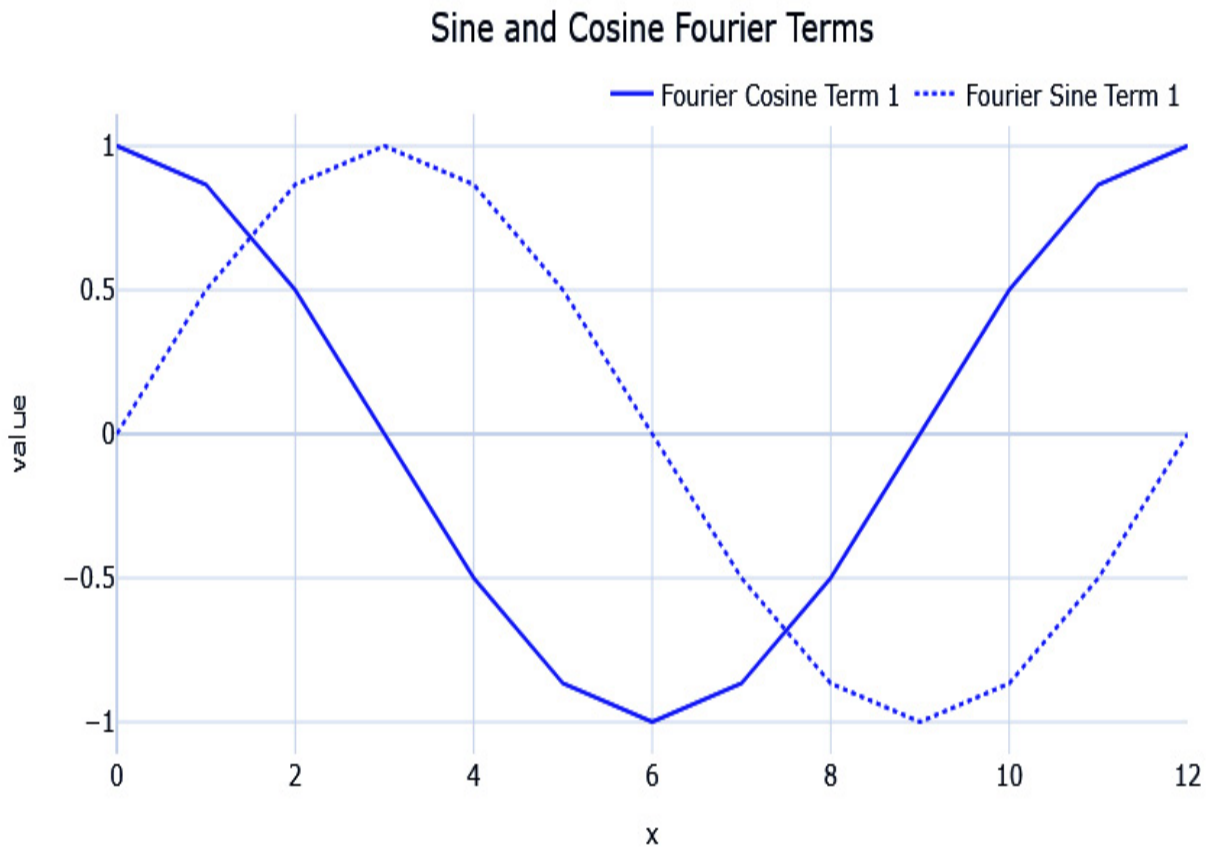


Figure 3.12 – Sine and cosine Fourier terms (n=1)

Now, let's see how we can use this in practice.

Implementations

NOTEBOOK ALERT

To follow along with the complete code for decomposing time series, use the **02-Decomposing Time Series.ipynb** *notebook in the* **chapter03** *folder.*

There are four implementations that we will cover here in the following subsections.

seasonal_decompose from statsmodel

statsmodels.tsa.seasonal has a function called **seasonal_decompose**. This is an implementation that uses moving averages for the trend component and period-adjusted averages for the seasonal component. It supports both additive and multiplicative modes of decomposition. However, it doesn't tolerate missing values. Let's see how we can use it:

#Does not support missing values, so using imputed ts instead

```
res = seasonal_decompose(ts, period=7*48, model='
```

A few key parameters to keep in mind are as follows:

1. **period** is the seasonal period you expect the pattern to repeat.
2. **model** takes **additive** or **multiplicative** as arguments to determine the type of decomposition.
3. **filt** takes in an array that is used as the weights in the moving average (convolution, to be specific). It can also be used to define the window over which we need our moving average. We can increase it to smooth out the trend component to some extent.

4. **extrapolate_trend** is a parameter that we can use to extend the trend component to both sides to avoid the missing values that are generated when applying the moving average filter.
5. **two_sided** is a parameter that lets us define how the moving averages are calculated. If **True**, which it is by default, the moving average is calculated using the past as well as future values because the window for the moving average is centered. If **False**, it only uses past values to calculate the moving average.

Let's see how well we have been able to decompose one of the time series in our datasets. We used **period=7*48** to capture a weekday-hourly profile and **filt=np.repeat(1/(30*48), 30*48)** to make the moving average over 30 days with uniform weights:

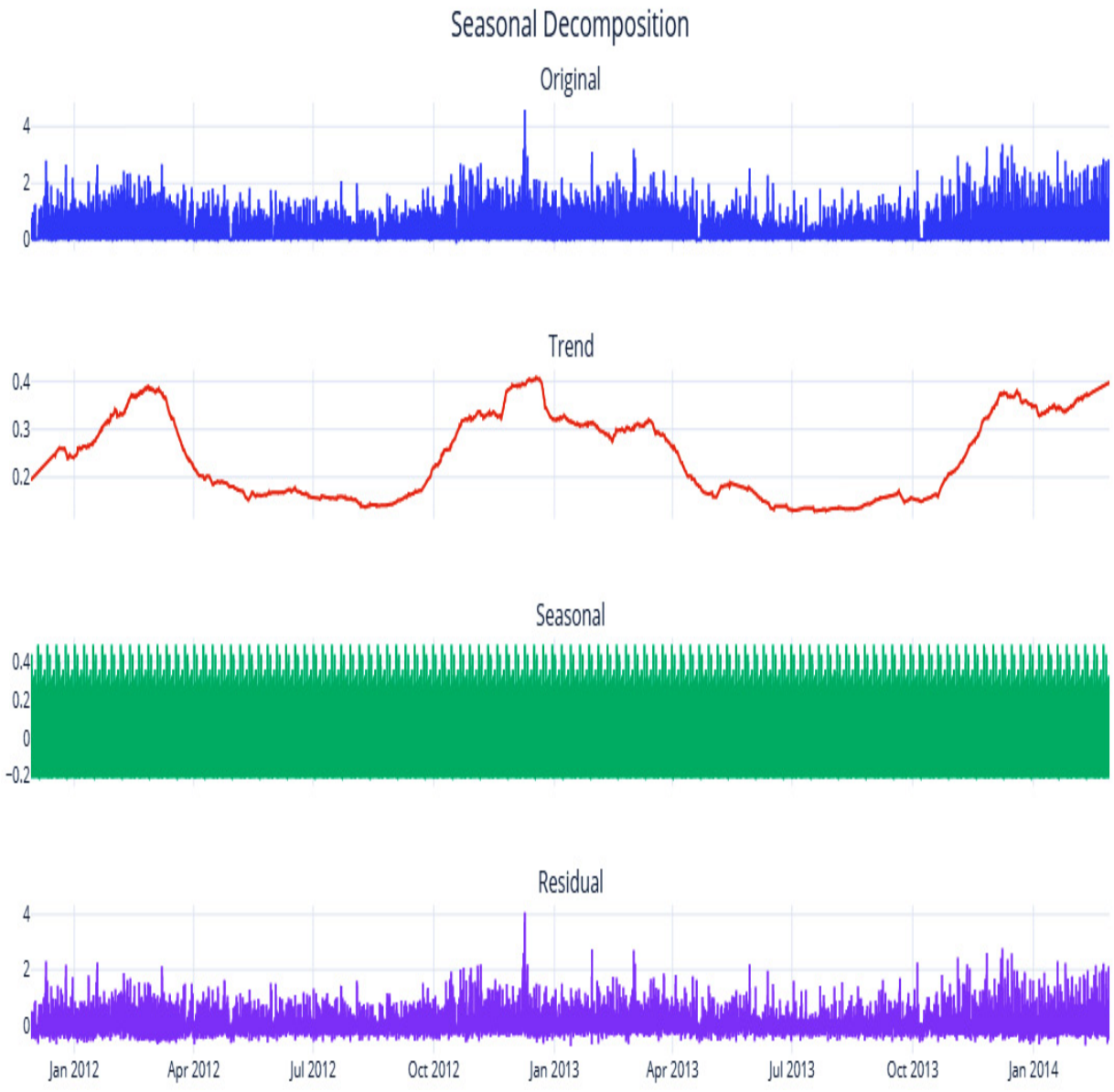


Figure 3.13 – Seasonal decomposition using statsmodels

We can't see the seasonal pattern because it's too small in the grand scale of the plot. The associated notebook has zoomed-in plots to help you understand the seasonal pattern. Even with a large window (for example, 20 days) of smoothing, the trend still has some noise in it. We may be able to

reduce this a bit more by increasing the window, but there is a better alternative, as we will see now.

Seasonality and trend decomposition using LOESS (STL)

As we saw earlier, LOESS is much more suited for trend estimation. **Seasonality and trend decomposition using LOESS** (**STL**) is an implementation that uses LOESS for trend estimation and period averages for seasonality. Although **statsmodels** has an implementation, we have reimplemented it for better performance and flexibility. This implementation can be found in this book's GitHub repository under **src.decomposition.seasonal.py**. It expects a pandas DataFrame or series with a datetime index as an input. Let's see how we can use this:

```
stl = STL(seasonality_period=7*48, model = "addit
res_new = stl.fit(ts_df.energy_consumption)
```

The key parameters here are as follows:

1. **seasonality_period** is the seasonal period you expect the pattern to repeat.
2. **model** takes **additive** or **multiplicative** as arguments to determine the type of decomposition.
3. **lo_frac** is the fraction of the data that will be used to fit the LOESS regression.

4. **lo_delta** is the fractional distance within which we use a linear interpolation instead of weighted regression. Using a non-zero **lo_delta** significantly decreases computation time.

Let's see what this decomposition looks like. Here, we used **seasonality_period=7*48** to capture a weekday-hourly profile:

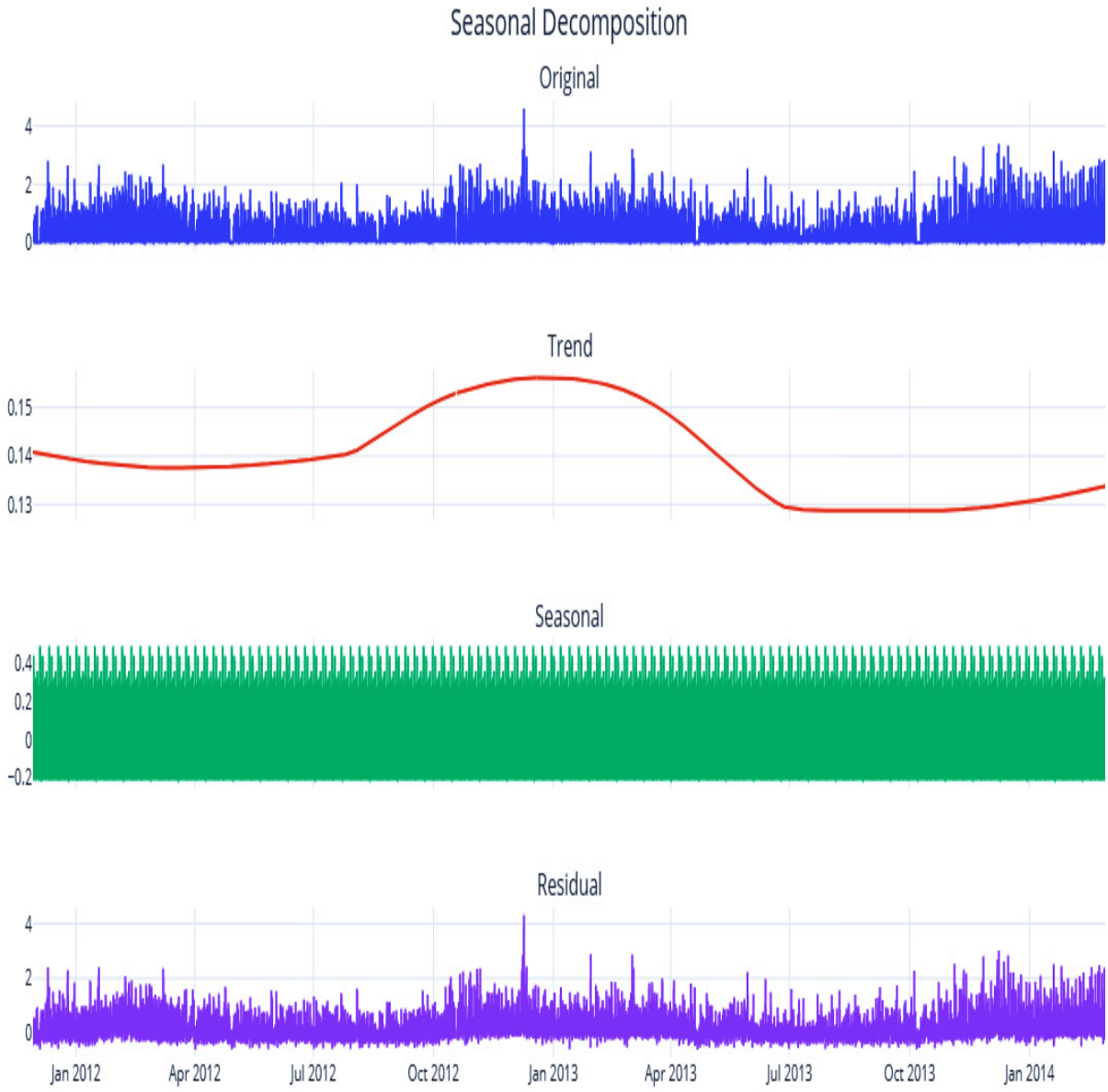


Figure 3.14 – STL decomposition

Let's also look at the decomposition for just 1 month to see the extracted seasonality patterns clearer:

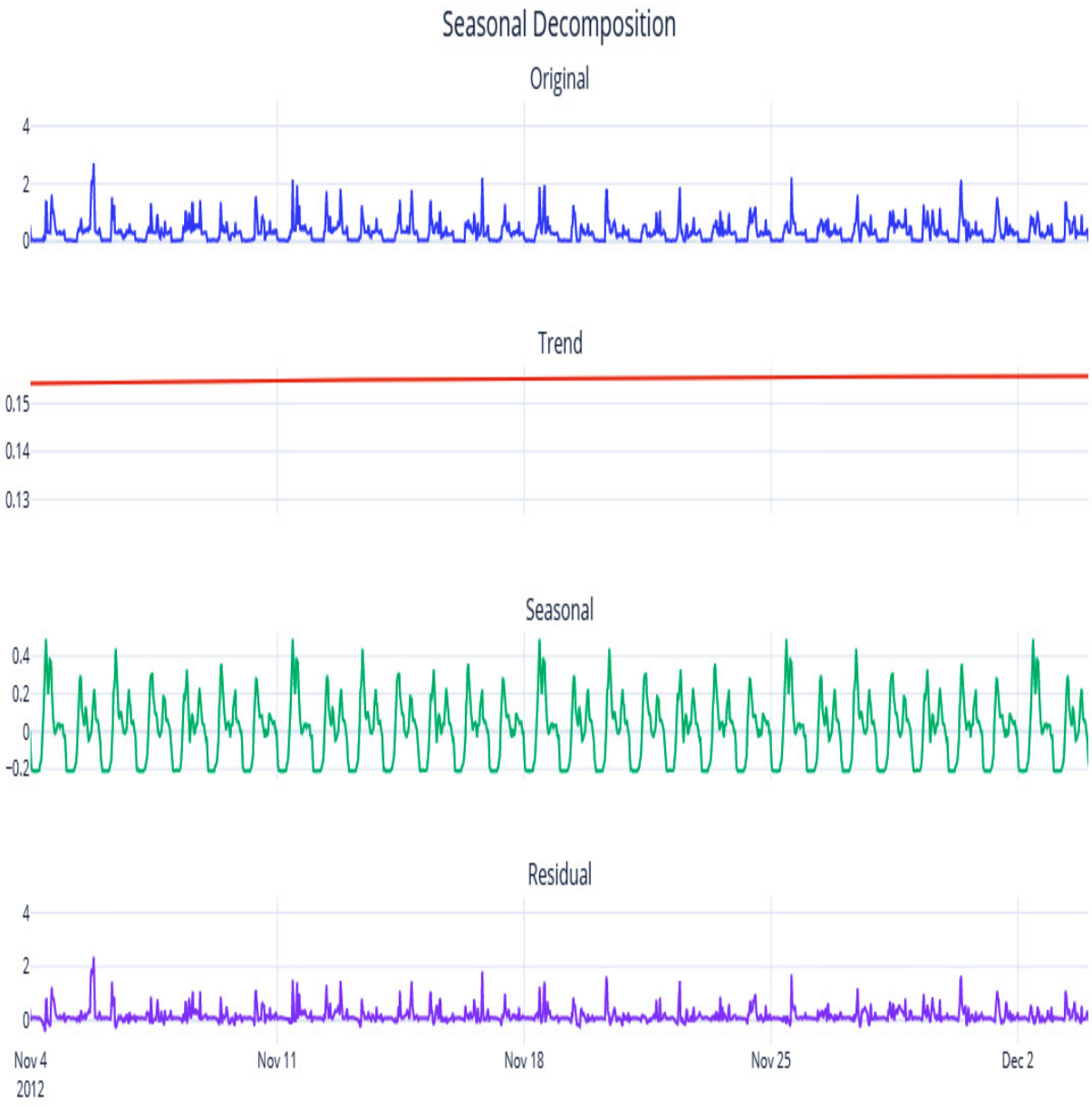


Figure 3.15 – STL decomposition (zoomed-in for a month)

The trend is smooth enough now and seasonality has also been captured. Here, we can clearly see the hourly peaks and valleys and the higher peaks on weekends. But since we are relying on averages to derive the seasonality, it is also highly influenced by outliers. A few very high or very low values

in the time series will skew your seasonality profile that's been derived from period averages. Another disadvantage of this technique is that the "goodness" of the seasonality that's been extracted suffers when the difference between the resolution of the data and the expected seasonality cycle is greater. For instance, when extracting a yearly seasonality on daily or sub-daily data, this would make the extracted seasonality very noisy. This technique will also not work if you have less than two cycles of the expected seasonality – for instance, if we want to extract a yearly seasonality, but we have less than 2 years of data.

Fourier decomposition

We can find the Python implementation for decomposing a time series using Fourier terms in **src.decomposition.seasonal.py**. It uses LOESS for trend detection and Fourier terms for seasonality extraction. There are two ways we can use it. First, we can specify **seasonality_period** as one of the **pandas** datetime properties (such as **hour**, **week_of_day**, and so on):

```
stl = FourierDecomposition(seasonality_period="ho
res_new = stl.fit(pd.Series(ts.squeeze(), index=t
```

Alternatively, we can create any custom seasonality array that's the same length as the time series that has an ordinal representation of the seasonality. If it is an annual seasonality of daily data, the array would have

a minimum value of **1** and a maximum value of **365** as it increases by one every day of the year:

#Making a custom seasonality term

```
ts_df["dayofweek"] = ts_df.index.dayofweek
ts_df["hour"] = ts_df.index.hour
```

#Creating a sorted unique combination df

```
map_df = ts_df[["dayofweek","hour"]].drop_duplica
```

 # Assigning an ordinal variable to capture the order

```
map_df["map"] = np.arange(1, len(map_df)+1)
```

 # mapping the ordinal mapping back to the original df and getting the seasonality array

```
seasonality = ts_df.merge(map_df, on=["dayofweek"
stl = FourierDecomposition(model = "additive", n_
res_new = stl.fit(pd.Series(ts, index=ts_df.index
```

The key parameters that are involved in this process are as follows:

1. **seasonality_period** is the seasonality to be extracted from the *datetime index*. **pandas** datetime properties such as **week_of_day**, **month**, and so on can be used to specify the most prominent seasonality. If left set to **None**, you need to provide the seasonality array while calling **fit**.
2. **model** takes **additive** or **multiplicative** as arguments to determine the type of decomposition.
3. **n_fourier_terms** determines the number of Fourier terms to be used to extract the seasonality. The more we increase this parameter, the more complex the seasonality that is extracted from the data.
4. **lo_frac** is the fraction of the data that will be used to fit the LOESS regression.
5. **lo_delta** is the fractional distance within which we use linear interpolation instead of weighted regression. Using a non-zero **lo_delta** significantly decreases computation time.

Let's see the zoomed-in plot for the decomposition using **FourierDecomposition**:

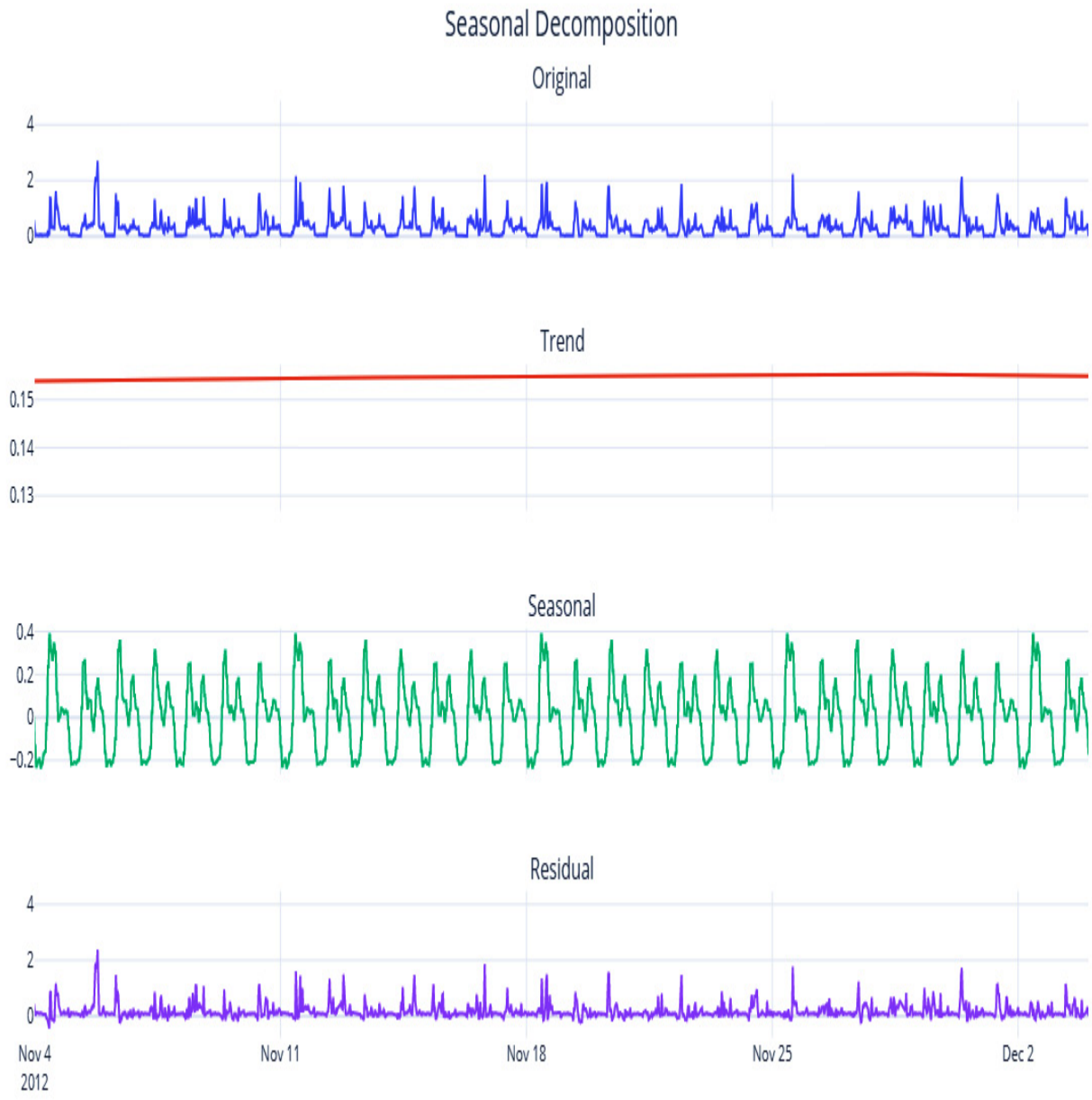


Figure 3.16 – Decomposition using Fourier terms (zoomed-in for a month)

The trend is going to be the same as the STL one because we are using LOESS here as well. The seasonality profile may be slightly different and robust to outliers because we are doing regularized regression using the Fourier terms on the signal. Another advantage is that we have decoupled

the resolution of the data and the expected seasonality. Now, extracting a yearly seasonality on sub-daily data is not as challenging as with period averages.

So far, we have only seen techniques that extract one seasonality per series; mostly, we extract the major seasonality. So, what do we do when we have multiple seasonal patterns?

Multiple seasonality decomposition using LOESS (MSTL)

Time series with high-frequency data (such as daily, hourly, or minutely data) are prone to exhibit multiple seasonal patterns. For instance, there may be an hourly seasonality pattern, a weekly seasonality pattern, and a yearly seasonality pattern. But if we extract only the dominant pattern and leave the rest to residuals, we are not doing justice to the decomposition. Kasun Bandara et al. proposed an extension of STL decomposition for multiple seasonality, known as **multiple seasonal-trend decomposition using LOESS** (**MSTL**), and a corresponding implementation is present in the R ecosystem. A very similar implementation in Python can be found in `src.decomposition.seasonal.py`. In addition to MSTL, the implementation extracts multiple seasonality using Fourier terms.

> *REFERENCE CHECK*
>
> *The research paper by Kasun Bandara et al. is cited in the References section as reference 1.*

Let's look at an example of how we can use this:

```
stl = MultiSeasonalDecomposition(seasonal_model='
res_new = stl.fit(pd.Series(ts, index=ts_df.index
```

The key parameters here are as follows:

1. **seasonality_periods** is the list of expected seasonalities. For **STL**, it is a list of seasonal periods, while for **FourierDecomposition**, it is a list of strings that denotes **pandas** datetime properties.
2. **seasonality_model** takes **fourier** or **averages** as arguments to determine the type of seasonality decomposition.
3. **model** takes **additive** or **multiplicative** as arguments to determine the type of decomposition.
4. **n_fourier_terms** determines the number of Fourier terms to be used to extract the seasonality. As we increase this parameter, the more complex the seasonality that is extracted from the data.
5. **lo_frac** is the fraction of the data that will be used to fit the LOESS regression.
6. **lo_delta** is the fractional distance within which we use linear interpolation instead of weighted regression. Using a non-zero **lo_delta** significantly decreases computation time.

Let's see what the decomposition looks like when using Fourier decomposition:

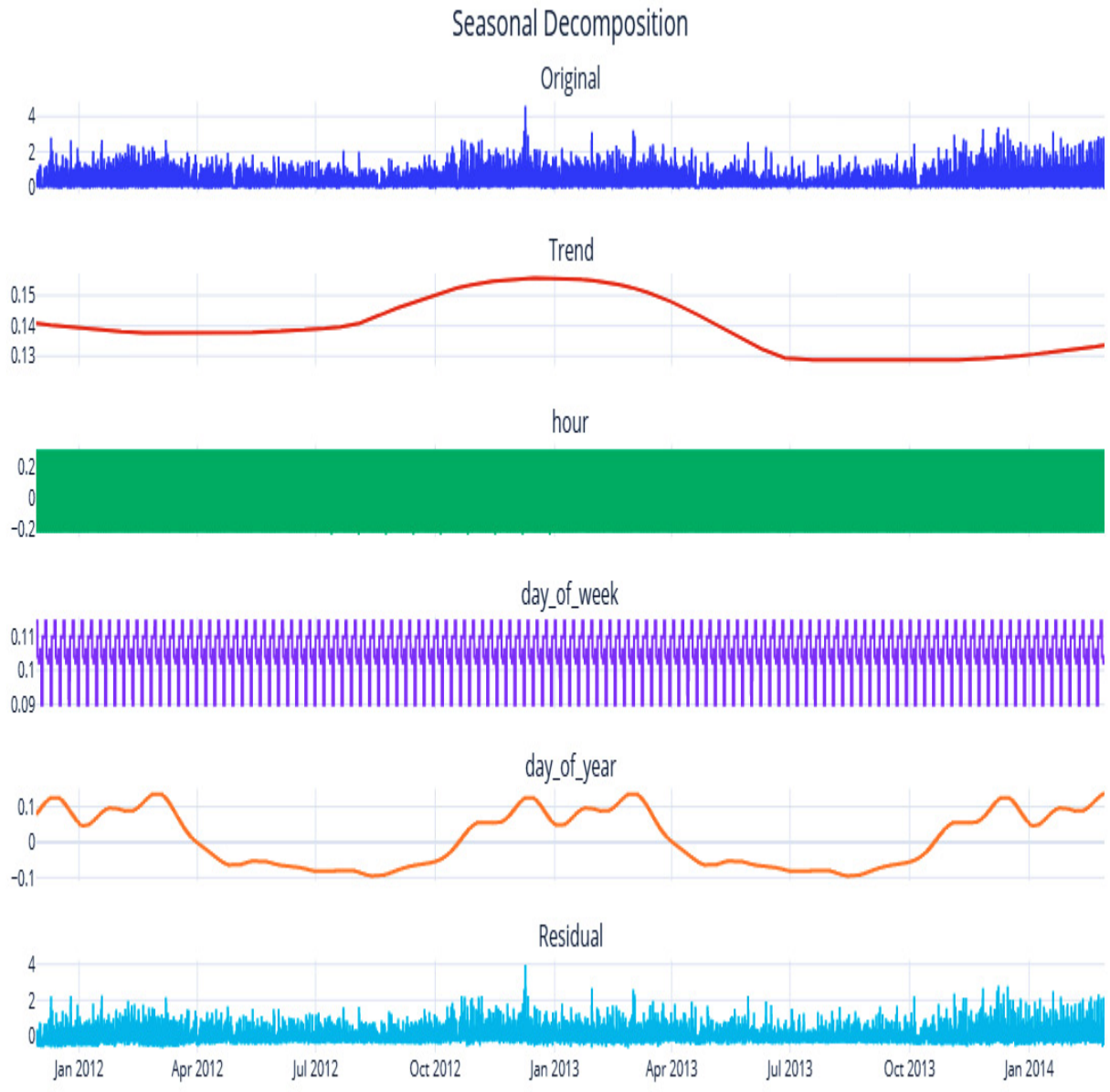


Figure 3.17 – Multiple seasonality decomposition using Fourier terms

Here, we can see that the **day_of_week** seasonality has been extracted. To see the **day_of_week** and **hour** seasonal components, we need to zoom in a bit:

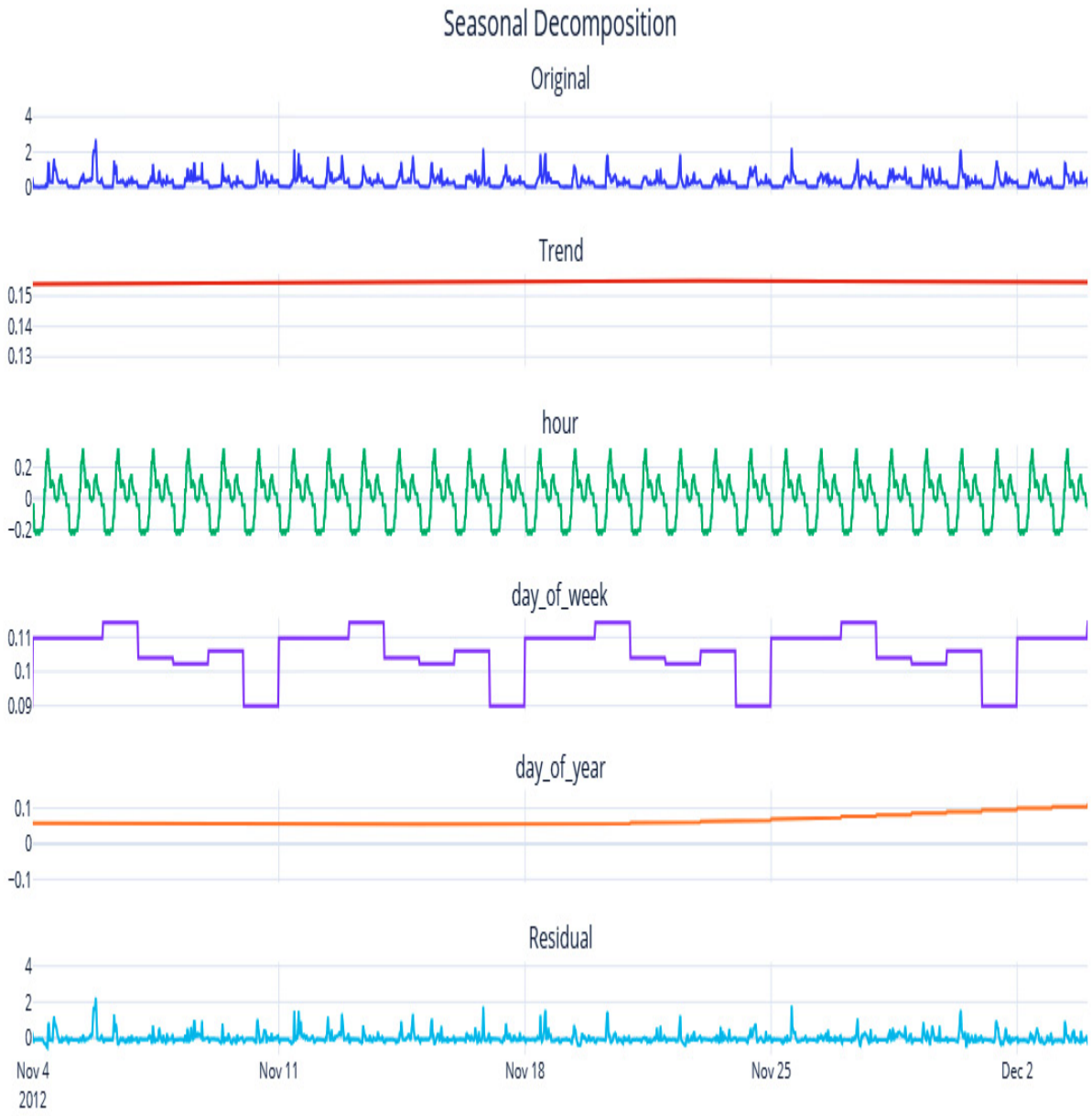


Figure 3.18 – Multiple seasonality decomposition using Fourier terms (zoomed-in for a month)

Here, we can observe that the **hour** seasonality has been extracted well and that it has also isolated the **day_of_week** seasonal component, which peaks on weekends. The **discrete step** nature of the **day_of_week** seasonal component is because the frequency of the data is half-hourly, and for 48 data points, **day_of_week** will be the same.

NOTE

MSTL has also been implemented in statsmodels and the accompanying notebook has the code to use that. The key difference between statsmodels and the implementation bundled within the code for the book is that the one in the book also has the option for using Fourier series based decomposition.

We have summarized the four techniques we've covered in the following table:

Implementation	Trend	Seasonal	Supports Multiple Seasonality?	Supports Missing?
seasonal_decompose (statsmodels)	Moving Averages	Period-Adjusted Averages	No	No
STL	LOESS	Period-Adjusted Averages	No	Yes
FourierDecomposition	LOESS	Fourier Terms	No	No
MultiSeasonalDecomposition	LOESS	Period-Adjusted Averages / Fourier Terms	Yes	No

Table 3.1 – Different seasonal decomposition techniques

Now, let's understand and analyze a time series dataset.

Detecting and treating outliers

An **outlier**, as its name suggests, is an observation that lies at an abnormal distance from the rest of the observations. If we are looking at a **data generating process** (**DGP**) as a stochastic process that generates the time

series, the outliers are the points that have the least probability of being generated from the DGP. This can be for many reasons, including faulty measurement equipment, incorrect data entry, and black-swan events, to name a few. Being able to detect such outliers and *treat* them may help your forecasting model understand the data better.

Outlier/anomaly detection is a specialized field itself in time series, but in this book, we are going to restrict ourselves to simpler techniques of identifying and treating outliers. This is because our main aim is not to detect outliers, but to clean the data for our forecasting models to perform better. If you want to learn more about anomaly detection, head over to the *Further reading* section for a few resources to get started.

Now, let's look at a few techniques for identifying outliers.

NOTEBOOK ALERT

> *To follow along with the complete code for detecting outliers, use the* **03-Outlier Detection.ipynb** *notebook in the* **chapter03** *folder.*

Standard deviation

This is a rule of thumb that almost everyone who has worked with data for some time would have heard of – if

$$\mu$$

is the mean of the time series and

$$\sigma$$

is the standard deviation, then anything that falls beyond

$$\mu \pm 3\,\sigma$$

is an **outlier**. The underlying theory is deeply rooted in statistics. If we assume that the values of the time series follow a normal distribution (which is a symmetrical distribution with very desirable properties), using probability theory, we can derive that 68% of the area under the normal distribution lies within one standard deviation on either side of the mean, about 95% of the area within two standard deviations, and about 99% of the area within three standard deviations. So, when we make the bounds as three standard deviations (by using the rule of thumb), what we are saying is that if any observation whose probability of belonging to the probability distribution is less than 1%, then they are an outlier. Moving slightly to more practical issues, this cutoff of three standard deviations is in no way sacrosanct. We need to try out different values of this multiple and determine the right multiple by subjectively evaluating the results we get. The higher the multiple is, the fewer outliers there will be.

For highly seasonal data, the naïve way of applying the rule to the raw time series will not work well. In such cases, we must deseasonalize the data using any of the techniques we discussed earlier and then apply the outliers

to the residuals. If we don't do that, we may flag a seasonal peak as an outlier, which is not what we want.

Another key assumption here is the normal distribution. However, in reality, a lot of the time series we come across may not be normal and hence the rule will lose its theoretical guarantees fast.

Interquartile range (IQR)

Another very similar technique is using the IQR instead of the standard deviation to define the bounds beyond which we mark the observations as outliers. IQR is the difference between the 3rd quartile (or the 75th percentile or 0.75 quantile) and the 1st quartile (or the 25th percentile or 0.25 quantile). The upper and lower bounds are defined as follows:

- Upper bound = Q3 + n x IQR
- Lower bound = Q1 - n x IQR

Here, *IQR = Q3-Q2*, and *n* is the multiple of IQRs that determines the width of the acceptable area.

For datasets where we observe high occurrences of outliers and wild variations, this is slightly more robust than the standard deviation. This is because the standard deviation and the mean are highly influenced by individual points in the dataset. If 2

was the rule of thumb in the earlier method, here, it is 1.5 times the IQR. This also ties back to the same normal distribution assumption, and 1.5 times the IQR is equivalent to ~3

(2.7

to be exact). The point about deseasonalizing before applying the rule applies here as well. It applies to all the techniques we will see here.

Isolation Forest

Isolation Forest is an unsupervised anomaly detection algorithm based on decision trees. A typical anomaly detection algorithm models the *normal* points and profiles outliers as any points that do not fit the *normal.* But Isolation Forest takes a different path and models the outliers directly. It does this by creating a forest of decision trees by randomly splitting the feature space. This technique works on the assumption that the outlier points fall in the outer periphery and are easier to fall into a leaf node of a tree. Therefore, you can find the outliers in short branches, whereas normal points, which are closer together, will require longer branches. The "anomaly score" of any point is determined by the depth of the tree to be

traversed before reaching that particular point. scikit-learn has an implementation of the algorithm under **sklearn.ensemble.IsolationForest**. Apart from the standard parameters for decision trees, the key parameter here is contamination. It is set to **auto** by default but can be set to any value between 0 and 0.5. This parameter specifies what percentage of the dataset you expect to be anomalous. But one thing we have to keep in mind is that **IsolationForest** does not consider time at all and just highlights values that fall *outside the norm*.

Extreme studentized deviate (ESD) and seasonal ESD (S-ESD)

This statistics-based technique is more sophisticated than the basic

$$\pm \sigma$$

technique, but still uses the same assumption of normality. It is based on another statistical test, called Grubbs's Test, which is used to find a *single outlier* in a normally distributed dataset. ESD iteratively uses Grubbs's test by identifying and removing an outlier at each step. It also adjusts the critical value based on the number of points left. For a more detailed understanding of the test, go to the *Further reading* section, where we have provided a couple of resources about ESD and S-ESD. In 2017, Hochenbaum et al. from Twitter Research proposed to use the generalized ESD with deseasonalization as a method of detecting outliers for time series.

We have adapted an existing implementation of the algorithm for our use case, and it is available in this book's GitHub repository. While all the other

methods leave it to the user to determine the right level of outliers by tweaking a few parameters, S-ESD only takes in an upper bound on the number of expected outliers and then identifies the outliers independently. For instance, we set the upper bound to 800 and the algorithm identified ~400 outliers in the data we are working with.

REFERENCE CHECK

> *The research paper by Hochenbaum et al. is cited in the References section as reference 2.*

Let's see how the outliers were detected using all the techniques we have reviewed:

	# of Outliers	% of Outliers
3SD	802	2.12%
2SD on Residuals	728	1.92%
4IQR	747	1.97%
4SD on Residuals	468	1.24%
Isolation Forest	364	0.96%
Isolation Forest on Residuals	359	0.95%
ESD	420	1.11%
S-ESD	424	1.12%

Figure 3.19 – Outliers detected using different techniques

Now that we've learned how to detect outliers, let's talk about how we can treat them and clean the dataset.

Treating outliers

The first question that we must answer is whether or not we should correct the outliers we have identified. The statistical tests that identify outliers automatically should go through another level of human verification. If we blindly "treat" outliers, we might be chopping off a valuable pattern that will help us forecast the time series. If you are only forecasting a handful of

time series, then it still makes sense to look at the outliers and anchor them to reality by looking at the causes for such outliers.

But when you have thousands of time series, a human can't inspect all the outliers, so we will have to resort to automated techniques. A common practice is to replace an outlier with a heuristic such as the maximum, minimum, 75th percentile, and so on. A better method is to consider the outliers as missing data and use any of the techniques we discussed earlier to impute the outliers.

One thing we must keep in mind is that outlier correction is not a necessary step in forecasting, especially when using modern methods such as machine learning or deep learning. Whether we do outlier correction or not is something we have to experiment with and figure out.

Well done! This was a pretty busy chapter, with a lot of concepts and code, so congratulations on finishing it. Feel free to head back and revise a few topics as needed.

Summary

In this chapter, we learned about the key components of a time series and familiarized ourselves with terms such as trend, seasonality, and so on. We also reviewed a few time series-specific visualization techniques that will come in handy during EDA. Then, we learned about techniques that let you

decompose a time series into its components and saw techniques for detecting outliers in the data. Finally, we learned how to treat the identified outliers. Now, you are all set to start forecasting the time series, which we will start in the next chapter.

References

The following are the references for this chapter:

1. Kasun Bandara and Rob J Hyndman and Christoph Bergmeir. (2021). *MSTL: A Seasonal-Trend Decomposition Algorithm for Time Series with Multiple Seasonal Patterns*. arXiv:2107.13462 [stat.AP]. https://arxiv.org/abs/2107.13462.
2. Hochenbaum, J., Vallis, O., & Kejariwal, A. (2017). *Automatic Anomaly Detection in the Cloud Via Statistical Learning*. ArXiv, abs/1704.07706. https://arxiv.org/abs/1704.07706.

Further reading

To learn more about the topics that were covered in this chapter, take a look at the following resources:

- Fourier Series: https://www.setzeus.com/public-blog-post/the-fourier-series.

- Fourier Series from Khan Academy: https://www.youtube.com/watch?v=UKHBWzoOKsY.
- Fourier Transform - https://betterexplained.com/articles/an-interactive-guide-to-the-fourier-transform/.
- Ane Blázquez-García, Angel Conde, Usue Mori, and Jose A. Lozano. (2021). *A Review on Outlier/Anomaly Detection in Time Series Data.* arXiv:2002.04236. https://arxiv.org/abs/2002.04236.
- Braei, M., & Wagner, S. (2020). *Anomaly Detection in Univariate Time-series: A Survey on the State-of-the-Art*. ArXiv, abs/2004.00433. https://arxiv.org/abs/2004.00433.
- Generalized ESD Test for Outliers: https://www.itl.nist.gov/div898/handbook/eda/section3/eda35h3.htm.

4 Setting a Strong Baseline Forecast

Join our book community on Discord

https://packt.link/EarlyAccess/

In the previous chapter, we saw some techniques we can use to understand **time series data**, do some **Exploratory Data Analysis** (**EDA**), and so on. But now, let's get to the crux of the matter – **time series forecasting**. The point of understanding the dataset and looking at patterns, seasonality, and so on was to make the job of forecasting that series easier. And with any machine learning exercise, one of the first things we need to establish before going further is a **baseline**.

A baseline is a simple model that provides reasonable results without requiring a lot of time to come up with them. Many people think of baselines as something that is derived from common sense, such as an average or some rule of thumb. But as a best practice, a baseline can be as sophisticated as we want it to be, so long as it is quickly and easily implemented. Any further progress we want to make will be in terms of the performance of this baseline.

In this chapter, we will look at a few classical techniques that can be used as baselines, and a strong baseline at that. Some may feel that the forecasting techniques we will be discussing in this chapter shouldn't be baselines, but we are keeping them

in here because these techniques have stood the test of time – and for good reason. They are also very mature and can be applied with very little effort, thanks to the awesome open source libraries that implement them. There can be many types of problems/datasets where it is difficult to beat the baseline techniques we will discuss in this chapter, and in those cases, there is no shame in just sticking to one of these baseline techniques.

In this chapter, we will cover the following topics:

1. Setting up a test harness
2. Generating strong baseline forecasts
3. Assessing the forecastability of a time series

Technical requirements

You will need to set up the Anaconda environment following the instructions in the Preface of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks

You will need to run the following notebooks before using the code in this chapter:

- `02-Preprocessing London Smart Meter Dataset.ipynb` preprocessing notebook from `Chapter02`.

The code for this chapter can be found at https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E/tree/main/notebooks/Chapter04.

Setting up a test harness

Before we start forecasting and setting up baselines, we need to set up a **test harness**. In software testing, a test harness is a collection of code and the inputs that have been configured to test a program under various situations. In terms of machine learning, a test harness is a set of code and data that can be used to evaluate algorithms. It is important to set up a test harness so that we can evaluate all future algorithms in a standard and quick way.

The first thing we need is **holdout** (**test**) and **validation** datasets.

Creating holdout (test) and validation datasets

As a standard practice, in machine learning, we set aside two parts of the dataset, name them *validation data* and *test data*, and don't use them at all to train the model. The validation data is used in the modelling process to assess the quality of the model. To select between different model classes, tune the hyperparameters, perform feature selection, and so on, we need a dataset. Test data is like the final test of your chosen model. It tells you how well your model is doing in unseen data. If validation data is like the mid-term exams, the test data is your final exam.

In regular regression or classification, we usually sample a few records at random and set them aside. But while dealing with time series, we need to respect the temporal aspect of the dataset. Therefore, a best practice is to set aside the latest part of the dataset as the test data. Another rule of thumb is to set equal-sized validation and test datasets so that the key modelling decisions we make based on the validation data are as close as possible to the test data. The dataset that we introduced in Chapter 2, *Acquiring and Processing Time Series Data* (the London Smart Energy dataset), contains the energy consumption readings of households in London from November

2011 to February 2014. So, we are going to put aside January 2014 as the validation data and February 2014 as the test data.

Let's open **01-Setting up Experiment Harness.ipynb** from the **chapter04** folder and run it. In the notebook, we must create the train-test split both before and after filling the missing values with **SeasonalInterpolation** and save them accordingly. Once the notebook finishes running, you will have created the following files in the pre-processed folder with the 2014 data saved separately:

- **selected_blocks_train.parquet**
- **selected_blocks_val.parquet**
- **selected_blocks_test.parquet**
- **selected_blocks_train_missing_imputed.parquet**
- **selected_blocks_val_missing_imputed.parquet**
- **selected_blocks_test_missing_imputed.parquet**

Now that we have a fixed dataset that can be used to fairly evaluate multiple algorithms, we need a way to evaluate the different forecasts.

Choosing an evaluation metric

In machine learning, we have a handful of metrics that can be used to measure continuous outputs, mainly **Mean Absolute Error** (**MAE**) and **Mean Squared Error** (**MSE**). But in the time series forecasting realm, there are scores of metrics with no real consensus on which ones to use. One of the reasons for this overwhelming number of metrics is that no one metric measures every characteristic of a forecast. Therefore, we have a whole chapter devoted to this topic (Chapter 18, *Evaluating Forecasts – Forecast Metrics*). For now, we will just review a few

metrics, all of which we are going to use to measure the forecasts. We are just going to consider them at face value:

- **Mean Absolute Error** (**MAE**): MAE is a very simple metric. It is the average of the unsigned error between the forecast at timestep

  $$t(f_t)$$

  and the observed value at time

  $$t(y_t).$$

  . The formula is as follows:

$$MAE = \frac{1}{N \times L} \times \sum_{i}^{N} \sum_{j}^{L} |f_{i,j} - y_{i,j}|$$

Here, *N* is the number of time series, *L* is the length of time series (in this case, the length of the test period), and *f* and *y* are the forecast and observed values, respectively.

- **Mean Squared Error** (**MSE**): MSE is the average of the squared error between the forecast

$$(f_t)$$

and observed

$$(y_t).$$

values:

$$MSE = \frac{1}{N \times L} \times \sum_{i}^{N} \sum_{\mathrm{j}}^{L} \left(f_{i,j} - y_{i,j}\right)^2$$

- **Mean Absolute Scaled Error** (**MASE**): MASE is slightly more complicated than MSE or MAE but gives us a slightly better measure to overcome the scale-dependent nature of the previous two measures. If we have multiple time series with different average values, MAE and MSE will show higher errors for the high-value time series as opposed to the low-valued time series. MASE overcomes this by scaling the errors based on the in-sample MAE from the **naïve forecasting method** (which is one of the most basic forecasts possible; we will review it later in this chapter). Intuitively, MASE gives us the measure of how much better our forecast is as compared to the naïve forecast:

$$MASE = \frac{\frac{1}{L} \times \sum_{i}^{L} |f_i - y_i|}{\frac{1}{L-1} \times \sum_{j=2}^{L} |y_j - y_{j-1}|}$$

- **Forecast Bias** (**FB**): This is a metric with slightly different aspects from the other metrics we've seen. While the other metrics help assess the *correctness* of the forecast, irrespective of the direction of the error, forecast bias lets us understand the overall *bias* in the model. Forecast bias is a metric that helps us understand whether the forecast is continuously over- or under-forecasting. We calculate forecast bias as the difference between the sum of the forecast and the sum of the observed values, expressed as a percentage over the sum of all actuals:

$$FB = \frac{\sum_{i}^{N} \sum_{j}^{L} f_{i,j} - \sum_{i}^{N} \sum_{j}^{L} y_{i,j}}{\sum_{i}^{N} \sum_{j}^{L} y_{i,j}}$$

Now, our test harness is ready. We also know how to evaluate and compare forecasts that have been generated from different models on a single, fixed holdout dataset with a set of predetermined metrics. Now, it's time to start forecasting.

Generating strong baseline forecasts

Time series forecasting has been around since the early 1920s, and through the years, many brilliant people have come up with different models, some statistical and some heuristic-based. I refer to them collectively as **classical statistical models** or **econometrics models**, although they are not strictly statistical/econometric.

In this section, we are going to review a few such models that can form really strong baselines when we want to try modern techniques in forecasting. As an exercise, we are going to use an excellent open source library for time series forecasting – **NIXTLA** (https://github.com/Nixtla). The **02-Baseline Forecasts using NIXTLA.ipynb** notebook contains the code for this section so that you can follow along.

Before we start looking at forecasting techniques, let's quickly understand how to use the **NIXTLA** library to generate the forecasts. We are going to pick one consumer from the dataset and try out all the baseline techniques on the validation dataset one by one.

The first thing we need to do is select the consumer we want using the unique ID for each customer, the **LCLid** column (from the expanded form of data), and set the timestamp as the index of the DataFrame:

```
ts_train = train_df.loc[train_df.LCLid=="MAC000193",['L
ts_val = val_df.loc[val_df.LCLid=="MAC000193", ['LCLid'
ts_test = test_df.loc[test_df.LCLid=="MAC000193", ['LCL
```

NIXTLA has the flexibility to work directly with either **pandas** or **Polars** dataframes. By default, NIXTLA looks for 3 columns:

`id_col` : By default, it expects a column 'unique_id'. This column uniquely identifies the time series. If you only have 1 time series, add a dummy column with the same unique identifier.

`time_col` : By default, it expects a column 'ds'. This is the column of your timestamp.

`target_col` : By default, it expects a column 'y'. This column is what you want NIXTLA to forecast.

This is very convenient as there is no need for further manipulation to go from data to modelling. NIXTLA follows the **sci-kit** learn style with **.fit()** and **.predict()**, and also adopts a **.forecast()** method. Forecast is a memory efficient method that avoids storing the partial model outputs, whereas the Scikit-learn interface stores the fitted models.

```
sf = StatsForecast(
    models=[model],
    freq=freq,
    n_jobs=-1,
    fallback_model=Naive()
)
sf.fit( df = _ts_train,
     id_col = 'LCLid',
     time_col = 'timestamp',
     target_col = 'energy_consumption',
)
baseline_test_pred_df = sf.predict(len(ts_test) )
Or
# Efficiently fit and predict without storing memory
```

```
y_pred = sf.forecast(
    h=len(ts_test),
    df=ts_train,
    id_col = 'LCLid',
    time_col = 'timestamp',
    target_col = 'energy_consumption',
)
```

When we call **.predict or .forecast**, we have to tell the model how long into the future we have to predict. This is called the horizon of the forecast. In our case, we need to predict our test period, which we can easily do by just taking the length of the **ts_test** array.

We can also calculate the metrics we discussed earlier in the test harness easily using NIXTLA's classes. For added flexibility, we can loop through a list of metrics to get multiple measurements for each forecast.

```
# Calculate metrics
metrics = [mase, mae, mse, rmse, smape, forecast_bias]
for metric in metrics:
    metric_name = metric.__name__
    if metric_name == 'mase':
        evaluation[metric_name] =
metric(results[target_col].values,            results[m
ts_train[target_col].values, seasonality=48)
    else:
        evaluation[metric_name] =
metric(results[target_col].values,
results[model_name].values)
Notice for MASE, the training set is also included.
```

For ease of experimentation, we have encapsulated all of this into a handy function, **evaluate_performance**, in the notebook. This returns the predictions and the calculated metrics in a dataframe.

Now, let's start looking at a few very simple methods of forecasting.

Naïve forecast

A naïve forecast is as simple as you can get. The forecast is just the last/most recent observation in a time series. If the latest observation in a time series is 10, then the forecast for all future timesteps is 10. This can be implemented as follows using the **Naive** class in **NIXTLA**:

```
from statsforecast.models import Naive
models = Naive()
```

Once we have initialized the model, we can call our helpful **evaluate_performance** function in the notebook to run and record the forecast and metrics.

Let's visualize the forecast we just generated:

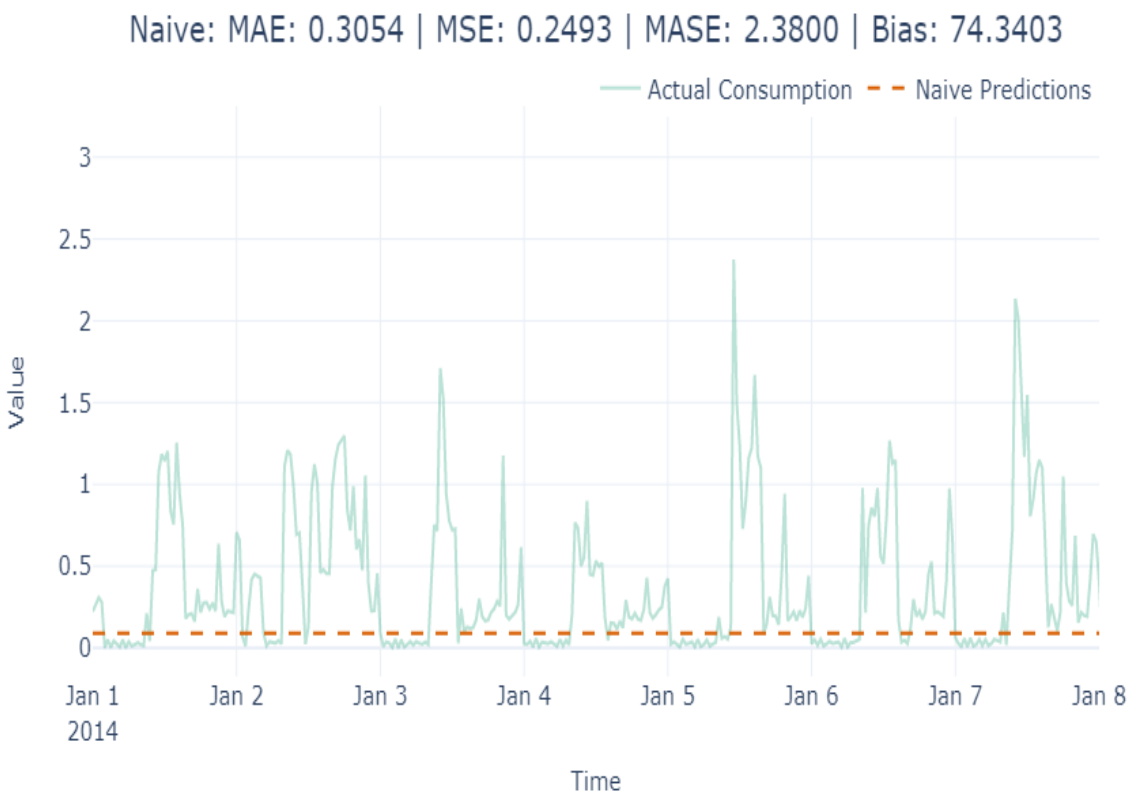


Figure 4.1 – Naïve forecast

Here, we can see that the forecast is a straight line and completely ignores any pattern in the series. This is by far the simplest way to forecast, hence why it is naïve. Now, let's look at another simple method.

Moving average forecast

While a naïve forecast memorizes the most recent past, it also memorizes the noise at any timestep. **A moving average forecast** is another simple method that tries to overcome the pure memorization of the naïve method. Instead of taking the latest observation, it takes the mean of the latest *n* steps as the forecast. Moving average is

not one of the models present in **NIXTLA**, but we have implemented a NIXTLA-compatible model in this book's GitHub repository in the **chapter04** folder:

```
from src.forecasting.baselines import NaiveMovingAverag
#Taking a moving average over 48 timesteps, i.e, one da
naive_model = NaiveMovingAverage(window=48)
```

Let's look at the forecast we generated:

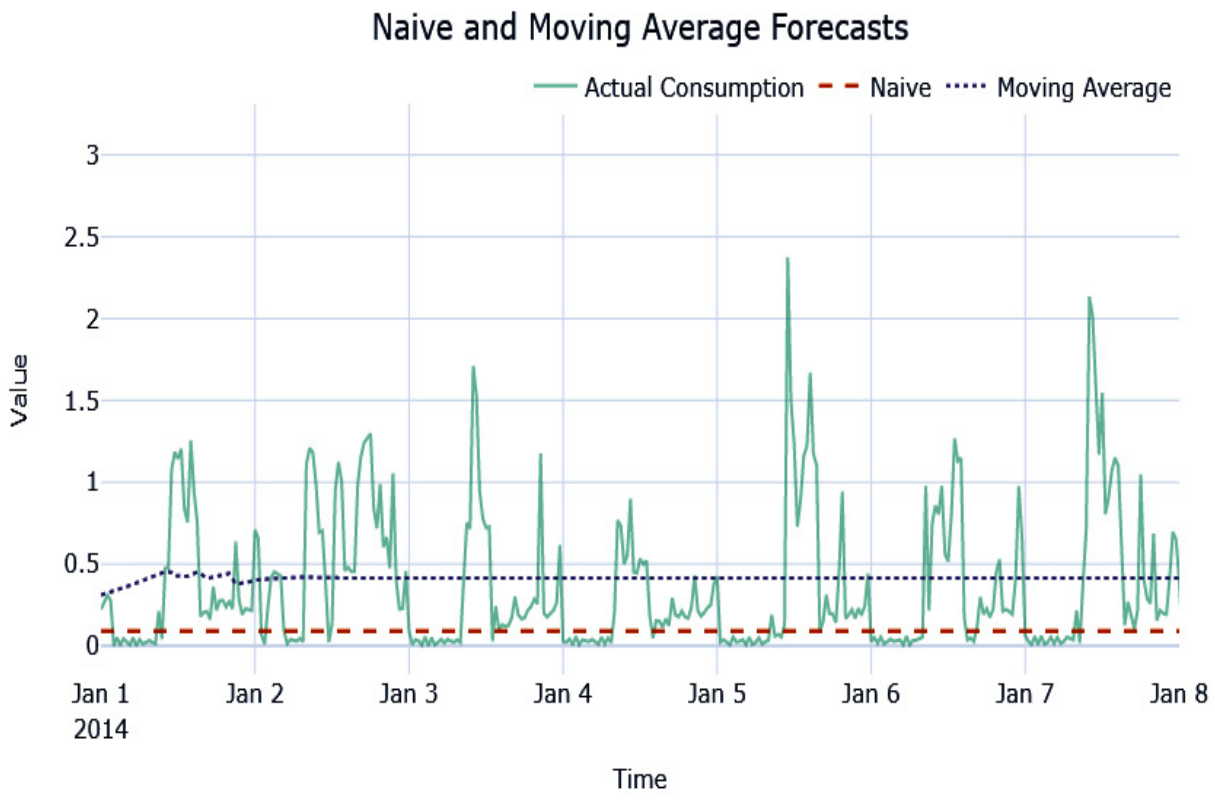


Figure 4.2 – Moving average forecast

This forecast is also almost a straight line. Now, let's look at another simple method, but one that considers seasonality as well.

Seasonal naive forecast

A seasonal naive forecast is a twist on the simple naive method. Whereas in the naive method, we took the last observation (

$$Y_{t-1}$$

), in seasonal naïve, we take the

$$Y_{t-k}$$

observation. So, we look back *k* steps for each forecast. This enables the algorithm to mimic the last seasonality cycle. For instance, if we set **k=48*7**, we will be able to mimic the latest seasonal weekly cycle. This method is implemented in NIXTLA and we can use it like so:

```
from statsforecast.models import SeasonalNaive
seasonal_naive = SeasonalNaive(season_length=48*7)
```

Let's see what this forecast looks like:

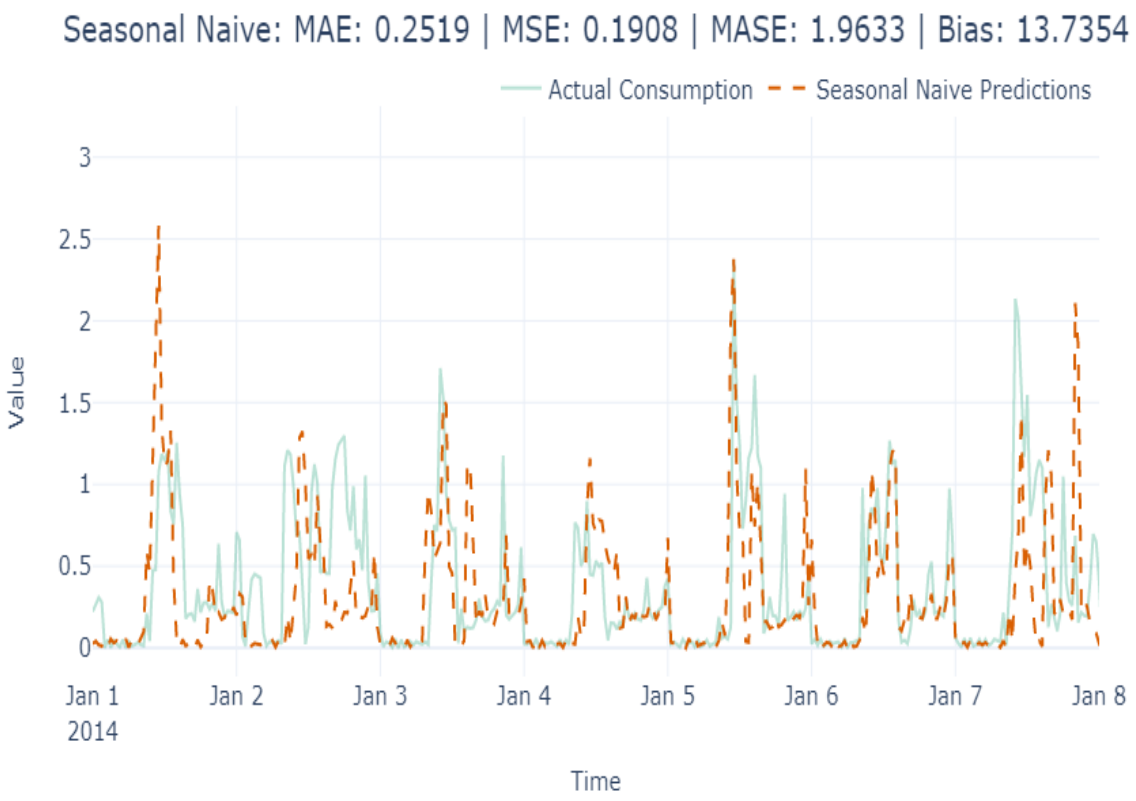


Figure 4.3 – Seasonal naïve forecast

Here, we can see that the forecast is trying to mimic the seasonality pattern. However, it's not very accurate because it is blindly following the last seasonal cycle.

Now that we've looked at a few simple methods, let's look at a few statistical models.

Exponential smoothing (ETS)

Exponential smoothing (**ETS**) is one of the most popular methods for generating forecasts. It has been around since the late 1950s and has proved its mettle and stood the test of time. There are a few different variants of ETS – **single exponential smoothing**, **double exponential smoothing**, **Holt-Winters' seasonal smoothing**,

and so on. But all of them have one key idea that has been used in different ways. In the naïve method, we were just using the latest observation, which is like saying only the most recent data point in history matters and no data point before that matters. On the other hand, the moving average method considers the last *n* observations to be equally important and takes the mean of them.

ETS combines both these intuitions and says that all the history is important, but the recent history is more important. Therefore, the forecast is generated using a weighted average where the weights decrease exponentially as we move farther into the history:

$$f_t = \alpha \cdot y_{t-1} + \alpha \cdot (1-\alpha) \cdot y_{t-2} + \alpha \cdot (1-\alpha)^2 \cdot y_{t-3} + \cdots$$

Here,

$$0 \leq \alpha \leq 1$$

is the smoothing parameter that lets us decide how fast or slow the weights should decay,

$$y_t$$

is the actuals at timestep *t*, and

$$f_t$$

is the forecast at timestep *t*.

Simple exponential smoothing (**SES**) is when you simply apply this smoothing procedure to the history. This is more suited for time series that have no trends or seasonality, and the forecast is going to be a flat line. The forecast is generated using the following formula:

Forecast Equation	$f_t = \alpha y_{t-1} + (1-\alpha) f_{t-1}$

Double exponential smoothing (**DES**) extends the smoothing idea to model trends as well. It has two smoothing equations – one for the level and the other for the trend. Once you have the estimate of the level and trend, you can combine them. This forecast is not necessarily flat because the estimated trend is used to extrapolate it into the future. The forecast is generated according to the following formula:

Forecast Equation	$f_{t+h} = l_t + h \cdot b_t$
Level Equation	$l_t = \alpha \cdot y_t + (1 - \alpha) \cdot (l_{t-1} + b_{t-1})$
Trend Equation	$b_t = \beta \cdot (l_t - l_{t-1}) + (1 - \beta) \cdot b_{t-1}$

First, we estimate the level (

$$l_t$$

) using the *Level Equation* with the available observations. Then, we estimate the trend using the *Trend Equation*. Finally, to get the forecast, we combine

$$l_t$$

and

$$b_t$$

using the *Forecast Equation*. Researchers have found empirical evidence that this kind of constant extrapolation can result in over-forecasts over the long-term forecast. This is because, in the real world, time series data doesn't increase at a constant rate forever. Motivated by this, an addition to this has also been introduced that dampens the trend by a factor of

$$0 < \phi < 1$$

, such that when

$$\phi = 1$$

, there is no damping, and it is identical to double exponential smoothing. **Triple exponential smoothing** or **Holt -Winters** (**HW**) takes this one step forward by including another smoothing term to model the seasonality. This has three parameters (

$$\alpha, \beta, \gamma$$

) for the smoothing and uses a seasonality period (*m*) as input parameters. You can also choose between additive or multiplicative seasonality. The forecast equations for the additive model are as follows:

Forecast Equation	$f_{t+h} = l_t + h \cdot b_t + s_{t+h-m(k+1)}$
Level Equation	$l_t = \alpha \cdot y_t - s\_\{t-m\} + (1-\alpha) \cdot (l_{t-1} + b_{t-1})$
Trend Equation	$b_t = \beta \cdot (l_t - l_{t-1}) + (1-\beta) \cdot b_{t-1}$
Seasonality Equation	$s_t = \gamma(y_t - l_{t-1} - b_{t-1}) + (1-\gamma)s_{t-m}$

These formulae are also used like in the double exponential case. Instead of estimating level and trend, we estimate level, trend, and seasonality separately.

The family of exponential smoothing methods is not limited to the three that we just discussed. A way to think about the different models is in terms of the trend and seasonal components of these models. The trend can either be no trend, additive, or additive damped. The seasonality can be no seasonality, additive, or multiplicative. Every combination of these parameters is a different technique in the family, as shown in the following table:

Trend component	Seasonal component		
	N (None)	A (Additive)	M (Multiplicative)
N (None)	*Simple Exponential Smoothing*	-	-
A (Additive)	*Double Exponential Smoothing*	*Additive Holt-Winters*	*Multiplicative Holt-Winters*
Ad (Additive damped)	*Damped Double Exponential Smoothing*	-	*Damped Holt-Winters*

Table 4.1 – Exponential smoothing family

NIXTLA has an entire family of exponential smoothing methods.

Let's see how we can initialize the ETS model in **NIXTLA**:

```
from statsforecast.models import (SimpleExponentialSmoo
exp_smooth = HoltWinters(error_type = 'A', season_lengt
```

Here the `error_type = 'A'` refers to additive error. The user has the option for either additive error or multiplicative error, which could be called using `error_type = 'M'.` NIXTLA models has an option to use `AutoETS().` This model will automatically choose which exponential smoothing model is the best option. Simple exponential smoothing, double exponential smoothing (Holt's

Method) or triple exponential smoothing (Holt-Winters Method). It will also choose which parameters and error types are best for each individual time series. Refer to the GitHub notebooks for examples on how to use `AutoETS()`.

Let's see what the forecast using ETS looks like in Figure 4.4:

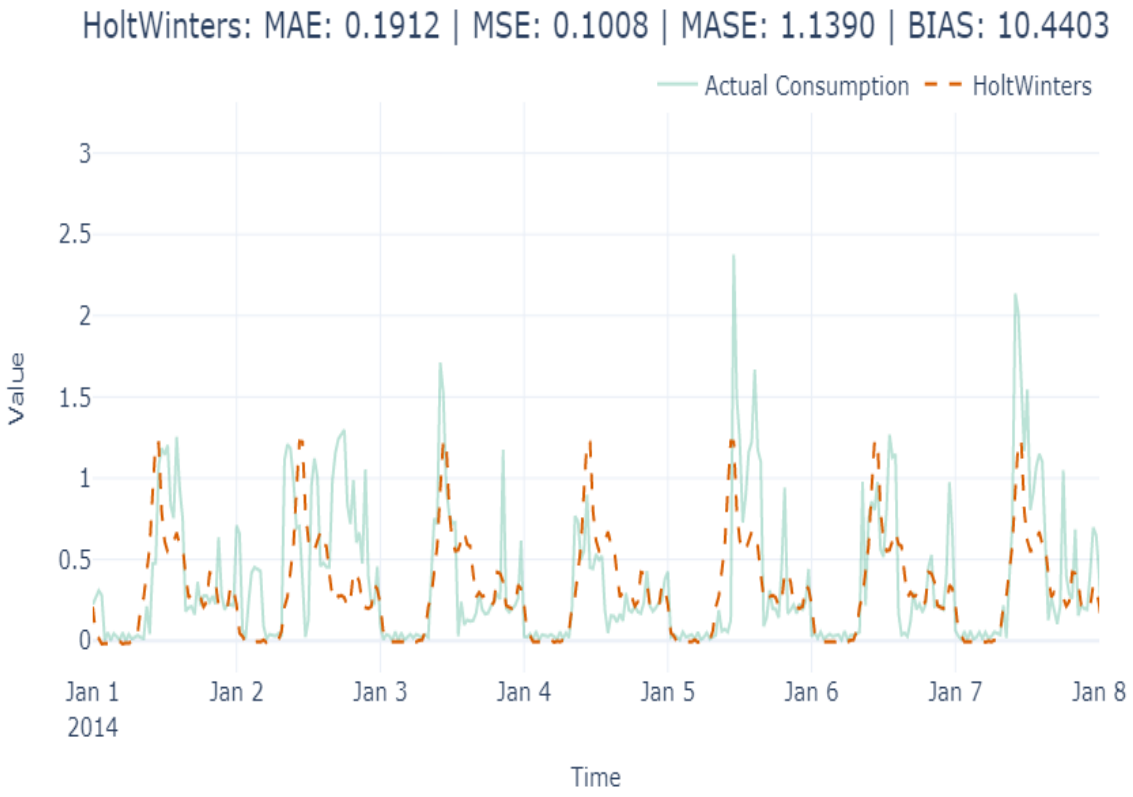


Figure 4.4 – Exponential smoothing forecast

The forecast has captured the seasonality but has failed to capture the peaks. But we can see the improvement in MAE already.

Now, let's look at one of the most popular forecasting methods out there.

ARIMA

Autoregressive Integrated Moving Average (**ARIMA**) models are the other class of methods that, like ETS, have stood the test of time and are one of the most popular classical methods of forecasting. The ETS family of methods is modelled around trend and seasonality, while ARIMA relies on **autocorrelation** (the correlation of

$$y_t$$

with

$$y_{t-1}$$

,

$$y_{t-2}$$

, and so on).
The simplest in the family are the *AR(p)* models, which use **linear regression** with *p* previous timesteps or, in other words, *p* lags. Mathematically, it can be written as follows:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots \phi_p y_{t-p} + \epsilon_t$$

Here, *c* is the intercept and

$$\epsilon_t$$

is the noise or error at timestep *t*.

The next in the family are *MA(q)* models, in which instead of past observed values, we use the past *q* errors in the forecast (which is assumed to be pure white noise) to come up with a forecast:

$$y_t = c + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots \theta_q \epsilon_{t-q}$$

Here,

$$\epsilon$$

is white noise and *c* is the intercept. This is not typically used on its own but in conjunction with AR(p) models, which makes the next one on our list *ARMA(p,q)* models. ARMA models are defined as

$$y_t = AR(p) + MA(q)$$

.

In all the ARIMA models, there is one underlying assumption – the *time series is stationary* (we talked about stationarity in Chapter 1, *Introducing Time Series*, and will elaborate on this in Chapter 6, *Feature Engineering for Time Series Forecasting*). There are many ways to make the series stationary but taking the difference of

successive values is one such technique. This is known as **differencing**. Sometimes, we need to do differencing once, while other times, we have to perform successive differencing before the time series becomes stationary. The number of times we do the differencing operation is called the *order of differencing*. The I in ARIMA, and the final piece of the puzzle, stands for *Integrated*. It defines the order of differencing we need to do before the series becomes stationary and is denoted by *d*.

So, the complete *ARIMA(p,d,q)* model says that we do *d*th order of differencing and then consider the last *p* terms in an autoregressive manner, and then include the last *q* moving average terms to come up with the forecast.

The ARIMA models we have discussed so far only handle non-seasonal time series. But using the same concepts we discussed, but on a seasonal cycle, we get **Seasonal ARIMA**. *p*, *d*, and *q* are slightly tweaked so that they work on the seasonal period, *m*. To differentiate them from the normal *p*, *d*, and*q*, we call the seasonal values *P*, *D*, and *Q*. For instance, if *p* meant taking the last *p* lags, *P* means taking the last *P* seasonal lags. If

$$p_1$$

is

$$y_{t-1}$$

,

$$P_1$$

would be

$$y_{t-m}$$

. Similarly, *D* means the order of seasonal differencing.
Picking the right *p*, *d*, and *q* and *P*, *D*, and *Q* values is not very intuitive, and we will have to resort to statistical tests to find them. However, this becomes a bit impractical when you are forecasting many time series. An automatic way of iterating through the different parameters and finding the best *p*, *d*, and *q*, and *P*, *D*, and *Q* values for the data is called **Auto ARIMA**. In Python, NIXTLA has implemented this method **AutoARIMA()**. **NIXTLA** also has a normal ARIMA implementation as well, which is much faster but requires p,d,q to be entered manually.

PRACTICAL CONSIDERATIONS

> *Although ARIMA and Auto ARIMA can give you good-performing models in many cases, they can be quite slow when you have long seasonal periods and a long time series. In our case, where we have almost 27k observations in the history, ARIMA becomes very slow and a memory hog. Even when subsetting the data, a single AutoARIMA fit takes around 60 minutes. Letting go of the seasonal parameters brings down the runtime drastically, but for a seasonal time series such as energy consumption, it doesn't make sense. Auto ARIMA includes many such fits to identify the best parameters and therefore becomes impractical for long time series datasets. Almost all the implementations in the Python ecosystem suffer from this drawback. NIXLTA claims to have the fastest and most accurate version of AutoARIMA, faster than the original R method as well.*

Let's see how we can apply **ARIMA** and **AutoARIMA** using **NIXTLA**:

```
from statsforecast.models import (ARIMA, AutoARIMA)
#ARIMA model by specifying parameters
arima_model = ARIMA(order = (2,1,1), seasonal_order = (
#AutoARIMA model by specifying max limits for parameter
auto_arima_model = AutoARIMA( max_p = 2, max_d=1, max_q
```

For the entire list of parameters for **AutoARIMA**, head over to the **NIXTLA** documentation at https://nixtlaverse.nixtla.io/statsforecast/docs/models/autoarima.html.

Let's see what the ETS and ARIMA forecasts look like for the households we were experimenting with:

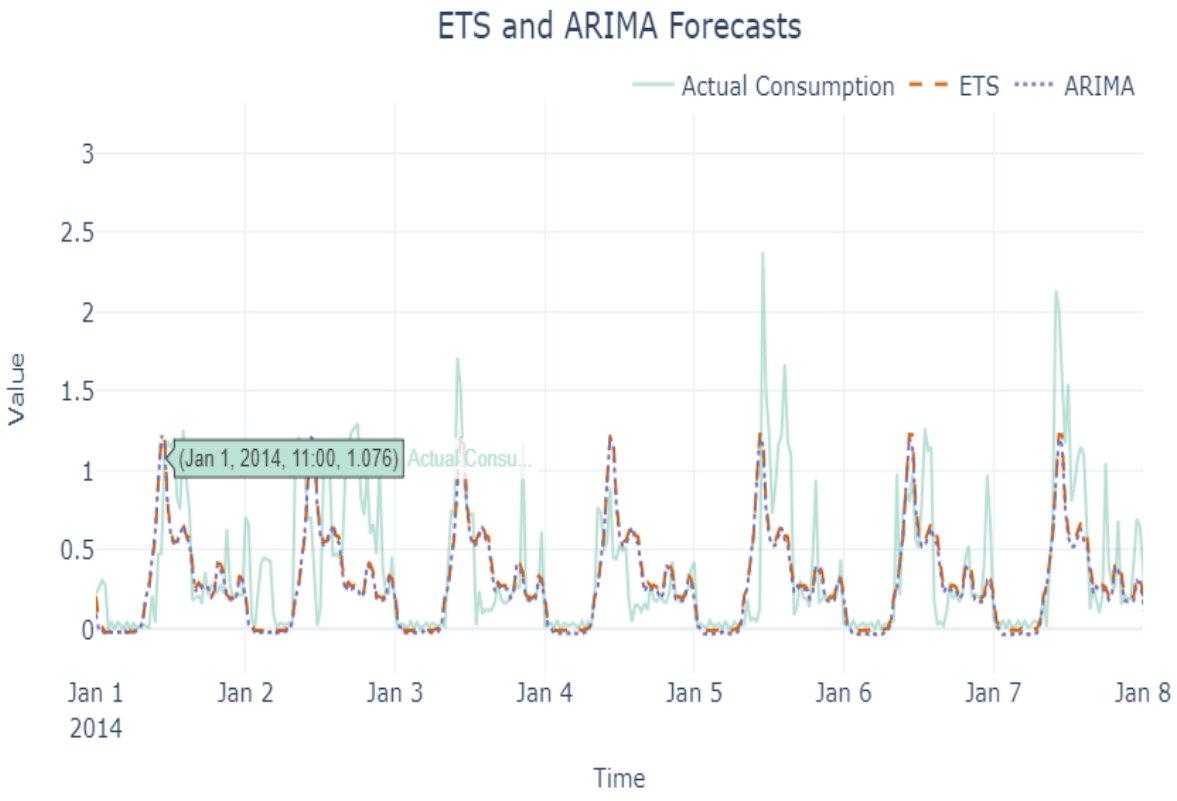


Figure 4.5 – ETS and ARIMA forecasts (Color Figure)

With NIXTLA, both ETS and ARIMA have done a good job at capturing both the seasonality and the peaks. The resulting MAE scores are also very similar with 0.191 vs. 0.203 respectively. Now, let's look at another method – the Theta Forecast.

Theta Forecast

The **Theta Forecast** was the top-performing submission in the M3 forecasting competition that was held in 2002. The method relies on a parameter,

$$\theta$$

, that amplifies or smooths the local curvature of a time series, depending on the value chosen. Using

, we smooth or amplify the original time series. These smoothed lines are called **theta lines**. V. Assimakopoulos and K. Nikolopoulos proposed this method as a decomposition approach to forecasting. Although in theory any number of theta lines can be used, the originally proposed method used two theta lines,

and

, and took an average of the forecast of the two theta lines as the final forecast.

SIDE NOTE

The M-competitions are forecasting competitions organized by Spyros Makridakis, a leading forecasting researcher. They typically curate a dataset of time series, lay down the metrics with which the forecasts will be evaluated, and open these competitions to researchers all around the world to get the best forecast possible. These competitions are considered to be some of the biggest and most popular time series forecasting competitions in the world. At the time of writing, six such competitions have already been completed. To learn more about the latest competition, visit this website: https://mofc.unic.ac.cy/the-m6-competition/.

In 2002, Rob Hyndman et al. simplified the Theta method and showed that we can use ETS with a drift term to get equivalent results to the original Theta method, which is what is adapted into most of the implementations of the method that exist today. The major steps that are involved in the Theta Forecast (which is implemented in **NIXTLA**) are as follows:

1. **Deseasonalization**: Apply a classical multiplicative decomposition to remove the seasonal component from the time series (if exists). This focuses the analysis on the underlying trend and cyclical components. Deseasonalization is done using s**tatsmodels.tsa.seasonal.seasonal_decompose**. This step creates a new deseasonalized time series, .

Theta Coefficients Application: Decompose the deseasonalized series into 2 "Theta" lines using coefficients and . These coefficients modify the second difference of the time series to either dampen (local fluctuations.

1. **Extrapolation of Theta Lines**: Treat each Theta line as a separate series and forecast them into the future. This is done using linear regression for the Theta line

where producing a straight line, and simple exponential smoothing for the Theta line where

2. **Recomposition**: Combine the forecasts from the two theta lines. The original method uses equal weighting for both lines, which integrates the long-term trend and short-term movements effectively.
3. **Reseasonalize:** If the data was deseasonalized in the beginning.

NIXTLA has a very different variations of the THETA method. More information on the specifics of the NIXTLA implementation can be found here:

https://nixtlaverse.nixtla.io/statsforecast/docs/models/autotheta.html

Let's see how we can use it practically:

```
theta_model = Theta(season_length =48, decomposition_ty
```

The key parameters here are as follows:

- **season_length & decomposition_type:** These parameters are used for the initial seasonal decomposition. If left empty, the implementation automatically tests for seasonality and deseasonalizes the time series automatically using multiplicative decomposition. It is recommended to set these parameters with our domain knowledge if we know them. Decompositive type can be 'multiplicative' (default) or 'additive'.

1. :

Let's visualize the forecast we just generated using the Theta Forecast:

Figure 4.6 – The Theta Forecast

REFERENCE CHECK

> *The research paper in which V. Assimakopoulos and K. Nikolopoulos proposed the Theta method is cited as reference 1 in the References section, while subsequent simplification by Rob Hyndman is cited as reference 2.*

TBATS

Sometimes a time series has more than 1 seasonality pattern or a non-integer seasonal period, commonly referred to as complex seasonality. An example would be an hourly forecast could have a daily seasonality for time of day, a weekly seasonality for day of week, and a yearly seasonality for day in year. Additionally, most time series models are designed for smaller integer seasonal periods, such as monthly (12) or quarterly (4) data, but yearly seasonality can pose a problem since a year is 364.25 days. TBATS was meant to combat these many challenges which pose problems for many forecasting models. However, with any automated approach, at times it is susceptible to poor forecasts.

- **TBATS** stands for
- **T**rigonometric seasonality
- **B**ox-Cox transformation
- **A**RMA errors
- **T**rend
- **S**easonal components

This model was first introduced by Rob J Hyndman, Alysha M De Livera, and Ralph D Snyder in 2011. There is also another variant of TBATS, referred to as BATS, which is without the trigonometric seasonality component. TBATS is from the state space model family. In state space forecasting models, the observed time series is assumed to be a combination of the underlying state variables and a measurement equation that relates the state variables to the observed data. The state variables capture the underlying patterns, trends, and relationships in the data.

BATS has parameters indicating the Box-Cox parameter, damping parameter, ARMA parameters and the seasonal periods Due to its flexibility, BATS model can be considered a family of models encompassing many other models we have seen earlier. For instance:

= Holt Winters Additive Seasonality

= Holt Winters Additive Double Seasonality

BATS has the flexibility for multiple seasonality; however, it has a limitation to only integer based seasonal periods, and with multiple seasonalities it can have a large number of states resulting in increasing model complexity. This is what TBATS was meant to address.

For reference, the TBATS parameter space is

The main advantages of TBATS are as follows:

1. Works with single, complex, and non-integer seasonality (Trigonometric Seasonality)

2. Handles nonlinear patterns common in real world time series (Box-Cox Transformation)
3. Handles autocorrelation in the residuals (ARMA errors)

To better understand the inner workings of TBATS, lets break down each step.

The order in which operations are done using TBATS is:

1. Box-Cox Transformation
2. Exponentially smoothed Trend
3. Seasonal Decomposition using Fourier series.
4. Autoregressive moving average (ARMA)
5. Parameter estimation through a likelihood based approach.

Box Cox Transformation

Box-Cox is a transformation in the family of power transformations.

In time series, making data stationary is an important step before forecasting (as discussed in chapter 1). Stationarity ensure that our data does not statistically change over time, and thus more accurately resemble a probability distribution. There are several possible transformations that could be applied. More details on various target transformations, including Box-Cox can be found in chapter 7.

As a preview, here is a sample output from a Box-Cox transformation. After the transformation, our data more closely resembles that of a normal distribution. Box-Cox transformations can only be used with positive data, but in practice this is often the case. Figure 4.7 shows an example of how a time series might look before and after a Box-Cox transformation.

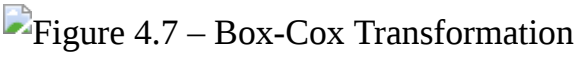


Figure 4.7 – Box-Cox Transformation

Exponentially Smoothed Trend

Using Locally Estimated Scatterplot Smoothing (LOESS), a smoothed trend is extracted from the time series.

LOESS works by applying a locally weighted, low-degree polynomial regression over the data points to create a smooth, flowing line through them. This technique is highly effective in capturing local trend variations without assuming a global form for the data, which makes it particularly useful for data with varying trends or seasonal variations.

Trigonometric Seasonality

The remaining residuals are then modelled using Fourier terms (discussed in chapter 3) to decompose the seasonality component.

The main advantage of using Fourier to model seasonality is its ability to model multiple seasonalities, as well as non-integer seasonality, such as yearly seasonality with daily data, since there is 364.25 days in a year. Most other decomposition methods cannot handle the non-integer period, and has to resort to rounding to 365 which can fail to identify the true seasonality. An example of what a decomposed time series would like using Fourier is below. The observed time series in this example is hourly data. Therefore, our seasonal periods are:

- daily = 24
- weekly = 24 * 7 = 168.

Here you can clearly see the defined seasonal patterns, the trend, and the remaining residuals. Figure 4.8 shows the decomposition of the trend and seasonality, which then the residuals are modelled using an ARMA process.

Figure 4.8 – Decomposed Time Series

Autoregressive Moving Average (ARMA)

ARMA was discussed earlier as a subset from the ARIMA family.

The ARMA model in TBATS is used to model the remaining residuals to capture any autocorrelations of the lagged variables. The Autoregressive (AR) component captures to correlation between an observation and several lagged observations. This deals with the momentum or continuation of the series. The moving average (MA) component models the error terms as a linear combination of errors at previous time periods, capturing information not explained by the AR part alone.

Parameter Optimization

To select the optimal parameter space, TBATS will fit several models and automatically select the best parameters. A few of the models TBATS fits internally are:

- With and without Box-Cox Transformation
- With and without Trend
- With and without Trend Damping
- Season and non-seasonal model
- ARMA(p,q) parameters

The final model is chosen by which combination of parameters minimizes the Akaike Information Criterion (AIC), and auto-ARIMA is used to determine the ARMA parameters.

As with all forecasting methods, there are benefits and trade-offs to different models. While TBATS offers some enhancements on many other models' shortcomings, the trade-off is the need to build many models which result in longer computation time. This can pose a problem if having to model multiple time series. Additionally, TBATS does not allow for the inclusion of exogenous variables.

Practitioner's Note:

> *TBATS cannot handle exogenous regression since it is related to ETS models per Hyndman himself, suggests it is unlikely to include covariates (Hyndman, 2014). If external regressors are to be used, other methods such as ARIMAX or SARIMAX should be used. If the time series has complex seasonality, you can add Fourier features as covariates to your ARIMAX or SARIMAX model to help capture the seasonal patterns.*

This is implemented in **NIXLA**, and we can use the implementation shown here:

```
TBATS_model = TBATS(seasonal_periods  = 48, use_trend=T
```

In NIXTLA, you can also use AutoTBATS to let the system optimize how to handle the various parameters.

Let's see what the TBATS forecast looks like:

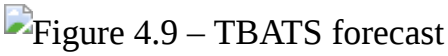


Figure 4.9 – TBATS forecast

Again, the seasonality pattern has been replicated, and is capturing most of the peaks in the forecast.

MSTL

The MSTL method in NIXTLA applies the LOESS technique to decompose a time series into its various seasonal components. Following this decomposition, it employs a specialized non-seasonal model to forecast the trend, and a SeasonalNaive model to predict each of the seasonal components. This approach allows for the detailed analysis and forecasting of time series with complex seasonal patterns.

```
MSTL_model = MSTL(season_length  = 48)
```

More information on the inner workings of MSTL was discussed in Chapter 3.

Let's see what the MSTL forecast looks like:

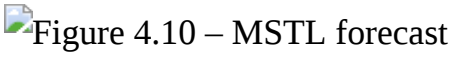


Figure 4.10 – MSTL forecast

Let's also take a look at how the different metrics that we chose did for each of these forecasts for the household we were experimenting with (from the notebook):

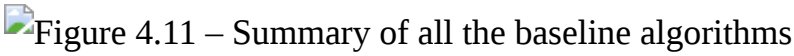


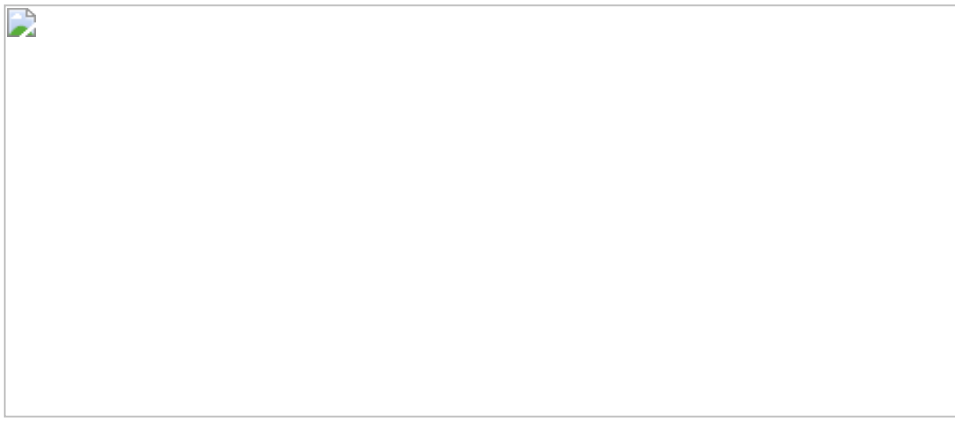
Figure 4.11 – Summary of all the baseline algorithms

Out of all the baseline algorithms we tried, AutoETS is performing the best on both MAE as well as MSE. ARIMA was the second-best model followed by TBATS. However, if you look at the **Time Elapsed** column, TBATS stands out taking just 7.4 seconds vs 19 seconds for ARIMA. Since they had similar performance, we will choose TBATS over ARIMA, along with AutoETS as our baseline, and run them on all 399 households in the dataset (both validation and test) we've chosen (the code for this is available in the **02-Baseline Forecasts using NIXTLA.ipynb** notebook).

Evaluating the baseline forecasts

Since we have the baseline forecasts generated from ETS as well as TBATS, we should also evaluate these forecasts. The aggregate metrics for all the selected households for both these methods are as follows:

Validation



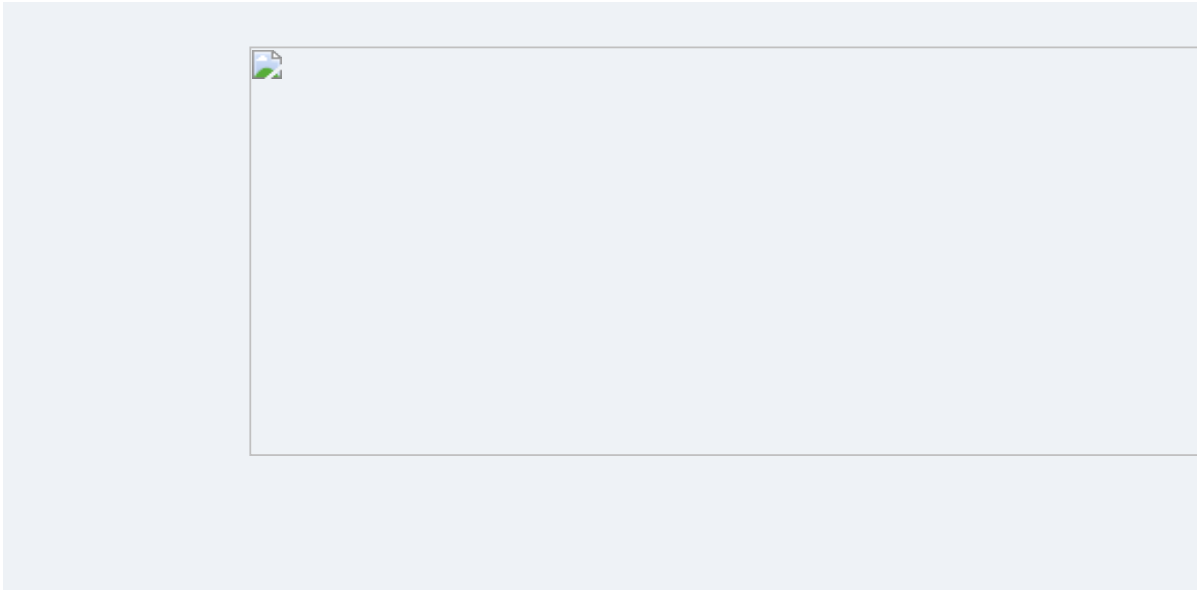


Figure 4.12 – The aggregate metrics of all the selected households (both validation and test)

It looks like AutoETS is performing much better in all three metrics. We also have these metrics calculated at a household level. Let's look at the distribution of these metrics in the validation dataset for all the selected households:

Figure 4.13 – The distribution of MASE and forecast bias of the baseline forecast in the validation dataset

Figure 4.13 – The distribution of MASE and forecast bias of the baseline forecast in the validation dataset

The MASE histogram of ETS seems to have a smaller spread than TBATS. ETS also has a lower median MASE than TBATS. We can see a similar pattern for forecast bias as well, with the forecast bias of ETS centered around zero and much less spread.

Back in Chapter 1, *Introducing Time Series*, we saw why every time series is not equally predictable and saw three factors to help us think about the issue – understanding the Data Generating Process (DGP), the amount of data, and adequately repeating the pattern. In most cases, the first two are pretty easy to evaluate, but the third one requires some analysis. Although the performance of baseline methods gives us some idea about how predictable any time series is, they still are model-dependent. So, instead of measuring how well a time series is forecastable, we might be better measuring how well the chosen model can approximate the time series. This is where a few techniques that are more fundamental (relying on the statistical properties of a time series) come in.

Assessing the forecastability of a time series

Although there are many statistical measures that we can use to assess the predictability of a time series, we will just look at a few that are easier to understand and practical when dealing with large time series datasets. The associated notebook (**02-Forecastability.ipynb**) contains the code to follow along.

Coefficient of Variation (CoV)

The **Coefficient of Variation** (**CoV**) relies on the intuition that the more variability that you find in a time series, the harder it is to predict it. And how do we measure variability in a random variable? **Standard deviation**.

In many real-world time series, the variation we see in the time series is dependent on the scale of the time series. Let's imagine that there are two retail products, *A* and *B*. *A* has a mean monthly sale of 15, while *B* has 50. If we look at a few real-world examples like this, we will see that if *A* and *B* have the same standard deviation, *B*,

which has a higher mean, is much more forecastable than *A*. To accommodate this phenomenon and to make sure we bring all the time series in a dataset to a common scale, we can use the CoV:

Here,

is the standard deviation and

is the mean of the time series, *n*.
The CoV is the relative dispersion of data points around the mean, which is much better than looking at the pure standard deviation.

The larger the value for the CoV, the worse the predictability of the time series. There is no hard cutoff, but a value of 0.49 is considered a rule of thumb to separate time series that are relatively easier to forecast from the hard ones. But depending on the general *hardness* of the dataset, we can tweak this cutoff. Something I have found useful is to plot a histogram of CoV values in a dataset and derive cutoffs based on that.

Even though the CoV is widely used in the industry, it suffers from a few key issues:

1. It doesn't consider seasonality. A sine or cosine wave will have a higher CoV than a horizontal line, but we know both are equally predictable.
2. It doesn't consider the trend. A linear trend will make a series have a higher CoV, but we know it is equally predictable like a horizontal line is.
3. It doesn't handle negative values in the time series. If you have negative values, it makes the mean smaller, thereby inflating the CoV.

To overcome these shortcomings, we propose another derived measure.

Residual variability (RV)

The thought behind residual variability (**RV**) is to try and measure the same kind of variability that we were trying to capture with the CoV but without the shortcomings. I was brainstorming on ways to avoid the problems of using the CoV, typically the seasonality issue, and was applying the CoV to the residuals after seasonal decomposition. It was then I realized that the residuals would have a few negative values and that the CoV wouldn't work well. Stefan de Kok, who is a thought leader in demand forecasting and probabilistic forecasting, suggested using the mean of the original actuals, which worked.

To calculate RV, you must perform the following steps:

1. Perform seasonal decomposition.
2. Calculate the standard deviation of the residuals or the irregular component.
3. Divide the standard deviation by the mean of the original observed values (before decomposition).

The key assumption here is that seasonality and trend are components that can be predicted. Therefore, our assessment of the predictability of a time series should only look at the variability of the residuals. But we cannot use CoV on the residuals because the residuals can have negative and positive values, so the mean of the residuals loses the interpretation of the level of the series and tends to zero. When residuals tend to zero, the CoV measure tends to infinity because of the division by mean. Therefore, we use the mean of the original series as the scaling factor.

Let's see how we can calculate RV for all the time series in our dataset (which are in a compact form):

```
block_df["rv"] = block_df.progress_apply(lambda x: calc
```

In this section, we looked at two measures that are based on the standard deviation of the time series. Now, let's look at assessing the forecastability of a time series.

Entropy-based measures

Entropy is a ubiquitous term in science. We see it popping up in Physics, quantum mechanics, social sciences, and information theory. And everywhere, it is used to talk about a measure of chaos or lack of predictability in a system. The entropy we are most interested in now is the one from Information Theory. Information Theory involves quantifying, storing, and communicating digital information.

Claude E. Shannon presented the qualitative and quantitative model of communication as a statistical process in his seminal paper *A Mathematical Theory of Communication*. While the paper introduced a lot of ideas, some of the concepts that

are relevant to us are Information Entropy and the concept of a *bit* – a fundamental unit of measurement of information.

REFERENCE CHECK

> *A Mathematical Theory of Communication by Claude E. Shannon is cited as reference 3 in the References section.*

The theory in itself is quite a lot to cover, but to summarize the key bits of information, take a look at the following short glossary:

- Information is nothing but a sequence of *symbols*, which can be transmitted from the *receiver* to the *sender* through a medium, which is called a *channel*. For instance, when we are texting somebody, the sequence of symbols are the letters/words of the language in which we are texting; the channel is the electronic medium.
- *Entropy* can be thought of as the amount of *uncertainty* or *surprise* in a sequence of symbols given some distribution of the symbols.
- *A bit*, as we mentioned earlier, is a unit of information and is a binary digit. It can either be 0 or 1.

Now, if we were to transfer 1 bit of information, it would reduce the uncertainty of the receiver by 2. To understand this better, let's consider a coin toss. We toss the coin in the air, and as it is spinning through the air, we don't know whether it is going to be heads or tails. But we know it is going to be one of these two. When the coin hits the ground and finally comes to rest, we find that it is heads. We can represent whether the coin toss is heads or tails with 1 bit of information (0 for heads and 1 for tails). So, the information that was passed to us when the coin fell reduced the possible

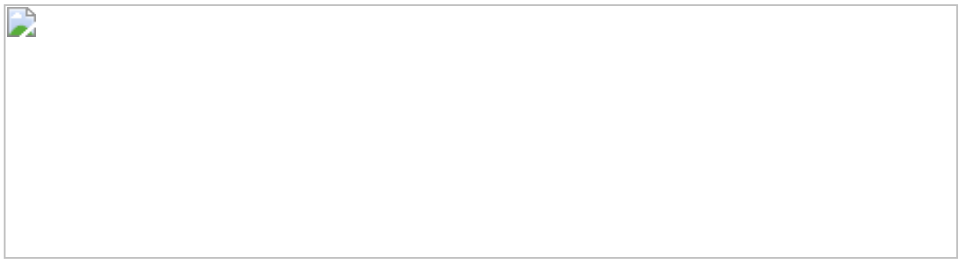
outcomes from two to one (heads). This transfer was possible with 1 bit of information.

In Information Theory, the entropy of a discrete random variable is the average level of *information*, *surprise*, or *uncertainty* inherent in the variable's possible outcomes. In more technical parlance, it is the expected number of bits required for the best possible encoding scheme of the information present in the random variable.

ADDITIONAL READING

> *If you want to intuitively understand entropy, cross-entropy, Kullback-Leibler divergence, and so on, head over to the Further reading section. There are a couple of links to blogs (one of which is my own) where we try to lay down the intuition behind these metrics).*

Entropy is formally defined as follows:



Here, X is the discrete random variable with possible outcomes, and

. Each of those outcomes has a probability of occurring, which is denoted by and

.

To develop some intuition around this, we can think that the more spread out a probability distribution is, the more chaos is in the distribution, and thus more entropy. Let's quickly check this in code:

```
# Creating an array with a well balanced probability di
flat = np.array([0.1,0.2, 0.3,0.2, 0.2])
# Calculating Entropy
print((-np.log2(flat)* flat).sum())
>> 2.2464393446710154
# Creating an array with a peak in probability
sharp = np.array([0.1,0.6, 0.1,0.1, 0.1])
# Calculating Entropy
print((-np.log2(sharp)* sharp).sum())
>> 1.7709505944546688
```

Here, we can see that the probability distribution that spread its mass has higher entropy.

In the context of a time series, *n* is the total number of time series observations, and

is the probability for each symbol of the time series alphabet. A sharp distribution means that the time series values are concentrated on a small area and should be easier to predict. On the other hand, a wide or flat distribution means that the time

series value can be equally likely across a wider range of values and hence is difficult to predict.
If we have two time series – one containing the result of a coin toss and the other containing the result of a dice throw – the dice throw would have any output between one and six, whereas the coin toss would be either zero or one. The coin toss time series would have lower entropy and be easier to predict than the dice throw time series.

But since time series is typically continuous, and entropy requires a discrete random variable, we can resort to a few strategies to convert the continuous time series into a discrete one. Many strategies, such as quantization or binning, can be applied, which leads to a myriad of complexity measures. Let's review one such measure that is useful and practical.

Spectral entropy

To calculate the entropy of a time series, we need to discretize the time series. One way to do that is by using FFT and **power spectral density (PSD)**. This discretization of the continuous time series is used to calculate spectral entropy.

We learned what Fourier Transform is earlier in this chapter and used it to generate a baseline forecast. But using FFT, we can also estimate a quantity called power spectral density. This answers the question, *How much of the signal is at a particular frequency?* There are many ways of estimating power spectral density from a time series, but one of the easiest ways is by using the **Welch method**, which is a non-parametric method based on Discrete Fourier Transform. This is also implemented as a handy function with the **periodogram(x)** signature in **scipy**.

The returned *PSD* will have a length equal to the number of frequencies estimated, but these are densities and not well-defined probabilities. So, we need to normalize

PSD to be between zero and one:

Here, *F* is the number of frequencies that are part of the returned power spectrum density.

Now that we have the probabilities, we can just plug this into the entropy formula and arrive at the spectral entropy:

When we introduced **entropy-based measures**, we saw that the more spread out the probability mass of a distribution is, the higher the entropy is. In this context, the more frequencies across which the spectral density is spread, the higher the spectral entropy. So, a higher spectral entropy means the time series is more complex and therefore more difficult to forecast.

Since FFT has an assumption of stationarity, it is recommended that we make the series stationary before using spectral entropy as a metric. We can even apply this metric to a detrended and deseasonalized time series, which we can refer to as

residual spectral entropy. This book's GitHub repository contains an implementation of spectral entropy under **src.forecastability.entropy.spectral_entropy**. This implementation also has a parameter, **transform_stationary**, which, if set to **True**, will detrend the series before we apply spectral entropy. Let's see how we can calculate spectral entropy for our dataset:

```
from src.forecastability.entropy import spectral_entrop
block_df["spectral_entropy"] = block_df.energy_consumpt
block_df["residual_spectral_entropy"] = block_df.residu
```

There are other entropy-based measures such as approximate entropy and sample entropy, but we will not cover those in this book. They are more computationally intensive and don't tend to work for time series that contain fewer than 200 values. If you are interested in learning more about these measures, head over to the *Further reading* section.

Another metric that takes a slightly different path is the Kaboudan metric.

Kaboudan metric

In 1999, Kaboudan defined a metric for time series predictability, calling it the

-metric. The idea behind it is very simple. If we block -shuffle a time series, we are essentially destroying the information in the time series. **Block shuffling** is the process of dividing the time series into blocks and then shuffling those blocks. So, if we calculate the **sum of squared errors** (**SSE**) of a forecast that's been trained on a

time series and then contrast it with the SSE of a forecast trained on a shuffled time series, we can infer the predictability of the time series. The formula to calculate this is as follows:

Here,

is the SSE of the forecast that was generated from the original time series, while

is the SSE of the forecast that was generated from the block-shuffled series. If the time series contains some predictable signals,

would be lower than

and

would approach one. This is because there was some information or patterns that were broken due to the block shuffling. On the other hand, if a series is just white noise (which is unpredictable by definition) there would be hardly any difference between

and

, and

would approach zero. In 2002, Duan investigated this metric and suggested some modifications in his thesis. One of the problems he identified, especially in long time series, is that the

values are found in a narrow band around 1 and suggested a slight modification to the formula. We call this the **modified Kaboudan metric**. The measure on the lower side is also clipped to zero. Sometimes, the metric can go below zero because

is lower than

, which is because the series is unpredictable and, by pure chance, block shuffling made the SSE lower:

REFERENCE CHECK

> *The research paper that proposed the Kaboudan metric is cited as reference 4 in the References section. The subsequent modification that Duan suggested is cited as reference 5.*

This modified version, as well as the original, has been implemented in this book's GitHub repository.

There is no restriction on the forecasting model you use to generate the forecast, which makes it a bit more flexible. Ideally, we can choose one of the classical statistical methods that is fast enough to be applied to the whole dataset. But this also makes the Kaboudan metric dependent on the model, and the limitations of the model

are inherent in the metric. The metric measures a combination of how difficult a series is to forecast and how difficult it is for the model to forecast the series.

Again, both metrics are implemented in this book's GitHub repository. Let's see how we can use them:

```
from src.forecastability.kaboudan import kaboudan_metri
model = Theta(theta=3, seasonality_period=48*7, season_
block_df["kaboudan_metric"] = [kaboudan_metric(r[0], mo
block_df["modified_kaboudan_metric"] = [modified_kaboud
```

Although there are many more metrics we can use for this purpose, the metrics we just reviewed for assessing forecastability cover a lot of the popular use cases and should be more than enough to gauge any time series dataset in regards to the difficulty of forecasting it. We can use these metrics to compare onetime series with another time series or to profile a whole set of related time series in a dataset with another dataset for benchmarking purposes.

ADDITIONAL READING

> *If you want to delve a little deeper and analyze the behavior of these metrics, how similar they are to each other, and how effective they are in measuring forecastability, go to the end of the* **03-Forecastability.ipynb** *notebook. We compute rank correlations among these metrics to understand how similar these metrics are. We can also find rank correlations with the computed metrics from the best performing baseline method to understand how well these metrics did in estimating the forecastability of a time series. I strongly encourage you to play around*

with the notebook and understand the differences between the different metrics. Pick a few time series and check how the different metrics give you slightly different interpretations.

Congratulations on generating your baseline forecasts – the first set of forecasts we have generated using this book! Feel free to head over to the notebooks and play around with the parameters of the methods and see how forecasts change. It'll help you develop an intuition around what the baseline methods are doing. If you are interested in learning more about how to make these baseline methods better, head over to the *Further reading* section, where we have provided a link to the paper *The Wisdom of the Data: Getting the Most Out of Univariate Time Series Forecasting*, by F. Petropoulos and E. Spiliotis.

Summary

And with this, we have come to the end of *Section 1*, *Getting Familiar with Time Series*. We have come a long way from just understanding what a time series is to generating competitive baseline forecasts. Along the way, we learned how to handle missing values and outliers and how to manipulate time series data using pandas. We used all those skills on a real-world dataset regarding energy consumption. We also looked at ways to visualize and decompose time series. In this chapter, we set up a test harness, learned how to use the **NIXTLA** library to generate a baseline forecast, and looked at a few metrics that can be used to understand the forecastability of a time series. For some of you, this may be a refresher, and we hope this chapter added some value in terms of some subtleties and practical considerations. For the rest of you, we hope you are in a good place, foundationally, to start venturing into modern techniques using machine learning in the next section of the book.

In the next chapter, we will discuss the basics of machine learning and delve into time series forecasting.

References

The following references were provided in this chapter:

- Assimakopoulos, Vassilis and Nikolopoulos, K.. (2000). *The theta model: A decomposition approach to forecasting*. International Journal of Forecasting. 16. 521-530. https://www.researchgate.net/publication/223049702_The_theta_model_A_decomposition_approach_to_forecasting.
- Rob J. Hyndman, Baki Billah. (2003). *Unmasking the Theta method*. International Journal of Forecasting. 19. 287-290. https://robjhyndman.com/papers/Theta.pdf.
- Shannon, C.E. (1948), *A Mathematical Theory of Communication*. Bell System Technical Journal, 27: 379-423. https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf.
- Kaboudan, M. (1999). *A measure of time series' predictability using genetic programming applied to stock returns*. Journal of Forecasting, 18, 345-357: http://www.aiecon.org/conference/efmaci2004/pdf/GP_Basics_paper.pdf.
- Duan, M. (2002). *TIME SERIES PREDICTABILITY*: https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.68.1898&rep=rep1&type=pdf.
- De Livera, A. M., & Hyndman, R. J. (2009). *Forecasting time series with complex seasonal patterns using exponential smoothing* (Department of Econometrics and Business Statistics Working Paper Series 15/09)

- Hyndman, Rob. "Rob J Hyndman - TBATS with Regressors." Rob J Hyndman, 6 Oct. 2014, http://robjhyndman.com/hyndsight/tbats-with-regressors

Further reading

To learn more about the topics that were covered in this chapter, take a look at the following resources:

- *Information Theory and Entropy*, by Manu Joseph: https://deep-and-shallow.com/2020/01/09/deep-learning-and-information-theory/.
- *Visual Information*, by Chris Olah: https://colah.github.io/posts/2015-09-Visual-Information.
- Fourier Transform: https://betterexplained.com/articles/an-interactive-guide-to-the-fourier-transform/.
- *Fourier Transform* by 3blue1brown – a visual introduction: https://www.youtube.com/watch?v=spUNpyF58BY&vl=en.
- *Understanding Fourier Transform by Example*, by Richie Vink: https://www.ritchievink.com/blog/2017/04/23/understanding-the-fourier-transform-by-example/.
- Delgado-Bonal A, Marshak A. *Approximate Entropy and Sample Entropy: A Comprehensive Tutorial*. Entropy. 2019; 21(6):541: https://www.mdpi.com/1099-4300/21/6/541.
- Yentes, J.M., Hunt, N., Schmid, K.K. et al. *The Appropriate Use of Approximate Entropy and Sample Entropy with Short Data Sets*. Ann Biomed Eng 41, 349–365 (2013): https://doi.org/10.1007/s10439-012-0668-3
- Ponce-Flores M, Frausto-Solís J, Santamaría-Bonfil G, Pérez-Ortega J, González-Barbosa JJ. *Time Series Complexities and Their Relationship to Forecasting Performance*. Entropy. 2020; 22(1):89. https://www.mdpi.com/1099-4300/22/1/89

- Petropoulos F, Spiliotis E. *The Wisdom of the Data: Getting the Most Out of Univariate Time Series Forecasting*. Forecasting. 2021; 3(3):478-497. https://doi.org/10.3390/forecast3030029

5 Time Series Forecasting as Regression

Join our book community on Discord

https://packt.link/EarlyAccess/

In the previous part of the book, we have developed a fundamental understanding of time series and equipped ourselves with tools and techniques to analyze and visualize time series and even generate our first baseline forecasts. We have mainly covered classical and statistical techniques in this book so far. Let's now dip our toes into modern **machine learning** and learn how we can leverage this comparatively newer field for **time series forecasting**. Machine learning is a field that has grown in leaps and bounds in recent times, and being able to leverage these newer techniques for time series forecasting is a skill that will be invaluable in today's world.

In this chapter, we will be covering these main topics:

- Understanding the basics of machine learning

- Time series forecasting as regression
- Local versus global models

Understanding the basics of machine learning

Before we get started with using machine learning for time series, let's spend some time establishing what machine learning is and setting up a framework to demonstrate what it does (if you are already very comfortable with machine learning, feel free to skip ahead, or just stay with us and refresh the concepts). In 1959, Arthur Samuel defined machine learning as a "*field of study that gives computers the ability to learn without being explicitly programmed*." Traditionally, programming has been a paradigm under which we know a set of rules/logic to perform an action, and that action is performed on the given data to get the output that we want. But, machine learning flipped this on its head. In machine learning, we start with data and the output, and we ask the computer to tell us about the rules with which the desired output can be achieved from the data:

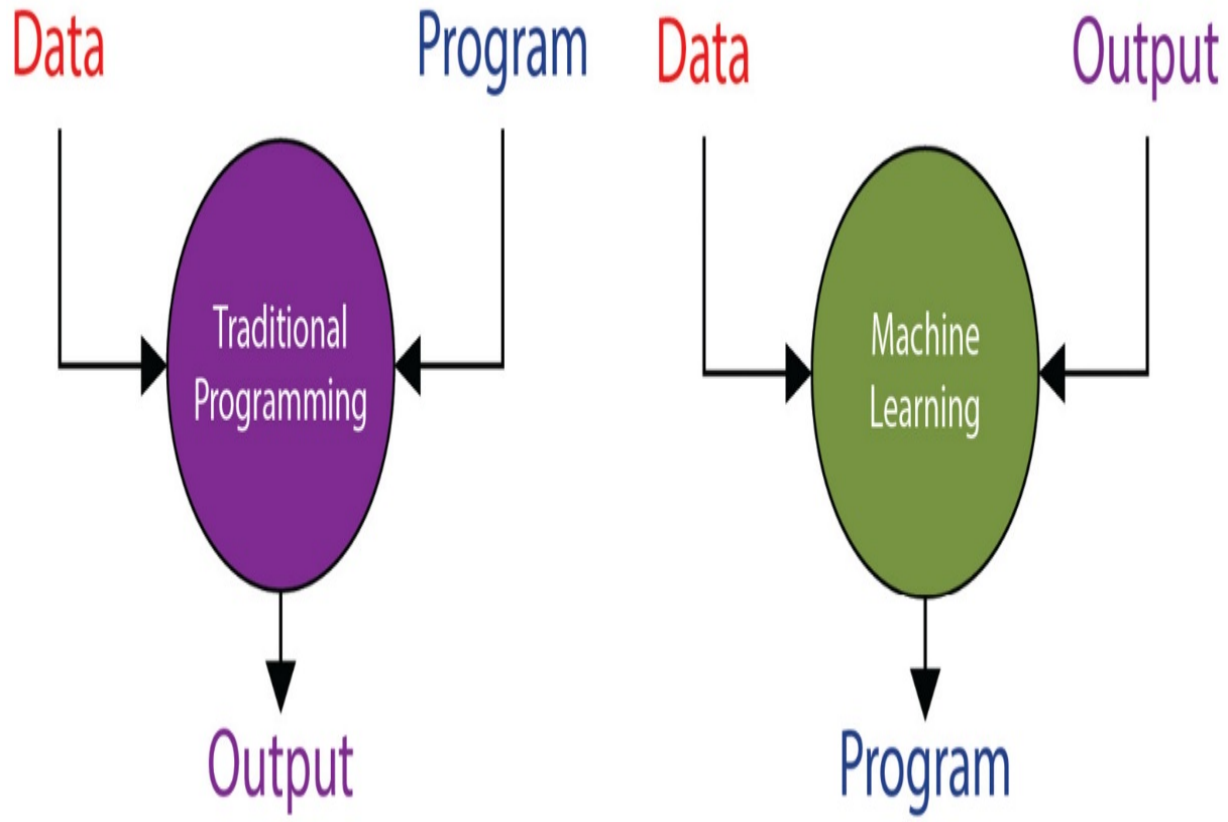


Figure 5.1 – Traditional programming versus machine learning

There are many kinds of problem settings in machine learning, such as supervised learning, unsupervised learning, self-supervised learning, and so on, but we will stick to supervised learning, which is the most popular one and the most applicable one to the contents of this book.

Let's start our discussion small and slowly build up to the whole schematic, which encapsulates most of the key components of a supervised machine learning problem:

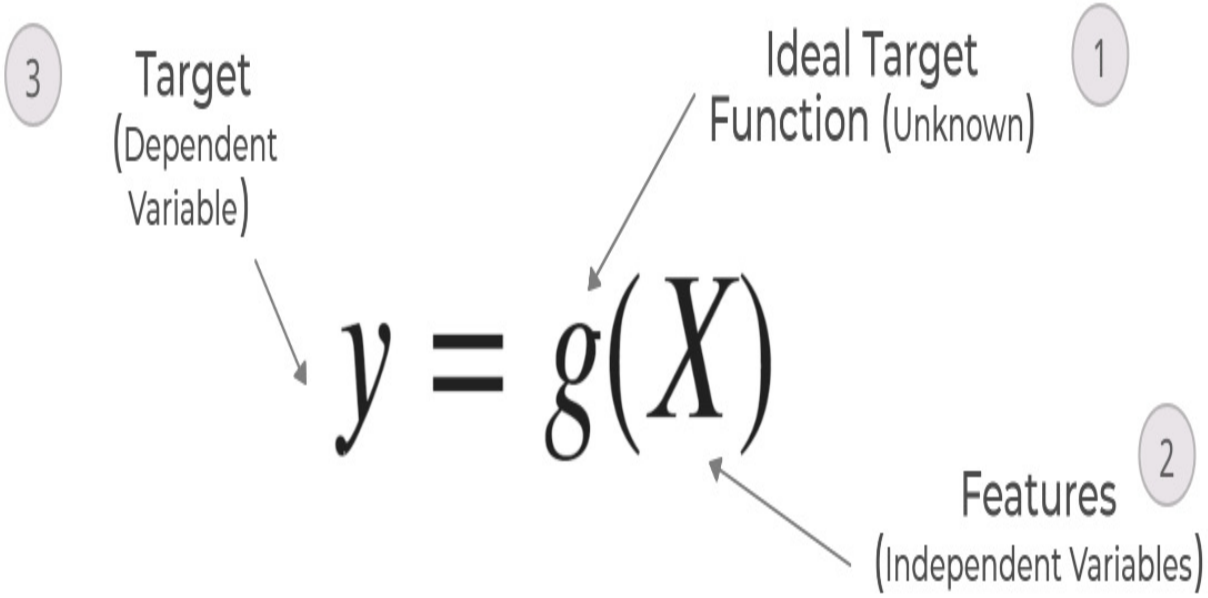


Figure 5.2 – Supervised machine learning schematic, part 1 – the ideal function

As we already discussed, what we want from machine learning is to *learn* from the data and come up with a set of rules/logic. The closest analogy in mathematics for logic/rules is a function, which takes in an input (here, data) and provides an output. Mathematically, it can be written as follows:

$$y = g(X)$$

where X is the set of features and g is the **ideal target function** (denoted by **1** in *Figure 5.2*) that maps the X input (denoted by **2** in the schematic) to the

target (ideal) output, *y* (denoted by **3** in the schematic). The ideal target function is largely an unknown function, similar to the **data generating process** (**DGP**) we saw in Chapter 1, *Introducing Time Series*, which is not in our control.

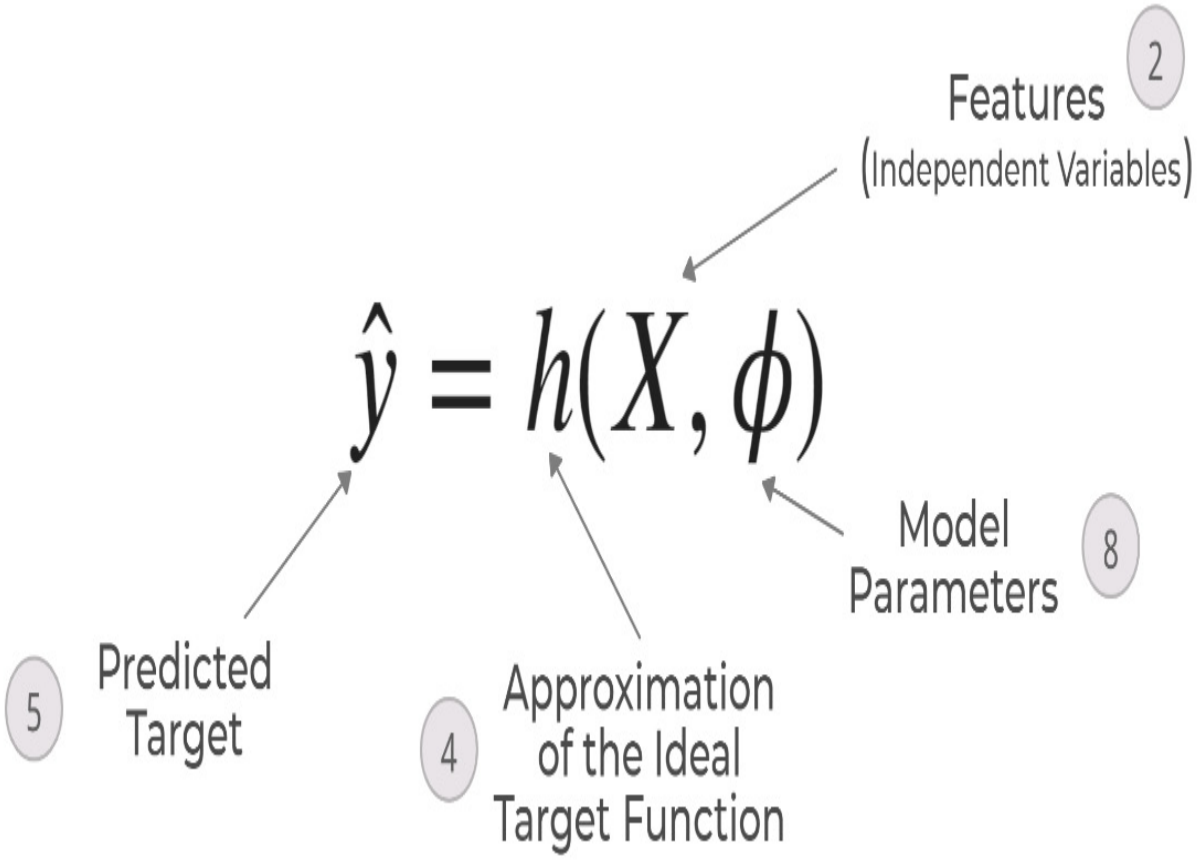


Figure 5.3 – Supervised machine learning schematic, part 2 – the learned approximation

But, we want the computer to *learn* this ideal target function. This approximation of the ideal target function is denoted by another function, *h* (**4** in the schematic), which takes in the same set of features, *X*, and outputs a predicted target, (**5** in the schematic). are the parameters of the *h* function (or model parameters):

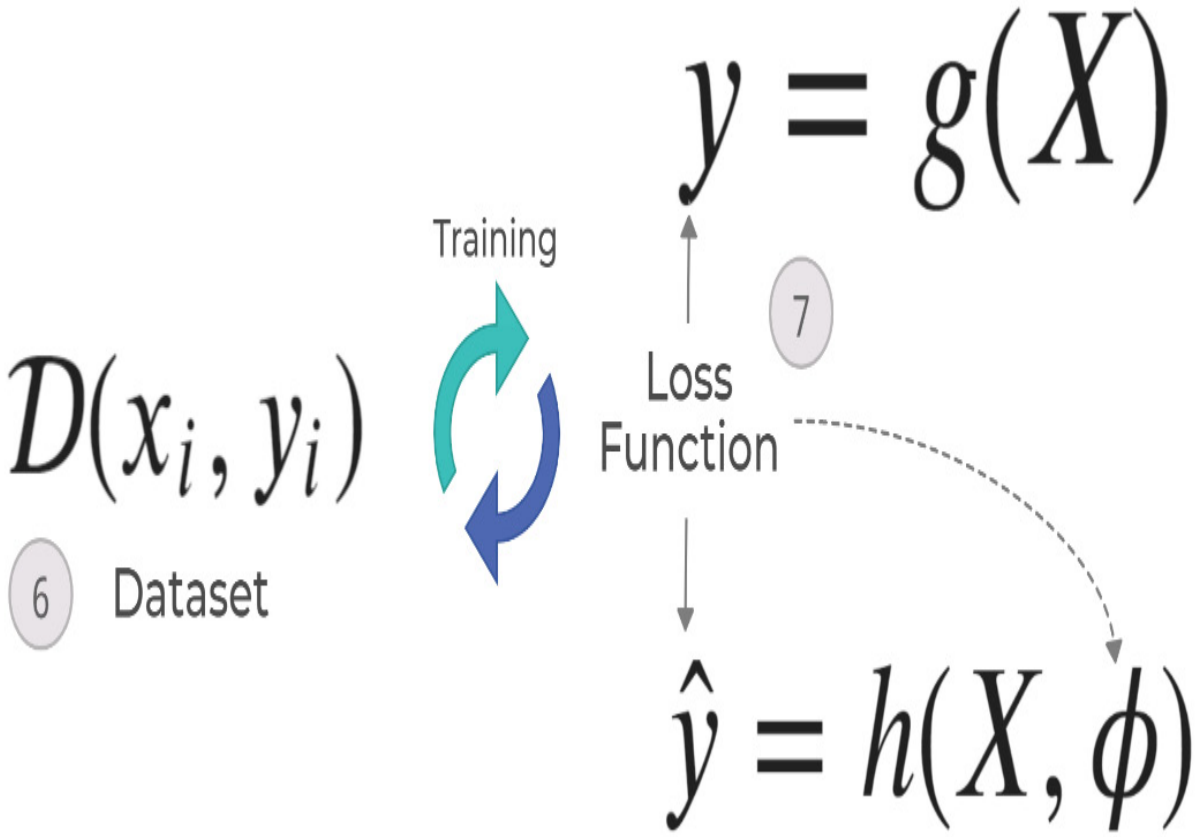


Figure 5.4 – Supervised machine learning schematic, part 3 – putting it all together

Now, how do we find this approximation *h* function and its parameters, ? With the dataset of examples (**6** in the schematic). The supervised machine learning problem works on the premise that we are able to collect a set of examples that shows the features, *X*, and the corresponding target, *y*, which is also referred to as *labels* in literature. It is from this set of examples (the dataset) that the computer *learns* the approximation function, *h*, and the optimal model parameters, . In the preceding diagram, the only real unknown entity is the ideal target function, *g*. So, we can use the training dataset, *D*, to get predicted targets for every sample in the dataset. We already know the ideal target for all the examples. We need a way to compare the ideal targets and predicted targets, and this is where the loss function (**7** in the schematic) comes in. This loss function tells us how far

away from the real truth we are with the approximated function, *h*. Although *h* can be any function, it is typically chosen from a set of a well-known class of functions, is the finite set of functions that can be fit to the data. This class of functions is what we colloquially call **models**. For instance, *h* can be chosen from all the linear functions or all the tree-based functions, and so on. Choosing an *h* from is done by a combination of hyperparameters (which the modeler specifies) and the model parameter, which is learned from data.

Now, all that is left is to run through the different functions so that we find the best approximation function, *h*, which gives us the lowest loss. This is an optimization process that we call **training**.

Let's also take a look at a few key concepts, which will be important in all our discussions ahead.

Supervised machine learning tasks

Machine learning can be used to solve a wide variety of tasks such as **regression**, **classification**, and **recommendation**. But, since classification and regression are the most popular classes of problems, we will spend just a little bit of time reviewing what they are.

The difference between classification and regression tasks is very simple. In the machine learning schematic (*Figure 5.2*), we talked about *y*, the target. This target can be either a real-valued number or a class of items. For instance, we could be predicting the stock price for next week or we could just predict whether the stock was going to go up or down. In the first case, we are predicting a real-valued number, which is called **regression**. In the

other case, we are predicting one out of two classes (*up* or *down*), and this is called **classification**.

Overfitting and underfitting

The biggest challenge in machine learning systems is that the model we trained must perform well on a new and unseen dataset. The ability of a machine learning model to do that is called the **generalization capability** of the model. The training process in a machine learning setup is akin to mathematical optimization, with one subtle difference. The aim of mathematical optimization is to arrive at the global maxima in the provided dataset. But in machine learning, the aim is to achieve low test error by using the training error as a proxy. How well a machine learning model is doing on training error and testing error is closely related to the concepts of overfitting and underfitting. Let's use an example to understand these terms.

The learning process of a machine learning model has many parallels to how humans learn. Suppose three students, *A*, *B*, and *C*, are studying for an examination. *A* is a slacker and went clubbing the night before. *B* decided to double down and memorize the textbook end to end. And, *C* paid attention in class and understood the topics up for the examination.

As expected, *A* flunked the examination, *C* got the highest score, and *B* did OK.

A flunked the examination because they didn't learn enough. This happens to machine learning models as well when they don't learn enough patterns, and this is called **underfitting**. This is characterized by high training errors and high test errors.

B didn't score as highly as expected; after all, they did memorize the whole text, word for word. But many questions in the examination weren't directly from the textbook and *B* wasn't able to answer them correctly. In other words, the questions in the examination were *new and unseen*. And because *B* memorized everything but didn't make an effort to understand the underlying concepts, *B* wasn't able to *generalize* the knowledge they had to new questions. This situation, in machine learning, is called **overfitting**. This is typically characterized by a big delta in training and test errors. Typically, we will see very low training errors and high test errors.

The third student, *C*, learned the right way and understood the underlying concepts and because of that, was able to *generalize* to *new and unseen* questions. This is the ideal state for a machine learning model, as well. This is characterized by reasonably low test errors and a small delta between training and test errors.

We just saw the two greatest challenges in machine learning. Now, let's also look at a few ways we have that can be used to tackle these challenges.

There is a close relationship between the **capacity** of a model and underfitting or overfitting. A model's capacity is its ability to be flexible enough to fit a wide variety of functions. Models with low capacity may struggle to fit the training data, leading to underfitting. Models with high capacity may overfit by memorizing the training data too much. Just to develop an intuition around this concept of capacity, let's look at an example. When we move from linear regression to polynomial regression, we are adding more capacity to the model. Instead of fitting just straight lines, we are letting the model fit curved lines as well. Machine learning models generally do well when their capacity is appropriate for the learning problem at hand.

$$\hat{y} = c + \sum_{i=1}^{N} w_i \times x_i$$

Figure 5.5 – Underfitting versus overfitting

Figure 5.5 shows a very popular case to illustrate overfitting and underfitting. We create a few random points using a known function and try to learn that by using those data samples. We can see that the linear regression, which is one of the simplest models, has underfitted the data by drawing a straight line through those points. Polynomial regression is linear

regression, but with some higher-order features. For now, you can consider the move from linear regression to polynomial regression with higher degrees as increasing the capacity of the model. So, when we use a degree of 4, we see that the learned function fits the data well and matches our ideal function. But if we keep increasing the capacity of the model and reach *degree = 15*, we see that the learned function is still passing through the training samples, but has learned a very different function, overfitting to the training data. Finding the optimal capacity to learn a generalizable function is one of the core challenges of machine learning.

While capacity is one aspect of the model, another aspect is **regularization**. Even with the same capacity, there are multiple functions a model can choose from the hypothesis space of all functions. With regularization, we try to give preference to a set of functions in the hypothesis space over the others. While all these functions are valid functions that can be chosen, we nudge the optimization process in such a way that we end up with a kind of function toward which we have a preference. Although regularization is a general term used to refer to any kind of constraint we place on the learning process to reduce the complexity of the learned function, more commonly, it is used in the form of a weight decay. Let's take an example of linear regression, which is when we fit a straight line to the input features by learning a weight associated with each feature.

A linear regression model can be written mathematically as follows:

Here, *N* is the number of features, *c* is the intercept, is the *i*th feature, and is the weight associated with the *i*th feature. We estimate the right weight (*L*) by considering this as an optimization problem that minimizes the error between

$$L + \lambda \sum_{i=1}^{N} w_i^2$$

and *y* (real output).

Now, with regularization, we add an additional term to *L*, which forces the weights to become smaller. Commonly, this is done using an *l1* or *l2* regularizer. An *l1* regularizer is when you add the sum of squared weights to *L*:

where

$$L + \lambda \sum_{i=1}^{N} |w_i|^2$$

is the regularization coefficient that determines how strongly we penalize the weights. An *l2* regularizer is when you add the sum of absolute weights to *L*:

In both cases, we are enforcing a preference for smaller weights over larger weights because it keeps the function from relying too much on any one feature from the ones used in the machine learning model. Regularization is an entire topic unto itself and if you want to learn more, head over to the *Further reading* section for a few resources on regularization.

Another really effective way to reduce overfitting is to simply train the model with more data. With a larger dataset, the chances of the model overfitting become less because of the sheer variety that can be captured in a large dataset.

Now, how do we tune the knobs to strike a balance between underfitting and overfitting? Let's look at it in the following section.

Hyperparameters and validation sets

Almost all machine learning models have a few hyperparameters associated with them. **Hyperparameters** are parameters of the model that are not learned from data but rather are set before the start of training. For instance, the weight of the regularization is a hyperparameter. Most hyperparameters either help us control the capacity of the model or apply regularization to the model. By controlling either capacity or regularization or both, we can

travel the frontier between underfitting and overfitting models and arrive at a model that is *just right*.

But since these hyperparameters have to be set outside of the algorithm, how do we estimate the best hyperparameters? Although it is not part of the core *learning process*, we learn the hyperparameters also from the data. But if we just use the training data to learn the hyperparameters, it will just choose the maximum possible model capacity, which results in overfitting. This is where we need a **validation set**, a part of the data that the training process does not have access to. But when the dataset is small (not hundreds of thousands of samples), the performance on a single validation set doesn't guarantee a fair evaluation. In such cases, we rely on **cross-validation**. The general trick is to repeat the training and evaluation procedure on different subsets of the original dataset. A common way of doing this is called **k-fold cross validation**, when the original dataset is divided into *k* equal, non-overlapping, and random subsets, and each subset is evaluated after training on all the other subsets. We have provided a link in the *Further reading* section if you want to read up about cross-validation techniques. Later in the book, we will also be covering this topic, but from the time series perspective, which has a few differences from the standard way of doing cross-validation.

SUGGESTED READING

> *Although we have touched the surface of machine learning in the book, there is a lot more, and to truly appreciate the rest of the book better, we suggest gaining more understanding of machine learning. We suggest starting with Machine Learning by Stanford (Andrew Ng) – https://www.coursera.org/learn/machine-learning. If you are in a hurry, the Machine Learning Crash Course by Google is also a good starting point – https://developers.google.com/machine-learning/crash-course/ml-intro.*

Now that we have a basic understanding of machine learning, let's start looking at how we can use it to do time series forecasting.

Time series forecasting as regression

A time series, as we saw in Chapter 1, *Introducing Time Series*, is a set of observations taken sequentially in time. And typically, time series forecasting is about trying to predict what these observations will be in the future. Given a sequence of observations of arbitrary length of history, we predict the future to an arbitrary horizon.

We saw that regression, or machine learning to predict a continuous variable, works on a dataset of examples, and each example is a set of input features and targets. We can see that regression, which is tasked with predicting a single output provided with a set of inputs, is fundamentally

incompatible with forecasting, where we are given a set of historical values and asked to predict the future values. This fundamental incompatibility between the time series and machine learning regression paradigms is why we cannot use regression for time series forecasting directly.

Moreover, time series forecasting, by definition, is an extrapolation problem, whereas regression, most of the time, is an interpolation one. Extrapolation is typically harder to solve using data-driven methods. Another key assumption in regression problems is that the samples used for training are **independent and identically distributed** (**IID**). But time series break that assumption as well because subsequent observations in a time series display considerable dependence.

However, to use the wide variety of techniques from machine learning, we need to cast time series forecasting as a regression. Thankfully, there are ways to convert a time series into a regression and get over the IID assumption by introducing some memory to the machine learning model through some features. Let's see how it can be done.

Time delay embedding

We talked about the ARIMA model in Chapter 4, *Setting a Strong Baseline Forecast*, and saw how it is an autoregressive model. We can use the same concept to convert a time series problem into a regression one. Let's use the following diagram to make the concept clear:

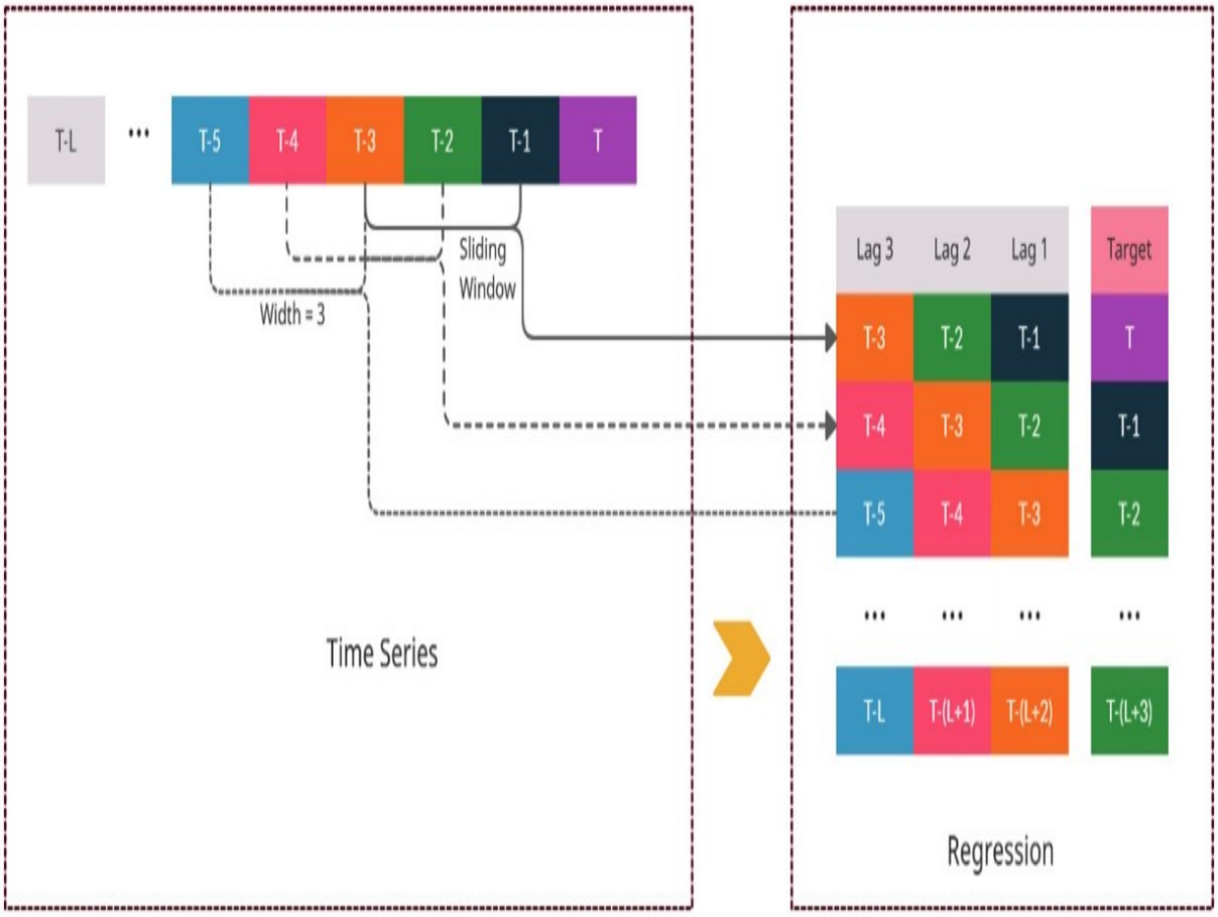


Figure 5.6 – Time series to regression conversion using a sliding window

Let's assume we have a time series with time steps, . Consider we are at time *t*, and we have a time series of the length of history as *L*. So, our time series will look something like in the diagram with

$$y_{t-1}, y_{t-2},$$

as the latest observation in the time series, and and so on as we move backward in time. In an ideal world, each observation in

$$y_{t-3}, y_{t-2}, y_{t-1}$$

should be conditioned on all the previous observations when we forecast. But, it is not practical because L can be arbitrarily long. We often restrict the forecasting function to use only the most recent M observations of the series, where $M < L$. These are called finite memory models or **Markov models** and M is called the order of autoregression, memory size, or the receptive field.

Therefore, in time delay embedding, we assume a window of arbitrary length $M < L$ and extract fixed-length subsequences from the time series by sliding the window over the length of the time series.

In the diagram, we have taken a sliding window with a memory size of 3. So, the first subsequence we can extract (if we are starting from the most recent and working backward) is . And

$$y_{t-4}, y_{t-3}, y_{t-2}$$

is the observation that comes right after the subsequence. This becomes our first example in the dataset (row 1 in the table in the diagram). Now, we slide the window one time step to the left (backward in time) and extract the new subsequence,

$$y_{t-1}$$

. The corresponding target would become . We repeat this process as we move back to the beginning of the time series, and at each step of the

sliding window, we add one more example to the dataset.
At the end of it, we have an aligned dataset with a fixed vector size of features (which will be equal to the window size) and a single target, which is what a typical machine learning dataset looks like.

Now that we have a table with three features, let's also assign semantic meaning to the three features. If we look at the right-most column in the table in the diagram, we can see that the time step present in the column is always one time step behind the target. We call it **Lag 1**. The second column from the right is always two time steps behind the target, and this is called **Lag 2**. Generalizing this, the feature that has observations that are *n* time steps behind the target, we call **Lag n**.

This transformation from time series to regression using **time-delay embedding** encodes the autoregressive structure of a time series in a way that can be utilized by standard regression frameworks. Another way we can think about using regression for time series forecasting is to perform **regression on time**.

Temporal embedding

If we rely on previous observations in autoregressive models, we rely on the concept of time for temporal embedding models. The core idea is that we forget the autoregressive nature of the time series and assume that any value in the time series is only dependent on time. We derive features that capture time, the passage of time, periodicity of time, and so on, from the

timestamps associated with the time series, and then we use these features to predict the target using a regression model. There are many ways to do this, from simply aligning a monotonically and uniformly increasing numerical column that captures the passage of time to sophisticated **Fourier** terms to capture the periodic components in time. We will talk about those techniques in detail in Chapter 6, *Feature Engineering for Time Series Forecasting*.

Before we wind up the chapter, let's also talk about a key concept that is gaining ground steadily in the time series forecasting space. A large part of this book embraces this new paradigm of forecasting.

Global forecasting models – a paradigm shift

Traditionally, each time series was treated in isolation. Because of that, traditional forecasting has always looked at the history of a single time series alone in fitting a forecasting function. But recently, because of the ease of collecting data in today's digital-first world, many companies have started collecting large amounts of time series from similar sources, or related time series.

For example, retailers such as Walmart collect data on sales of millions of products across thousands of stores. Companies such as Uber or Lyft collect the demand for rides from all the zones in a city. In the energy sector,

energy consumption data is collected across all consumers. All these sets of time series have shared behavior and are hence called **related time series**.

We can consider that all the time series in a related time series come from separate **data generating processes** (**DGPs**), and thereby model them all separately. We call these the **local** models of forecasting. An alternative to this approach is to assume that all the time series are coming from a single DGP. Instead of fitting a separate forecast function for each time series individually, we fit a single forecast function to all the related time series. This approach has been called **global** or **cross-learning** in literature.

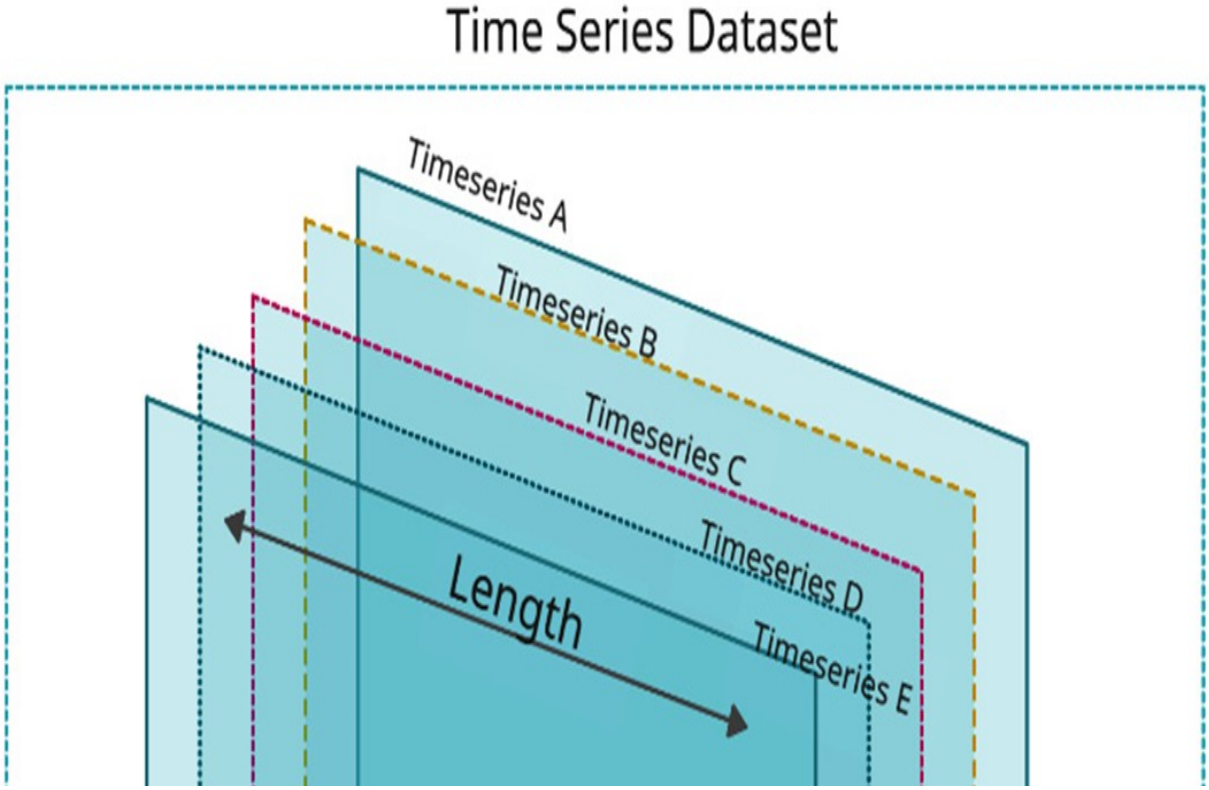
REFERENCE CHECK

> *The terminology global was introduced by David Salinas et al. in the DeepAR paper (reference number 1) and Cross-learning by Slawek Smyl (reference number 2).*

We saw earlier that having more data will lead to lower chances of overfitting and, therefore, lower generalization error (the difference between training and testing errors). This is exactly one of the shortcomings of the local approach. Traditionally, time series are not very long, and in many cases, it is difficult and time-consuming to collect more data as well. Fitting a machine learning model (with all its expressiveness) on small data is prone to overfitting. This is why time series models that enforce strong priors were used to forecast such time series, traditionally. But these strong

priors, which restrict the fitting of traditional time series models, can also lead to a form of underfitting and limit accuracy.

Strong and expressive data-driven models, as in machine learning, require a larger amount of data to have a model that generalizes to new and unseen data. A time series, by definition, is tied to time, and sometimes, collecting more data means waiting for months or years and that is not desirable. So, if we cannot increase the *length* of the time-series dataset, we can increase the *width* of the time series dataset. If we add multiple time series to the dataset, we increase the width of the dataset, and there by increase the amount of data the model is getting trained with. *Figure 5.7* shows the concept of increasing the width of a time series dataset visually:



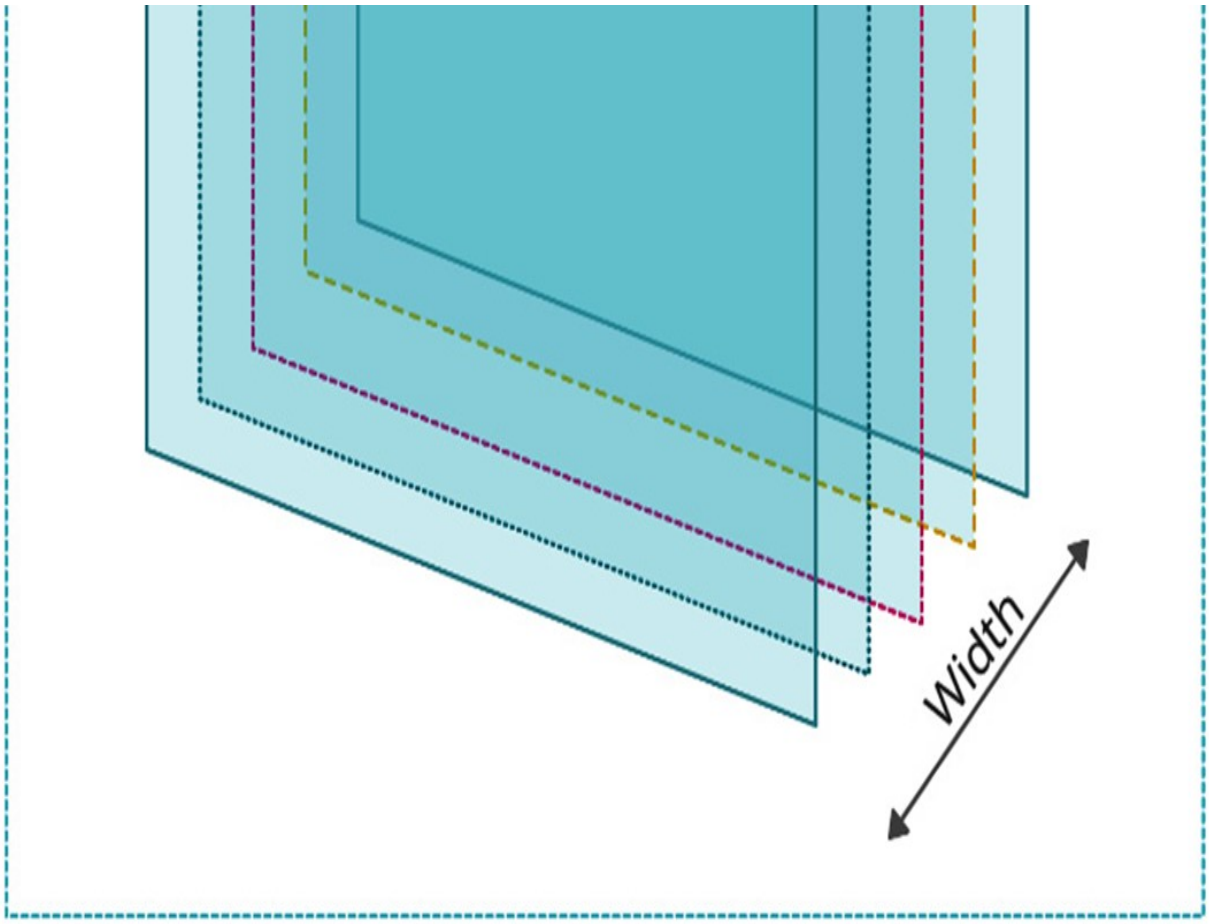


Figure 5.7 – The length and width of a time series dataset

This works in favor of machine learning models because with higher flexibility in fitting a forecast function and the addition of more data to work with, the machine learning model can learn a more complex forecast function than traditional time series models, which are typically shared between the related time series, in a completely data-driven way.

Another shortcoming of the local approach revolves around scalability. In the case of Walmart we mentioned earlier, there are millions of time series that need to be forecasted and it is not possible to have human oversight on

all these models. If we think about this from an engineering perspective, training and maintaining millions of models in a production system would give any engineer a nightmare. But under the global approach, we only train a single model for all these time series, which drastically reduces the number of models we need to maintain and yet can generate all the required forecasts.

This new paradigm of forecasting has gained traction and has consistently been shown to improve the local approaches in multiple time series competitions, mostly in datasets of related time series. In Kaggle competitions, such as *Rossman Store Sales* (2015), *Wikipedia WebTraffic Time Series Forecasting* (2017), *Corporación Favorita Grocery Sales Forecasting* (2018), and *M5 Competition* (2020), the winning entries were all global models—either machine learning or deep learning or a combination of both. The *Intermarché Forecasting Competition* (2021) also had global models as the winning submissions. Links to these competitions are provided in the *Further reading* section.

Although we have many empirical findings where the global models have outperformed local models for related time series, global models are still a relatively new area of research. *Montero-Manson and Hyndman* (2020) showed a few very interesting results and showed that any local method can be approximated by a global model with required complexity, and the most interesting finding they put forward is that the global model will perform better, even with unrelated time series. We will talk more about global

models and strategies for global models in Chapter 10, *Global Forecasting Models*.

REFERENCE CHECK

> *The Montero-Manson and Hyndman (2020) research paper is cited in References under reference number 3.*

Summary

We have started our journey beyond baseline forecasting methods and dipped our toes into the world of machine learning. After a brief refresher on machine learning, where we looked at key concepts such as overfitting, underfitting, regularization, and so on, we saw how we can convert a time series forecasting problem into a regression problem from the machine learning world. We also developed a conceptual understanding of different embeddings, such as time delay embedding and temporal embedding, which can be used to convert a time series problem into a regression problem. To wrap things up, we also learned about a new paradigm in time series forecasting – global models – and contrasted them with local models on a conceptual level. In the next few chapters, we will start putting these concepts into practice, and see techniques for feature engineering, and strategies for global models.

References

Following are the references that we used in this chapter:

1. David Salinas, Valentin Flunkert, Jan Gasthaus, Tim Januschowski (2020). *DeepAR: Probabilistic forecasting with autoregressive recurrent networks*. International Journal of Forecasting. 36-3. 1181-1191: https://doi.org/10.1016/j.ijforecast.2019.07.001
2. Slawek Smyl (2020). *A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting*. International Journal of Forecasting. 36-1: 75-85 https://doi.org/10.1016/j.ijforecast.2019.03.017
3. Montero-Manso, P., Hyndman, R.J.. (2020), *Principles and algorithms for forecasting groups of time series: Locality and globality*. arXiv:2008.00444[cs.LG]: https://arxiv.org/abs/2008.00444

Further reading

You can check out the following resources for further reading:

- *Regularization for Sparsity from Google Machine Learning Crash Course*: https://developers.google.com/machine-learning/crash-course/regularization-for-sparsity/l1-regularization
- *L1 and L2 Regularization from Foundations of Machine Learning, Bloomberg ML EDU*: https://www.youtube.com/watch?v=d6XDOS4btck
- *Cross-validation: evaluating estimator performance from scikit-learn*: https://scikit-learn.org/stable/modules/cross_validation.html

- *Rossmann Store Sales*: https://www.kaggle.com/c/rossmann-store-sales
- *Web Traffic Time Series Forecasting* – https://www.kaggle.com/c/web-traffic-time-series-forecasting
- *Corporación Favorita Grocery Sales Forecasting* – https://www.kaggle.com/c/favorita-grocery-sales-forecasting
- *M5 Forecasting – Accuracy* – https://www.kaggle.com/c/m5-forecasting-accuracy

6 Feature Engineering for Time Series Forecasting

Join our book community on Discord

https://packt.link/EarlyAccess/

In the previous chapter, we started looking at **machine learning** (**ML**) as a tool to solve the problem of **time series forecasting**. We also talked about a few techniques such as **time delay embedding** and **temporal embedding**, which cast time series forecasting problems as classical regression problems from the ML paradigm. In this chapter, we'll look at those techniques in detail and go through them in a practical sense using the dataset we have been working with throughout this book.

In this chapter, we will cover the following topics:

Feature engineering

Avoiding data leakage

Setting a forecast horizon

Time delay embedding

Temporal embedding

Technical requirements

- You will need to set up the **Anaconda** environment following the instructions in the *Preface* of the book to get a working environment with all the libraries and datasets required for the code in this book. Any additional library will be installed while running the notebooks

You will need to run the following notebooks before using the code in this chapter:

- **02-Preprocessing London Smart Meter Dataset.ipynb from Chapter02**
- **01-Setting up Experiment Harness.ipynb from Chapter04**

The code for this chapter can be found at https://github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-/tree/main/notebooks/Chapter06.

Feature engineering

Feature engineering, as the name suggests, is the process of engineering features from the data, mostly using domain knowledge, to make the learning process smoother and more efficient. In a typical ML setting, engineering good features is essential to get good performance from any ML model. Feature engineering is a highly subjective part of ML where each problem at hand has a different path – one that is hand-crafted to that problem.

When we are casting a time series problem as a regression problem, there are a few standard techniques that we can apply. This is a key step in the process because how well an ML model acquires an understanding of *time* is dependent on how well we engineer features to capture *time*. The baseline methods we covered in Chapter 4, *Setting a Strong Baseline Forecast*, are the methods that are created for the specific use case of time series forecasting and because of that, the temporal aspect of the problem is built into those models. For instance, ARIMA doesn't need any feature engineering to understand time because it is built into the model. But a standard regression model has no explicit

understanding of time, so we need to create good features to embed the temporal aspect of the problem.

In the previous chapter (Chapter 5, *Time Series Forecasting as Regression*), we talked about two main ideas to encode time into the regression framework: **time delay embedding** and **temporal embedding**. Although we touched on these concepts at a high level, it is time to dig deeper and see them in action.

NOTEBOOK ALERT

To follow along with the complete code, use the **01-Feature Engineering.ipynb** *notebook in the* **chapter06** *folder.*

We have already split the dataset that we were working on into train, validation, and test datasets. But since we are generating features that are based on previous observations, operationally, it is better when we have the train, validation, and test datasets combined. It will be clearer why shortly, but for now, let's take it on faith and move ahead. Now, let's go ahead and combine the two datasets:

```
# Reading the missing value imputed and train test split d
train_df = pd.read_parquet(preprocessed / "selected_blocks
val_df = pd.read_parquet(preprocessed / "selected_blocks_v
test_df = pd.read_parquet(preprocessed / "selected_blocks_
#Adding train, validation and test tags to distinguish the
train_df['type'] = "train"
val_df['type'] = "val"
test_df['type'] = "test"
full_df = pd.concat([train_df, val_df, test_df]).sort_valu
del train_df, test_df, val_df
```

Now, we have a **full_df** that combines the train, validation, and test datasets. Some of you may already have alarm bells ringing in your head at combining the train and test sets. What about **data leakage**? Let's check it out.

Avoiding data leakage

Data leakage occurs when the model is trained with some information that would not be available at the time of prediction. Typically, this leads to high performance in the training set, but very poor performance in unseen data. There are two types of data leakage:

- **Target leakage** is when the information about the target (that we are trying to predict) leaks into some of the features in the model, leading to an overreliance of the model on those features, ultimately leading to poor generalization. This includes features that use the target in any way.
- **Train-test contamination** is when there is some information leaking between the train and test datasets. This can happen because of careless handling and splitting of data. But it can also happen in more subtle ways, such as scaling the dataset before splitting the train and test sets.

When we are working with time series forecasting problems, the biggest and most common mistake that we can make is target leakage. We will have to think hard about each of the features to ensure we are not using any data that will not be available during prediction. The following diagram will help us remember and internalize this concept:

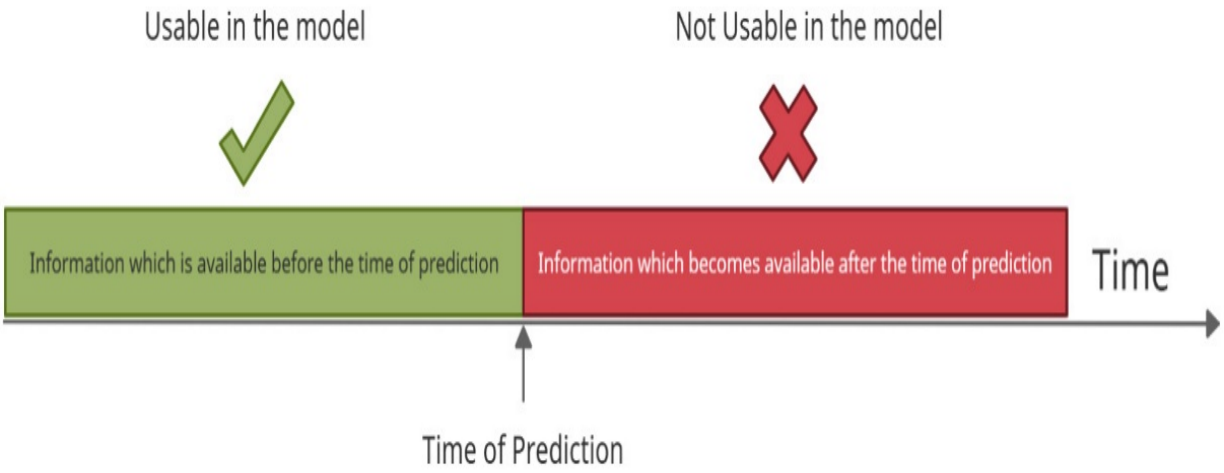


Figure 6.1 – Usable and not-usable information to avoid data leakage

To make this concept clearer and more relevant to the time series forecasting context, let's look at an example. Let's say we are forecasting sales for shampoo, and we are using sales for conditioner as a feature. We developed the model, trained it on the training data, and tested it on the validation data. The model is doing very well. The moment we start predicting for the future, we will see the problem. We don't know what the sales for conditioner are in the future either. While this example is pretty straightforward, there will be times when this becomes not so obvious. And that is why we need to exercise a fair amount of caution while creating features and always evaluate the features through the lens of, *will this feature be available at the time of prediction?*

BEST PRACTICES

There are many ways of identifying target leakage, apart from thinking hard about the features:

• If the model you've built is too good to be true, you most likely have a leakage problem

- *If any single feature has too much weightage in the feature importance of the model, that feature may have a problem with leakage*

- *Double-check the features that are highly correlated with the target*

Now, let's learn about forecast horizons.

Setting a forecast horizon

Although we generated forecasts earlier in this book, we never explicitly discussed **forecast horizons**. A forecast horizon is the number of time steps into the future we want to forecast at any point in time. For instance, if we want to forecast the next 24 hours for the electricity consumption dataset that we have been working with, the forecast horizon becomes 48 (because the data is half-hourly). In Chapter 5, *Time Series Forecasting as Regression,* where we generated baselines, we just predicted the entire test data at once. In such cases, the forecast horizon becomes equal to the length of the test data.

We never had to worry about this until now because, in the classical statistical methods of forecasting, this decision is decoupled from modeling. If we train a model, we can use that model to predict any future point without retraining. But with *time series forecasting as regression*, we have a constraint on the forecast horizon and it has its roots in data leakage. For now, let's only look at single-step-ahead forecasting. In the context of the dataset we are working with, this means that we will be answering the question, *what is the energy consumption in the next half an hour?* We will talk about multi-step forecasting and other mechanics of forecasting in *Section 4 – The Mechanics of Forecasting*.

Now that we have set some ground rules, let's start looking at the different feature engineering techniques. To follow along with the Jupyter notebook, head over to the **chapter06** folder and use the **01-Feature Engineering.ipynb** file.

Time delay embedding

The basic idea behind time delay embedding is to embed time in terms of recent observations. If you want to head back to Chapter 5, *Time Series Forecasting as Regression*, and review this concept (*Figure 5.1*), please go ahead and do so now.

In *Figure 5.1*, we talked about including previous observations of a time series as **lags**. However, there are a few more ways to capture recent and seasonal information using this concept. Let's take a look.

Lags or backshift

Let's assume we have a time series with time steps, . Consider that we are at time *t* and that we have a time series where the length of history is *L*. So, our time series will have as the latest observation in the time series, and then

$$y_{t-1}, y_{t-2},$$

and so on as we move back in time. So, lags, as explained in Chapter 5, *Time Series Forecasting as Regression*, are features that include the previous observations in the time series, as shown in the following diagram:

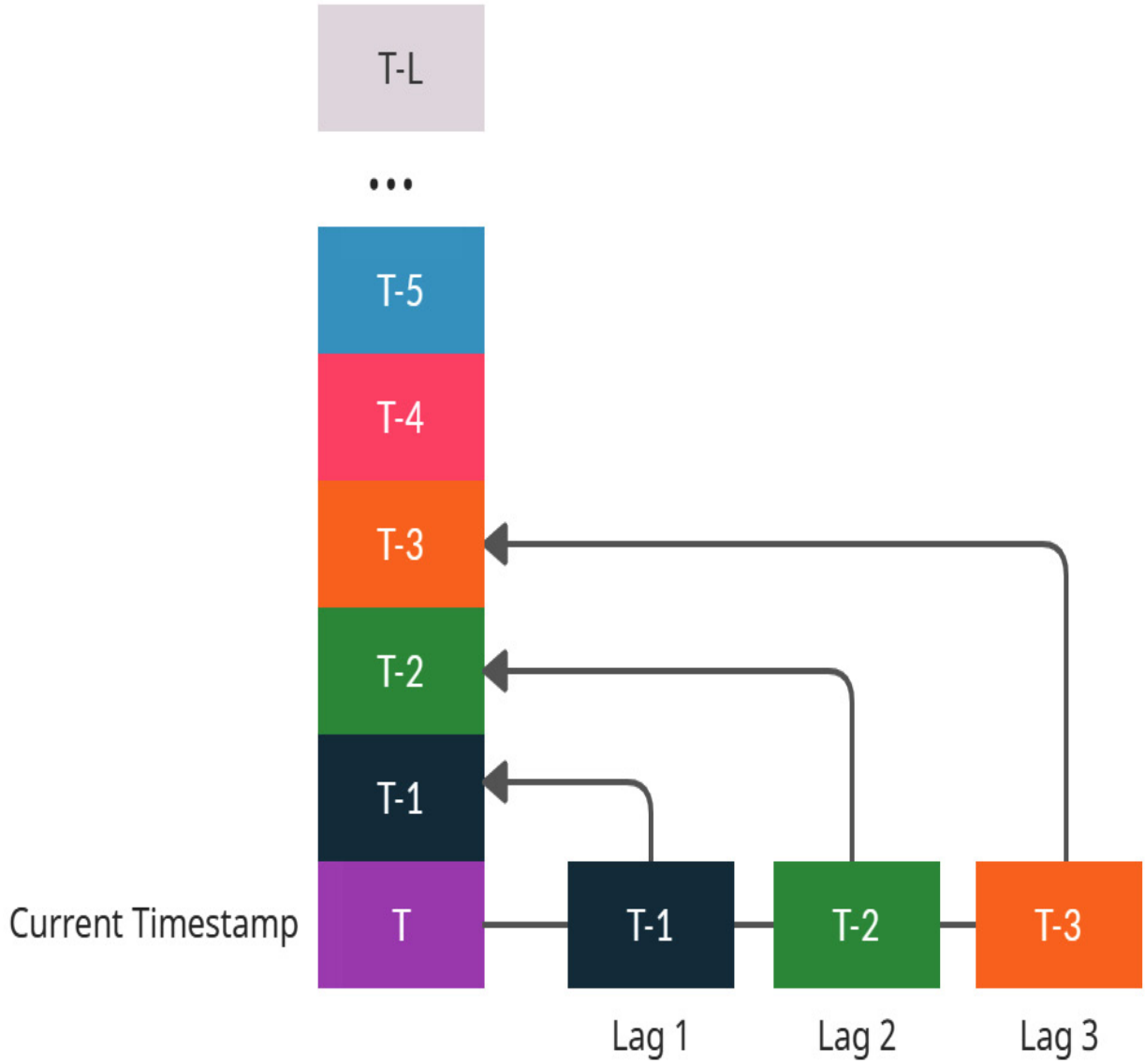


Figure 6.2 – Lag features (Color figure)

We can create multiple lags by including observations that are *a* timesteps before ( ); we will call this *Lag a*. In the preceding diagram, we have shown **Lag 1**, **Lag 2**, and **Lag 3**. However, we can add any number of lags we like. Now, let's learn how to do that in code:

```
df["lag_1"]=df["column"].shift(1)
```

Remember when we combined the train and test datasets and I asked you to take it in good faith? It's time to put a reason to that faith. If we consider the lag operation (or any autoregressive feature), it relies on a continuous representation along the time axis. If we consider the test dataset, for the first few rows (or earliest dates), the lags would be missing because it is part of the training dataset. So, by combining the two, we create a continuous representation along the time axis where standard functions in pandas such as **shift** can be utilized to create these features easily and efficiently.

It is as simple as that, but we need to perform the lag operation for each **LCLid** separately. We have included a helpful method in **src.feature_engineering.autoregressive_features** called **add_lags** that adds all the lags you want for each **LCLid** in a fast and efficient manner. Let's see how we can use that.

We are going to import the method and use a few of its parameters to configure the lag operation the way we want:

```
from src.feature_engineering.autoregressive_features impor
# Creating first 5 lags and then same 5 lags but from prev
lags = (
    (np.arange(5) + 1).tolist()
    + (np.arange(5) + 46).tolist()
    + (np.arange(5) + (48 * 7) - 2).tolist()
)
full_df, added_features = add_lags(
    full_df, lags=lags, column="energy_consumption", ts_id
)
```

Now, let's look at the parameters that we used in the previous code snippet:

- **lags**: This parameter takes in a list of integers denoting all the lags we need to create as features.
- **column**: The name of the column to be lagged. In our case, this is **energy_consumption**.
- **ts_id**: The name of the column that contains the unique ID of a time series. If **None**, it assumes the DataFrame only contains a single time series. In our case, **LCLid** is the name of that column.
- **use_32_bit**: This parameter doesn't do anything functionally but makes the DataFrames much smaller in memory, sacrificing the precision of the floating-point numbers.

This method returns the DataFrame with the lags added and a list with column names of the newly added features.

Rolling window aggregations

With lags, we were connecting the present points to single points in the past, but with rolling window features, we are connecting the present with an aggregate statistic of a window from the past. Instead of looking at the observation from previous time steps, we would be looking at an average of the observations from the last three timesteps. Take a look at the following diagram to understand this better:

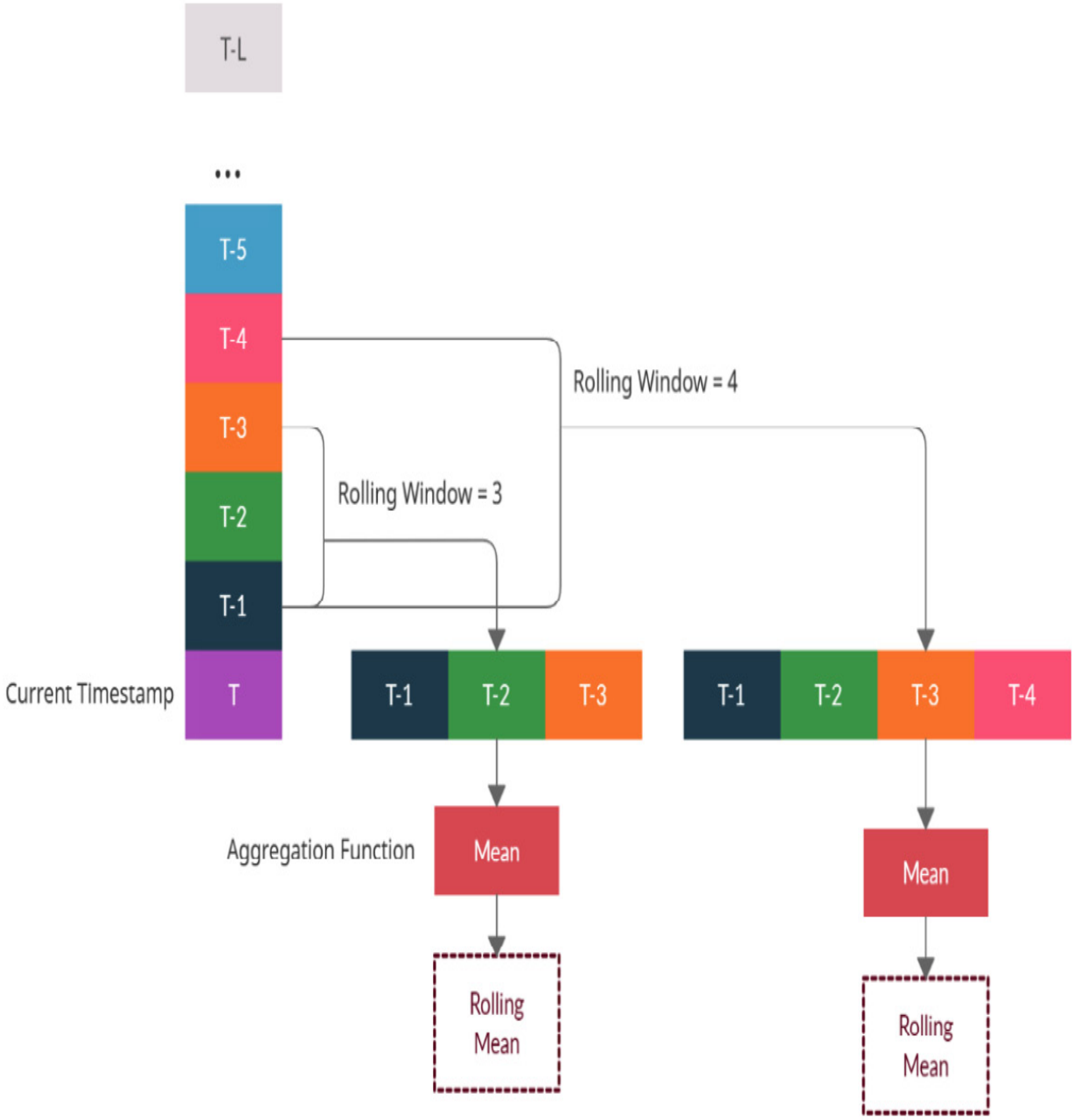


Figure 6.3 – Rolling window aggregation features(Color figure)

We can calculate rolling statistics with different widows, and each of them would capture slightly different aspects of the history. In the preceding diagram, we can see an example of a window of three and a window of four. When we are at timestep *t*, a rolling window of three would have

$$y_{t-3}, y_{t-2}, y_{t-1}$$

as the vector of past observations. Once we have these, we can apply any aggregation functions, such as the mean, standard deviation, min, max, and so on. Once we have a scalar value after the aggregation function, we can include that as a feature for timestep *t*.

NOTE

We are not including

in the vector of past observations because that leads to data leakage.

Let's see how we can do this with pandas:

```
# We shift by one to make sure there is no data leakage
df["rolling_3_mean"] = df["column"].shift(1).rolling(3).me
```

Similar to the lags, we need to do this operation for each **LCLid** column separately. We have included a helpful method in **src.feature_engineering.autoregressive_features** called **add_rolling_features** that adds all the rolling features you want for each **LCLid** in a fast and efficient manner. Let's see how we can use that.

We are going to import this method and use a few of its parameters to configure the rolling operation the way we want:

```
from src.feature_engineering.autoregressive_features impor
full_df, added_features = add_rolling_features(
    full_df,
    rolls=[3, 6, 12, 48],
    column="energy_consumption",
    agg_funcs=["mean", "std"],
    ts_id="LCLid",
```

```
        use_32_bit=True,
    )
```

Now, let's look at the parameters that we used in the previous code snippet:

- **rolls**: This parameter takes in a list of integers denoting all the windows over which we need to calculate the aggregate statistics.
- **column**: The name of the column to be lagged. In our case, this is **energy_consumption**.
- **agg_funcs**: This is a list of aggregations that we want to do for each window we declared in **rolls**. Allowable aggregation functions include **{mean, std, max, min}**.
- **n_shift**: This is the number of timesteps we need to shift before doing the rolling operation. This parameter avoids data leakage. Although we are shifting by one now, there are cases where we need to shift by more than one as well. This is typically used in multi-step forecasting, which we will cover in *Section 3 – Mechanics of Forecasting*.
- **ts_id**: The name of the column name that contains the unique ID of a time series. If **None**, it assumes that the DataFrame only has a single time series. In our case, **LCLid** is the name of that column.
- **use_32_bit**: This parameter doesn't do anything functionally but makes the DataFrames much smaller in memory, sacrificing the precision of the floating-point numbers.

This method returns the DataFrame with the rolling features added and a list with column names of the newly added features.

Seasonal rolling window aggregations

Seasonal rolling window aggregations are very similar to rolling window aggregations, but instead of taking past *n* consecutive observations in the window, this takes a seasonal window, skipping a constant number of timesteps between each item in a window. The following diagram will make this clearer:

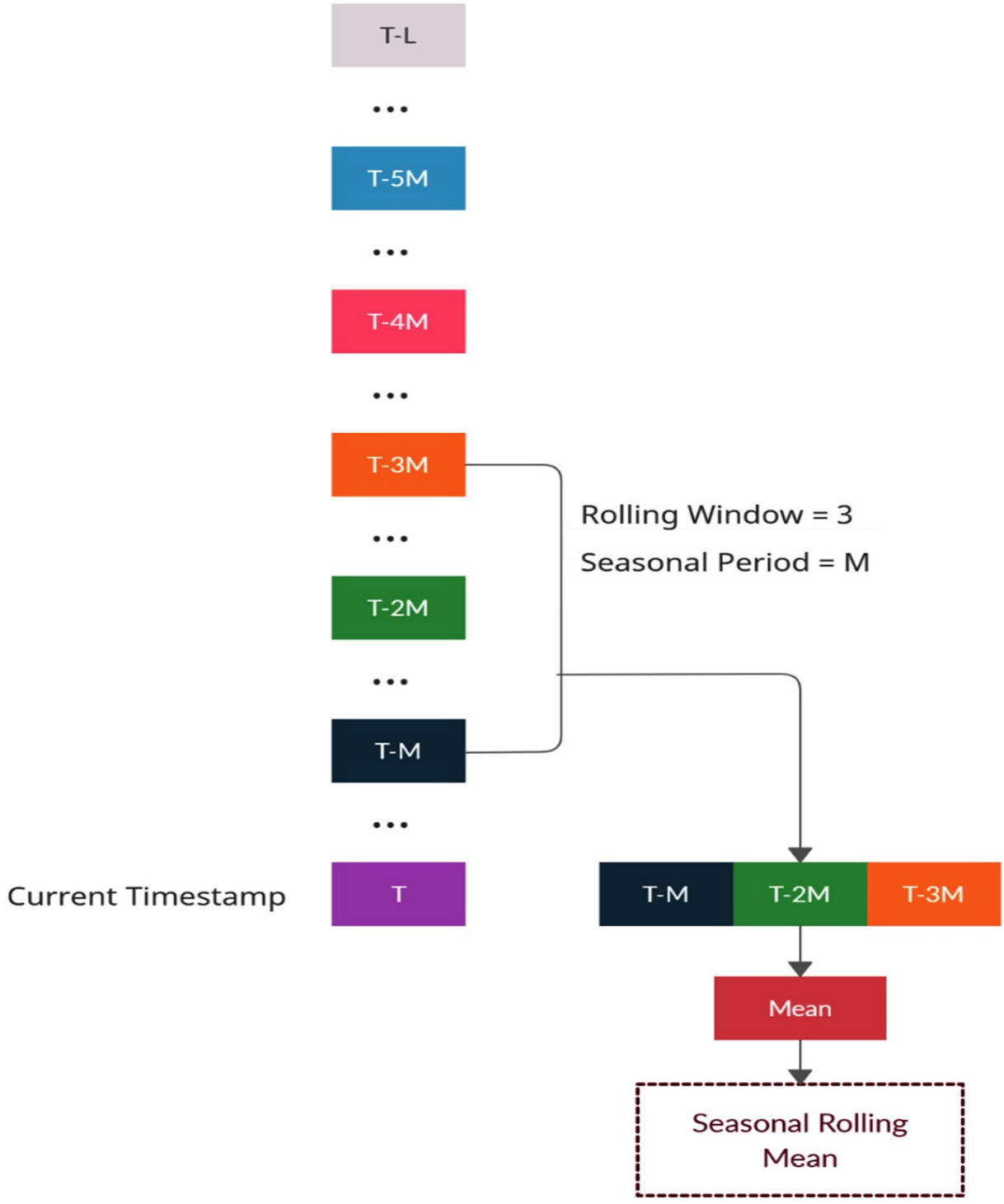


Figure 6.4 – Seasonal rolling window aggregations(Color figure)

The key parameter here is the seasonality period, which is commonly referred to as *m*. This is the number of timesteps after which we expect the seasonality pattern to repeat.

When we are at timestep *t*, a rolling window of three would have

$$y_{t-m}, y_{t-2m}, y_{t-3m}$$

as the vector of past observations. But the seasonal rolling window would skip *m* timesteps between each item in the window. This means that the observations that are there in the seasonal rolling window would be . And as usual, once we have the window vector, we just need to apply the aggregation function to get a scalar value and include that as a feature.

IMPORTANT NOTE

We are not including

as an element in the seasonal rolling window vector to avoid data leakage.

This is not an operation that you can do easily and efficiently with pandas. Some fancy NumPy indexing and Python loops should do the trick. We are using an implementation from github.com/jmoralez/window_ops/ that uses NumPy and Numba to make the operation fast and efficient.

Just like the features we saw earlier, we need to do this operation for each **LCLid** separately. We have included a helpful method in **src.feature_engineering.autoregressive_features** called **add_seasonal_rolling_features** that adds all the seasonal rolling features you want for each **LCLid** in a fast and efficient manner. Let's see how we can use that.

We are going to import the method and use a few parameters of the method to configure the seasonal rolling operation the way we want:

```
from src.feature_engineering.autoregressive_features impor
full_df, added_features = add_seasonal_rolling_features(
    full_df,
```

```
    rolls=[3],
    seasonal_periods=[48, 48 * 7],
    column="energy_consumption",
    agg_funcs=["mean", "std"],
    ts_id="LCLid",
    use_32_bit=True,
)
```

Now, let's look at the parameters that we used in the previous code snippet:

- **seasonal_periods**: This is a list of seasonal periods that should be used in the seasonal rolling windows. In the case of multiple seasonalities, we can include the seasonal rolling features of all the seasonalities.
- **rolls**: This parameter takes in a list of integers denoting all the windows over which we need to calculate aggregate statistics.
- **column**: The name of the column to be lagged. In our case, this is **energy_consumption**.
- **agg_funcs**: This is a list of aggregations that we want to do for each window we declared in **rolls**. The allowable aggregation functions are **{mean, std, max, min}**.
- **n_shift**: This is the number of seasonal timesteps we need to shift before doing the rolling operation. This parameter avoids data leakage.
- **ts_id**: The name of the column name that contains the unique ID of a time series. If **None**, it assumes the DataFrame only contains a single time series. In our case, **LCLid** is the name of that column.
- **Use_32_bit**: This parameter doesn't do anything functionally but makes the DataFrames much smaller in memory, sacrificing the precision of the floating-point numbers.

As always, the method returns the DataFrame with seasonal rolling features and a list containing the column names of the newly added features.

Exponentially weighted moving averages (EWMA)

With the rolling window mean operation, we were calculating the average of the window, and it works synonymously with the **moving average**. EWMA is the slightly smarter cousin of the moving average. While the moving average considers a rolling window and considers each item in the window equally on the computed average, EWMA tries to do a weighted average on the window, and the weights decay at an exponential rate. There is a parameter,

$$EWMA_t = \alpha \times y_t + (1 - \alpha) \times EWMA_{t-1}$$

, that determines how fast the weights decay. And because of this, we can consider all the history available as a window and let the parameter decide how much recency is included in EWMA. This can be written simply and recursively, as follows:

$$w\_\{t - k\} = \alpha \times (1 - \alpha)\^k$$

Here, we can see that the larger the value of , the more the average is skewed toward recent values (see *Figure 6.6* to get a visual intuition of how the weights would be). If we expand the recursion, the weights of each term work out to be: where *k* is the number of timesteps behind *t*. If we plot the weights, we can see them in an exponential decay; determines how fast the decay happens. Another way to think about is in terms of **span**. Span is the number of periods at which the decayed weights approach zero (not in a strictly mathematical way, but intuitively). and span are related through this equation: This will become clearer in the following diagram, where we have plotted how the weights decay for different values of :

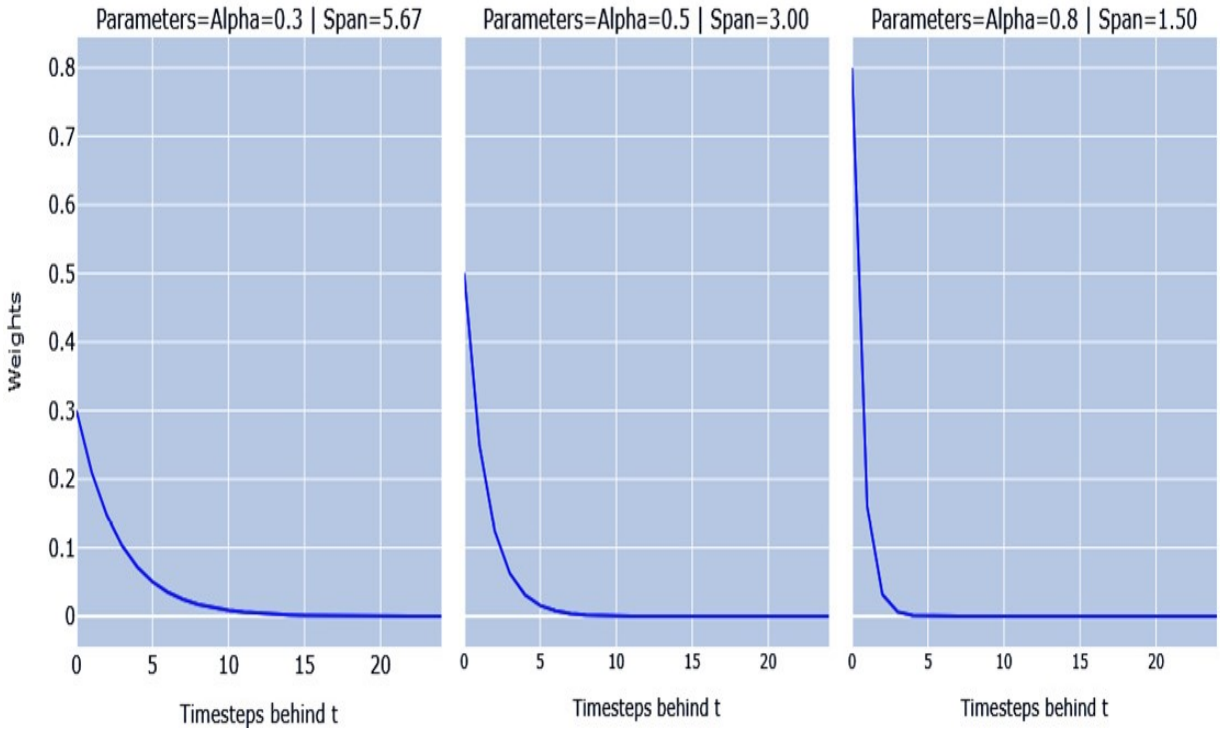


Figure 6.5 – Exponential weight decay for different values of

Here, we can see that the weight becomes small by the time we reach the span.

Intuitively, we can think of EWMA as an average of the entire history of the time series, but with parameters such as and **span**, we can make different periods of history more representative of the average. If we define a 60-period span, we can think that the last 60 time periods are what majorly drive the average. So, making EWMAs with different spans or s gives us representative features that capture different periods of history.

The overall process is depicted in the following diagram:

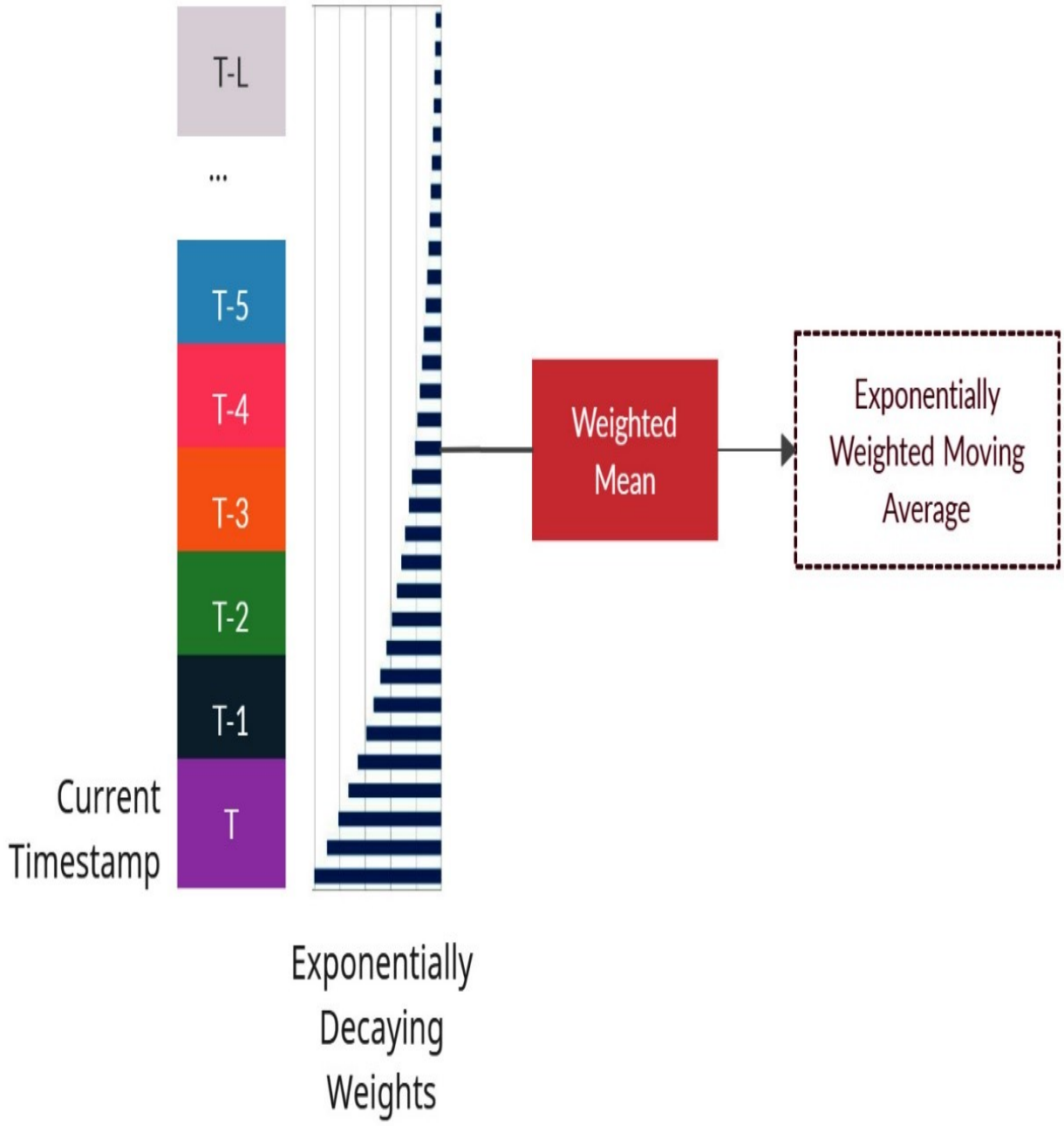


Figure 6.6 – EWMA features(Color figure)

Now, let's see how we can do this in pandas:

```
df["ewma"]=df['column'].shift(1).ewm(alpha=0.5).mean()
```

Like the other features we discussed earlier, EWMA also needs to be done for each **LCLid** separately. We have included a helpful method in **src.feature_engineering.autoregressive_features** called **add_ewma** that adds all the EWMA features you want for each **LCLid** in a fast and efficient manner. Let's see how we can use that.

We are going to import the method and use a few parameters of the method to configure EWMA the way we want to:

```
from src.feature_engineering.autoregressive_features impor
full_df, added_features = add_ewma(
    full_df,
    spans=[48 * 60, 48 * 7, 48],
    column="energy_consumption",
    ts_id="LCLid",
    use_32_bit=True,
)
```

Now, let's look at the parameters that we used in the previous code snippet:

- **alphas**: This is a list of all
  s we need to calculate the EWMA features for.
- **spans**: Alternatively, we can use this to list all the spans we need to calculate the EWMA features for. If you use this feature, **alphas** will be ignored.
- **column**: The name of the column to be lagged. In our case, this is **energy_consumption**.
- **n_shift**: This is the number of seasonal timesteps we need to shift before doing the rolling operation. This parameter avoids data leakage.
- **ts_id**: The name of the column name that has a unique ID for a time series. If **None**, it assumes the DataFrame only contains a single time series. In our case, **LCLid** is the

name of that column.
- **use_32_bit**: This parameter doesn't do anything functionally but makes the DataFrames much smaller in memory, sacrificing the precision of the floating-point numbers.

As always, the method returns the DataFrame containing EWMA features and a list with the column names of the newly added features.

These are a few standard ways of including time delay embedding in your ML model, but you are not restricted to just these. As always, feature engineering is a space that is not bound by rules and we can get as creative as we want and inject domain knowledge into the model. Apart from the features we have seen, we can include the difference in lag as custom lags that inject domain knowledge, and so on.

Now, let's look at the other class of features we can add via **temporal embedding**.

Temporal embedding

In Chapter 5, *Time Series Forecasting as Regression*, we briefly talked about temporal embedding as a process where we try to embed *time* into features that the ML model can leverage. If we think about *time* for a second, we will realize that there are two aspects of time that are important to us in the context of time series forecasting – *passage of time* and *periodicity of time*.

Let's look at a few features that can help us capture these aspects in an ML model.

Calendar features

The first set of features that we can extract are features based on calendars. Although the strict definition of time series is a set of observations taken sequentially in time, more often than not, we will have the timestamps of these collected observations alongside the

time series. We can utilize these timestamps and extract calendar features such as the month, quarter, day of the year, hour, minutes, and so on. These features capture the periodicity of time and help the ML model capture seasonality well. Only the calendar features that are temporally higher than the frequency of the time series make sense. For instance, an hour feature in a time series with a weekly frequency doesn't make sense, but a month feature and week feature make sense. We can utilize inbuilt datetime functionalities in pandas to create these features and treat these as categorical features in the model.

Time elapsed

This is another feature that captures the passage of time in an ML model. This feature increases monotonically as time increases, giving the ML model a sense of the passage of time. There are many ways to create this feature, but one of the easiest and most efficient ways is to use the integer representation of dates in NumPy:

```
df['time_elapsed'] = df['timestamp'].values.astype(np.int6
```

We have included a helpful method in **src.feature_engineering.temporal_features** called **add_temporal_features** that adds all relevant temporal features in an automated way. Let's see how we can use it.

We are going to import the method and use a few parameters of this method to configure and create the temporal features:

```
full_df, added_features = add_temporal_features(
    full_df,
    field_name="timestamp",
    frequency="30min",
    add_elapsed=True,
```

```
    drop=False,
    use_32_bit=True,
)
```

Now, let's look at the parameters that we used in the previous code snippet:

- **field_name**: This is the column name that contains the datetime that should be used to create features.
- **frequency**: We should provide the frequency of the time series as input so that the method automatically extracts the relevant features. These are standard pandas frequency strings.
- **add_elapsed**: This flag turns the creation of the time elapsed feature on or off.
- **use_32_bit**: This parameter doesn't do anything functionally but makes the DataFrames much smaller in memory, sacrificing the precision of the floating-point numbers.

Just like the previous methods we discussed, this also returns the new DataFrame with the temporal features added and a list containing the column names of the newly added features.

Fourier terms

Previously, we extracted a few calendar features such as the month, year, and so on and talked about using them as categorical variables in the ML model. Another way we can represent the same information, but on a continuous scale, is by using **Fourier terms**. We discussed Fourier series in Chapter 3, *Analyzing and Visualizing Time Series Data*. Just to recall, the sine-cosine form of the Fourier series is as follows:

Here, is the *N*-term approximation of the signal, *S*. Theoretically, when *N* is infinite, the resulting approximation is equal to the original signal. *P* is the maximum length of the cycle. We can create these cosine and sine functions as features to represent the seasonal cycle. If we are encoding the month, we know that the month goes from 1 to 12 and then

repeats itself. So, *P*, in this case, will be 12 and *x* will be 1, 2, …12. Therefore, for each *x*, we can calculate the cosine and sine terms and add them as features to the ML model. The following diagram shows the difference in representations between the month on an ordinal scale and as a Fourier series:

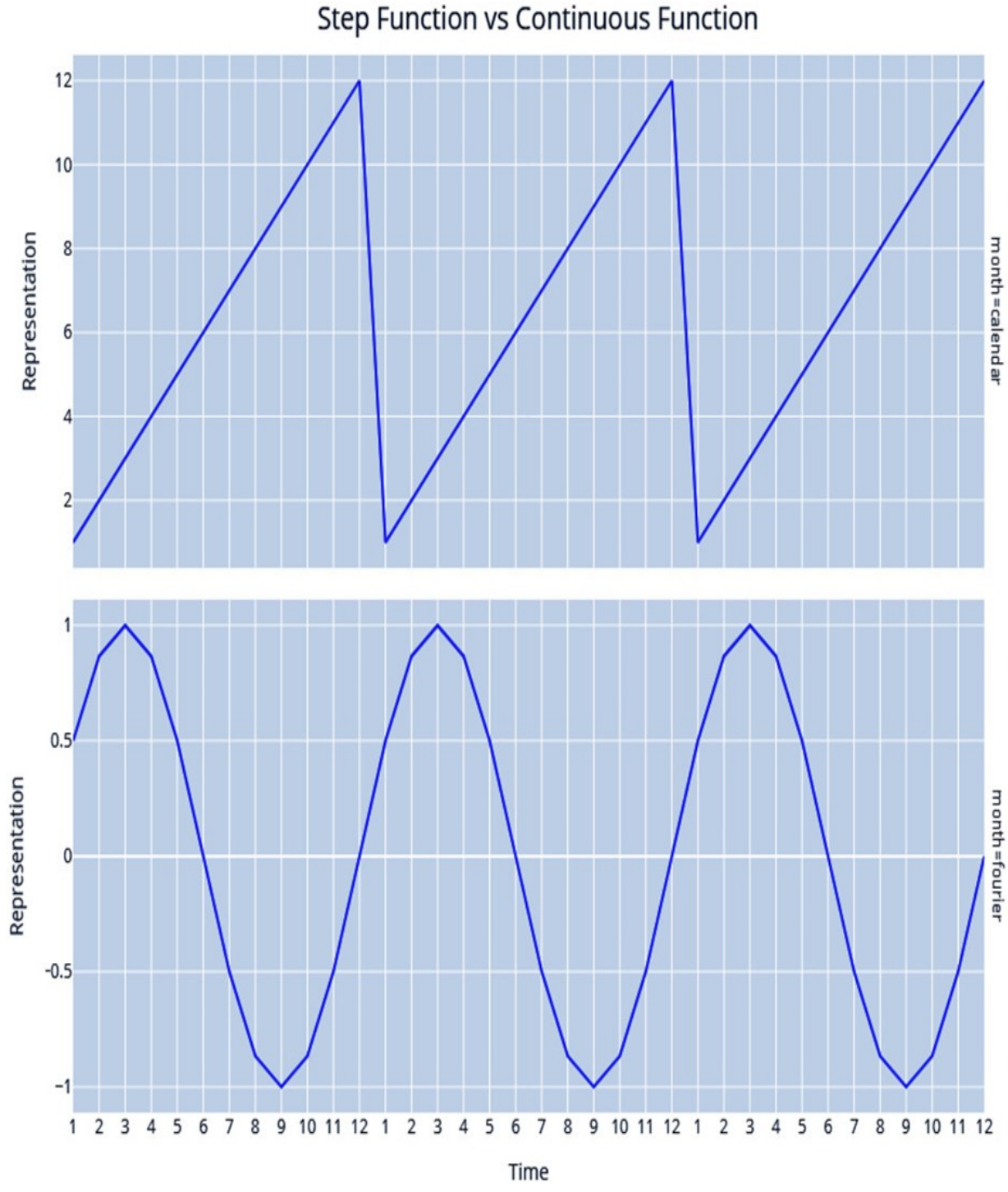


Figure 6.7 – Month as an ordinal step function (top) versus Fourier terms (bottom)

The preceding diagram shows just a single Fourier term; we can add multiple Fourier terms to help capture complex seasonality.

We cannot say that continuous representation of seasonality is better than categorical because it depends on the type of model you are using and the dataset. This is something we will have to find out empirically.

To make the process of adding Fourier features easy, we have made some easy-to-use methods available in **src.feature_engineering.temporal_features** in a file called **bulk_add_fourier_features** that adds Fourier features for all the calendar features we want in an automated way. Let's see how we can use that.

We are going to import the method and use a few parameters of the method to configure and create the Fourier series-based features:

```
full_df, added_features = bulk_add_fourier_features(
    full_df,
    ["timestamp_Month", "timestamp_Hour", "timestamp_Minut
    max_values=[12, 24, 60],
    n_fourier_terms=5,
    use_32_bit=True,
)
```

Now, let's look at the parameters that we used in the previous code snippet:

- **columns_to_encode**: This is the list of calendar features we need to encode using Fourier terms.
- **max_values**: This is a list of max values for the seasonal cycle for the calendar features in the same order as they are given in **columns_to_encode**. For instance, for **month** as a column to encode, we give **12** as the corresponding **max_value**. If not

given, **max_value** will be inferred. This is only recommended if the data you have contains at least a single complete seasonal cycle.

- **n_fourier_terms**: The number of Fourier terms to be added. This is synonymous to *n* in the equation for the Fourier series mentioned previously.
- **use_32_bit**: This parameter doesn't do anything functionally but makes the DataFrames much smaller in memory, sacrificing the precision of the floating-point numbers.

Just like the previous methods we've discussed, this also returns a new DataFrame with the Fourier features added and a list with column names of the newly added features.

After executing the **01-Feature Engineering.ipynb** notebook in **chapter06**, we will have the following feature engineered files written to disk:

- `selected_blocks_train_missing_imputed_feature_engg.parquet`
- `selected_blocks_val_missing_imputed_feature_engg.parquet`
- `selected_blocks_test_missing_imputed_feature_engg.parquet`

In this section, we looked at a few popular and effective ways of generating features for time series. But there are many more and depending on your problem and the domain, many of them will be relevant.

ADDITIONAL INFORMATION

> *The world of feature engineering is vast and there are a few open-source libraries that make exploring that space easier. A few of them include https://github.com/Nixtla/tsfeatures, https://tsfresh.readthedocs.io/en/latest/, and https://github.com/DynamicsAndNeuralSystems/catch22. A preprint by Ben D. Fulcher titled Feature-based time-series analysis at https://arxiv.org/abs/1709.08055 also gives a nice summary of the space.*

A newer library called functime(https://github.com/functime-org/functime) also provides fast feature engineering routines written in Polars and is worth checking out. A lot of the feature engineering discussed in the book can be made even faster using functime and Polars.

Summary

After a brief overview of the ML for time series forecasting paradigm in the previous chapter, in this chapter, we looked at this practically and saw how we can prepare the dataset with the required features to start using these models. We reviewed a few time series-specific feature engineering techniques such as lags, rolling, and seasonal features. All the techniques we learned in this chapter are tools with which we can quickly iterate through experiments to find out what works for our dataset. However, we only talked about feature engineering, which affects one side of the standard regression equation ( ). The other side, which is the target ( ) we are predicting, is also equally important. In the next chapter, we'll look at a few concepts such as stationarity and some transformations that affect the target.